PENGEMBANGAN *ACADEMIC CHATBOT HYBRID* BERBASIS WEB MENGGUNAKAN *FRAMEWORK* NLU-RASA DAN LLM-RAG DENGAN METODOLOGI ICONIX *PROCESS*

(Studi Kasus: FSM UNDIP)

SKRIPSI

Disusun Sebagai Salah Satu Syarat
untuk Memperoleh Gelar Sarjana Komputer
pada Program Studi Informatika

Disusun oleh:
Muthia Zhafira Sahnah
24060122130071

DEPARTEMEN INFORMATIKA
FAKULTAS SAINS DAN MATEMATIKA
UNIVERSITAS DIPONEGORO
2026

HALAMAN PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan di bawah ini :

Nama : Muthia Zhafira Sahnah

NIM : 24060122130071

Judul : Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*

Dengan ini saya menyatakan bahwa dalam skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan di dalam daftar pustaka.

Semarang, 30 Januari 2026



Muthia Zhafira Sahnah

24060122130071

HALAMAN PENGESAHAN

Judul : Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*

Nama : Muthia Zhafira Sahnah

NIM : 24060122130071

Telah diujikan pada sidang tugas akhir dan dinyatakan lulus pada tanggal 29 Januari 2026

Semarang, 30 Januari 2026

Pembimbing 1,	Pembimbing 2,
Dr. Indra Waspada, S.T., M.TI.	Adhe Setya Pramayoga, M.T.
NIP. 197902122008121002	NIP. 199112092024061001

Mengetahui,

Ketua Departemen Informatika	Ketua Panitia Penguji Tugas Akhir
Dr. Aris Sugiharto, S.Si., M.Kom.	Beta Noranita, S.Si., M.Kom.
NIP. 197108111997021004	NIP. 197308291998022001

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan skripsi ini dengan judul "Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*".

Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer di Program Studi Informatika, Fakultas Sains dan Matematika, Universitas Diponegoro. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Prof. Dr. Kusworo Adi, S.Si., M.T., selaku Dekan Fakultas Sains dan Matematika Universitas Diponegoro.
2. Bapak Dr. Aris Sugiharto, S.Si., M.Kom., selaku Ketua Departemen Informatika Fakultas Sains dan Matematika Universitas Diponegoro.
3. Bapak Dr. Indra Waspada, S.T., M.TI., selaku Dosen Pembimbing I yang telah memberikan bimbingan, ilmu, kesabaran, serta arahan yang sangat berharga bagi penulis dalam menyelesaikan penelitian ini.
4. Bapak Adhe Setya Pramayoga, M.T., selaku Dosen Pembimbing II yang telah memberikan bimbingan, saran, serta motivasi yang besar kepada penulis selama penyusunan skripsi.
5. Seluruh staf kependidikan dan staf administrasi di lingkungan Fakultas Sains dan Matematika Universitas Diponegoro, yang telah banyak membantu memberikan data informasi serta kemudahan dalam proses birokrasi penelitian ini.
6. Kedua orang tua tercinta, adik-adik tersayang, serta sahabat-sahabat penulis yang senantiasa memberikan doa, dukungan moral maupun material, serta motivasi yang tiada hentinya sehingga penulis dapat tetap semangat dalam menyelesaikan skripsi ini.

Semarang, 30 Januari 2026



Penulis

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Universitas Diponegoro, saya yang bertanda tangan di bawah ini:

Nama	:	Muthia Zhafira Sahnah
NIM	:	24060122130071
Program Studi	:	Informatika
Departemen	:	Informatika
Fakultas	:	Sains dan Matematika
Jenis Karya	:	Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan **Hak Bebas Royalti Noneksklusif (Non-exclusive Royalty Free Right)** kepada Universitas Diponegoro atas karya ilmiah saya yang berjudul:

Pengembangan Academic Chatbot Hybrid Berbasis Web Menggunakan Framework NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX Process

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Non-eksklusif ini Universitas Diponegoro berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan mempublikasikan skripsi saya tanpa meminta izin dari saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Semarang, 30 Januari 2026

Materai
Rp. 10.000,-

Muthia Zhafira Sahnah

24060122130071

ABSTRAK

Fakultas Sains dan Matematika (FSM) Universitas Diponegoro (UNDIP) menghadapi tantangan dalam memenuhi ekspektasi mahasiswa terhadap layanan informasi akademik yang responsif dan mudah diakses. Layanan informasi merupakan kebutuhan krusial, namun metode interaksi konvensional seringkali terbatas oleh waktu operasional dan kurang fleksibel dalam menjawab variasi pertanyaan mahasiswa. Penelitian ini bertujuan untuk merancang dan mengembangkan sistem chatbot layanan akademik berbasis web dengan pendekatan hybrid yang menggabungkan kerangka kerja Rasa sebagai *Natural Language Understanding* (NLU) untuk manajemen dialog terstruktur, dan metode *Retrieval-Augmented Generation* (RAG) berbasis *Large Language Model* (LLM) untuk menangani pertanyaan kontekstual berdasarkan dokumen SOP. Metodologi pengembangan yang digunakan adalah ICONIX *Process*, yang mencakup tahapan analisis kebutuhan, pemodelan *use case, robustness analysis, sequence diagram,* dan *class diagram*. Hasil implementasi berupa aplikasi web yang dibangun menggunakan *Framework* Next.js dan FastAPI. Berdasarkan hasil pengujian terhadap responden mahasiswa, sistem sudah memenuhi kebutuhan fungsional. Hal ini menunjukkan bahwa *chatbot* hibrida yang dikembangkan mampu meningkatkan interaktivitas layanan dan dapat diterima dengan baik oleh pengguna sebagai solusi penyediaan informasi akademik yang interaktif.

Kata kunci : *Chatbot, Hybrid* , Rasa , *Retrieval-Augmented Generation*, *Large Language Model*, Web, ICONIX

ABSTRACT

The Faculty of Science and Mathematics (FSM) at Diponegoro University (UNDIP) faces challenges in meeting student expectations for responsive and accessible academic information services. Information service is a crucial need; however, conventional interaction methods are often limited by operational hours and lack flexibility in addressing the variety of student inquiries. This study aims to design and develop a web-based academic service chatbot using a hybrid approach that integrates the Rasa *Framework* as a Natural Language Understanding (NLU) system for structured dialogue management, and the Retrieval-Augmented Generation (RAG) method based on Large Language Models (LLM) to handle contextual inquiries based on SOP documents. The development methodology employed is the ICONIX Process, which includes requirement analysis, use case modeling, robustness analysis, sequence diagrams, and class diagrams. The implementation results in a web application developed using the Next.js and FastAPI *Framework*. Based on testing results from student respondents, the system has met functional requirements. These results indicate that the developed hybrid chatbot enhances service interactivity and is well-accepted by users.

Keywords *:* Chatbot, Hybrid, Rasa, Retrieval-Augmented Generation, Large Language Model, Web, ICONIX

DAFTAR ISI

HALAMAN PERNYATAAN KEASLIAN SKRIPSI ii
HALAMAN PENGESAHAN iii
KATA PENGANTAR iv
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI v
ABSTRAK vi
ABSTRACT vii
DAFTAR ISI viii
DAFTAR TABEL xii
DAFTAR GAMBAR xiii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan dan Manfaat 3
1.3.1 Tujuan Penelitian 3
1.3.2 Manfaat Penelitian 3
1.4 Ruang Lingkup 4
1.5 Sistematika Penulisan 4
BAB II TINJAUAN PUSTAKA 6
2.1 Penelitian Terdahulu 6
2.2 Dasar Teori 8
2.2.1 Teori Terkait Sistem *Chatbot* 8
2.2.1.1 Sistem *Chatbot* 9
2.2.1.2 Rasa *Framework* 11
2.2.1.3 *Retrieval-Augmented Generation* (RAG) 12
2.2.1.4 *Large Language Model* (LLM) 14
2.2.1.5 Pendekatan *Hybrid* pada Rasa dan RAG 15
2.2.2 ICONIX *Process* 15
2.2.2.1 Tahap *Requirements* 16
2.2.2.2 *Milestone* 1: *Requirements Review* 18
2.2.2.3 Analysis/*Preliminary Design* 18
2.2.2.4 *Robustness* Diagram 19

2.2.2.5 *Milestone* 2 : *preliminary Design Review* 21
2.2.2.6 *Detailed Design* 21
2.2.2.7 *Milestone* 3 :*Critical Design Review* 22
2.2.2.8 Tahap *Implementation* 23
2.2.3 Teori UML 24
2.2.3.1 *Use case Diagram* 24
2.2.3.2 *Sequence Diagram* 25
2.2.3.3 *Class Diagram* 26
2.2.4 *Framework* dan Teknologi 28
2.2.4.1 Web Server dan Protokol Hypertext Transfer (HTTP) 28
2.2.4.2 FAST API 29
2.2.4.3 Next JS 29
2.2.4.4 Postgre SQL 30
2.2.5 *Software Testing* 30
2.2.6 Metode Kuisioner 32
BAB III METODE PENELITIAN 33
3.1 Tinjauan Pustaka dan Analisis Aplikasi Sejenis 33
3.2 Tahap *Requirements* 34
3.3 *Milestone* 1: *Requirements Review* 35
3.4 Tahap Analysis/*Preliminary Design* 35
3.5 *Milestone* 2: *Preliminary Design Review* 36
3.6 Tahap *Detailed Design* 36
3.7 *Milestone* 3: *Critical Design Review* 36
3.8 Tahap *Implementation* 37
BAB IV HASIL DAN PEMBAHASAN 39
4.1 Analisis Aplikasi Sejenis 39
4.2 Survei Kebutuhan Pengguna 40
4.3 Tahap *Requirements* 44
4.3.1 *Domain Modeling* 45
4.3.2 Prototipe Antarmuka 46
4.3.2.1 Desain Antarmuka Halaman *Login* (DA-CB-01) 46
4.3.2.2 Desain Antarmuka Halaman Percakapan (DA-CB-02) 46
4.3.2.3 Desain Antarmuka Halaman *Dashboard* Admin (DA-CB-03) 47

4.3.2.4 Desain Antarmuka Halaman Menu *Chat*logs (DA-CB-04) 47
4.3.2.5 Desain Antarmuka Halaman Pengelolaan *User* (DA-CB-05)..... 48
4.3.3 Daftar *Use case* 49
4.3.4 *Use case* Diagram..... 49
4.3.5 *Use case Scenario* 51
4.4 *Milestone* 1: *Requirements Review*..... 60
4.5 Tahap Analysis/*Preliminary Design* 64
4.5.1 *Robustness Analysis* 64
4.5.1.1 *Robustness Diagram* Autentikasi Pengguna (RD-CB-01)..... 64
4.5.1.2 *Robustness Diagram* Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot* (RD-CB-02)..... 65
4.5.1.3 *Robustness Diagram* Melihat Riwayat Pesan (RD-CB-03)..... 66
4.5.1.4 *Robustness Diagram* Melihat *Dashboard* Analitik (RD-CB-04)..... 67
4.5.1.5 *Robustness Diagram* Melihat & Memfilter *Chat Logs* (RD-CB-05)..... 68
4.5.1.6 *Robustness Diagram* Melihat Daftar Pengguna (RD-CB-06)..... 69
4.5.1.7 *Robustness Diagram* Menambah Pengguna Baru (RD-CB-07)..... 70
4.5.1.8 *Robustness Diagram* Memperbarui Pengguna (RD-CB-08)..... 70
4.5.1.9 *Robustness Diagram* Menghapus Pengguna (RD-CB-09)..... 71
4.5.2 *Updated Domain Model* 71
4.6 *Milestone* 2: *Preliminary Design Review* 73
4.7 Tahap *Detailed Design* 76
4.7.1 *Sequence Diagram* 76
4.7.1.1 *zSequence Diagram* Autentikasi Pengguna (SD-CB-01)..... 76
4.7.1.2 *Sequence Diagram* Mengirim Pesan *Chatbot* dan Menerima Jawaban *Chatbot* (SD-CB-02)..... 77
4.7.1.3 *Sequence Diagram* Melihat Riwayat Pesan (SD-CB-03) 78
4.7.1.4 *Sequence Diagram* Melihat *Dashboard* Analitik (SD-CB-04)..... 79
4.7.1.5 *Sequence Diagram* Melihat & Memfilter *Chat Logs* (SD-CB-05) 80
4.7.1.6 *Sequence Diagram* Melihat Daftar Pengguna (SD-CB-06) 81
4.7.1.7 *Sequence Diagram* Menambah Pengguna Baru (SD-CB-07) 82
4.7.1.8 *Sequence Diagram* Memperbarui Pengguna (SD-CB-08)..... 83
4.7.1.9 *Sequence Diagram* Menghapus Pengguna (SD-CB-09) 84
4.7.2 *Class Diagram*..... 85

4.8 *Milestone* 3: *Critical Design Review* 87
4.9 Tahap *Implementation* 90
4.9.1 Arsitektur Sistem 91
4.9.2 Lingkungan Implementasi 92
4.9.3 Pemetaan Kelas 92
4.9.4 Implementasi *Source Code* 93
4.9.5 Implementasi *Database* 101
4.9.6 Implementasi Antarmuka 103
4.9.7 Lingkungan Pengujian 103
4.9.8 Rencana Pengujian 104
4.9.9 Pembahasan Hasil Pengujian 104
BAB V KESIMPULAN DAN SARAN 106
5.1 Kesimpulan 106
5.2 Saran 107
DAFTAR PUSTAKA 108
LAMPIRAN-LAMPIRAN 111

DAFTAR TABEL

Tabel 2.1 Studi Literatur 6
Tabel 2.2 Simbol *Robustness* Diagram (Rosenberg dkk., 2007) 20
Tabel 2.3 *Use case Diagaram (Rosenberg dkk., 2007)* 24
Tabel 2.4 Simbol *Class Diagram (Booch dkk., 2005)* 27
Tabel 4.1 Analisis Aplikasi Sejenis 39
Tabel 4.2 *Software Requirements Spesification* (SRS) *Chatbot* FSM 44
Tabel 4.3 Daftar *Use case* 49
Tabel 4.4 *Use case Scenario* Autentikasi Pengguna 51
Tabel 4.5 *Use case Scenario* Mengirim Pesan *Chatbot* 52
Tabel 4.6 *Use case Scenario* Melihat Riwayat Pesan 53
Tabel 4.7 *Use case Scenario* Melihat *Dashboard* Analitik 54
Tabel 4.8 *Use case Scenario* Memfilter *Chat Logs* 55
Tabel 4.9 *Use case Scenario* Melihat Daftar Pengguna 56
Tabel 4.10 *Use case Scenario* Menambah Pengguna Baru 57
Tabel 4.11 *Use case Scenario* Memperbarui Pengguna 58
Tabel 4.12 *Use case Scenario* Menghapus Pengguna 59
Tabel 4.13 *Cross-artifact-Checking* Tahap *Requirements Review* 62
Tabel 4.14 *Cross-Artifact Checking* Tahap PDR 74
Tabel 4.15 Hasil *Cross-Artifact Checking* Tahap CDR 88
Tabel 4.16 Pemetaan Kelas 93
Tabel 4.17 Implementasi *Source Code* 94
Tabel 4.18 Percobaan Variasi *Threshold* pada Beberapa Skenario 95
Tabel 4.19 Implementasi Database *User* 101
Tabel 4.20 Implementasi Database *Conversations* 101
Tabel 4.21 Implementasi Database *Messages* 102
Tabel 4.22 Spesifikasi Perangkat Keras 103
Tabel 4.23 Spesifikasi Perangkat Lunak 103

DAFTAR GAMBAR

Gambar 2.1 *General Chatbot architecture* (Adamopoulou & Moussiades, 2020) 10
Gambar 2.2 Alur Proses Kerja Rasa (Kong & Wang, 2021)..... 12
Gambar 2.3 RAG *Framework (Rothman, 2024)* 14
Gambar 2.4 ICONIX *Process (Rosenberg dkk., 2007)* 16
Gambar 2.5 Tahap *Requirements (Rosenberg dkk., 2007)* 17
Gambar 2.6 *Preliminary Design (Rosenberg dkk., 2007)* 19
Gambar 2.7 *Detailed Design (Rosenberg dkk., 2007)* 22
Gambar 2.8 Critical Design *Review(Rosenberg dkk., 2007)* 23
Gambar 2.9 Simbol *Sequence Diagram* (Fowler, 2004) 26
Gambar 3. 1 Metodologi Penelitian..... 33
Gambar 4. 1 Diagram Sebaran Departemen Responeden 40
Gambar 4.2 Diagram Jawaban Kebiasaan Akses Mahasiswa terhadap Informasi Akademik di FSM Undip 41
Gambar 4. 3 Skala Likert Seberapa Sulit mecari informasi akademik..... 42
Gambar 4.4 Diagram Kendala Utama Saat Mencari Informasi Akademik..... 42
Gambar 4.5 Diagram Survey Harapan Infromasi yang terrsedi pada *Chatbot FSM*..... 43
Gambar 4. 6 Domain Model..... 45
Gambar 4. 7 Desain Antarmuka Halaman Login 46
Gambar 4. 8 Desain Antarmuka Halaman Percakapan 47
Gambar 4. 9 Desain Antarmuka Halaman Dashboard Admin 47
Gambar 4. 10 Desain Antarmuka Halaman Menu Chatlogs 48
Gambar 4.11 Desain Antarmuka Halaman Pengelolaan *User* 48
Gambar 4. 12 *Use Case* Diagram..... 50
Gambar 4.13 Robustness Diagram Autentikasi Pengguna (RD-CB-01)..... 65
Gambar 4.14 Robustness Diagram Proses Mengirim Pesan Chatbot dan Menerima Jawaban dari Chatbot (RD-CB-02) 66
Gambar 4.15 *Robustness* Diagram Melihat Riwayat Pesan (RD-CB-03)..... 67
Gambar 4.16 *Robustness* Diagram Melihat *Dashboard* Analitik (RD-CB-04)..... 68
Gambar 4.17 *Robustness* Diagram Melihat & Memfilter *Chat Logs* (RD-CB-05)..... 69
Gambar 4.18 *Robustness* Diagram Melihat Daftar Pengguna (RD-CB-06)..... 69
Gambar 4.19 Robustness Diagram Menambah Pengguna Baru (RD-CB-07) 70

Gambar 4.20 *Robustness* Diagram Memperbarui Pengguna (RD-CB-08)..... 71
Gambar 4.21 *Robustness* Diagram Menghapus Pengguna (RD-CB-09)..... 71
Gambar 4.22 *Updated Domain Model* 72
Gambar 4.23 Sequence Diagram Autentikasi Pengguna (SD-CB-01)..... 77
Gambar 4.24 Sequence Diagram Mengirim Pesan Chatbot dan Menerima Jawaban dari Chatbot (SD-CB-02)..... 78
Gambar 4.25 Sequence Diagram Melihat Riwayat Pesan (SD-CB-03)..... 79
Gambar 4.26 Sequence Diagram Melihat Dashboard Analitik (SD-CB-04) 80
Gambar 4.27 Sequence Diagram Melihat & Memfilter Chat Logs (SD-CB-05)..... 81
Gambar 4.28 Sequence Diagram Melihat Daftar Pengguna (SD-CB-06)..... 82
Gambar 4.29 Sequence Diagram Menambah Pengguna Baru (SD-CB-07)..... 83
Gambar 4.30 Sequence Diagram Memperbarui Pengguna (SD-CB-08)..... 84
Gambar 4.31 *Sequence Diagram* Menghapus Pengguna (SD-CB-09) 85
Gambar 4.32 Class Diagram..... 87
Gambar 4.33 *Hybrid* Rasa dan RAG..... 91
Gambar 4.34 Arsitektur Sistem *Chatbot Hybrid* Berbasis Rasa dan RAG 92

BAB I
PENDAHULUAN

Bagian pendahuluan ini menguraikan landasan penyusunan skripsi dengan memaparkan latar belakang masalah, rumusan masalah, tujuan, manfaat, ruang lingkup kajian, serta sistematika penulisan dari penelitian Tugas Akhir (TA) mengenai Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU- Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*.

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi (TIK) mendorong transformasi digital dalam berbagai aspek pendidikan tinggi (Farid Faqih dkk., 2025). Perguruan tinggi kini dituntut menyediakan layanan akademik yang cepat, efisien, dan sesuai ekspektasi mahasiswa. Kualitas layanan bahkan menjadi faktor utama penentu kepuasan dan citra institusi, melampaui promosi maupun fasilitas fisik (Susetyo dkk., 2022). Akses informasi akademik yang akurat dan *respons*if menjadi kebutuhan penting (Hakim dkk., 2024; Notokoesoemo, 2023).

Permasalahan serupa juga teridentifikasi di Fakultas Sains dan Matematika Universitas Diponegoro (FSM UNDIP). Minimnya pemahaman mahasiswa terhadap Standar Operasional Prosedur (SOP) memicu banyaknya pertanyaan repetitif yang ditujukan kepada staf akademik. Berdasarkan hasil survei mengenai kendala akses informasi, kondisi ini berdampak pada penumpukan antrean layanan serta waktu tunggu yang cukup lama. Oleh karena itu, diperlukan media interaktif yang fleksibel dan dapat diakses sewaktu-waktu guna mendukung kelancaran proses studi mahasiswa.

Sebagai alternatif solusi, penyediaan halaman *Frequently Asked Questions* (FAQ) pada situs web resmi seringkali menjadi opsi. Namun, sifatnya yang statis dan minim interaksi dinilai kurang optimal dalam merespons variasi pertanyaan mahasiswa secara instan. Sebaliknya, berbagai studi menegaskan efektivitas *chatbot* sebagai solusi yang lebih unggul karena mampu menyajikan informasi secara *real-time*, cepat, akurat, dan mudah diakses, serta berkontribusi dalam memberikan layanan informasi akademik yang interaktif (Adamopoulou & Moussiades, 2020; Hidayat dkk., 2025; Ichsanudin dkk., 2024).

Meskipun *chatbot* secara efektif dapat meningkatkan aksesibilitas informasi akademik, terdapat tantangan keilmuan dalam menentukan arsitektur yang optimal untuk menangani variasi pertanyaan yang kompleks namun tetap akurat. Tren penelitian menunjukkan dua pendekatan yang banyak digunakan, yaitu *intent-based* seperti Rasa dan generatif seperti *Retrieval-Augmented Generation* (RAG). Rasa memiliki keunggulan manajemen dialog terstruktur, akurasi tinggi untuk domain berpola tetap, serta fleksibilitas bersifat *open source* (Anindyati, 2023; Ruindungan & Jacobus, 2021). Namun, pendekatan ini sangat bergantung pada daftar *intent* dan variasi ungkapan yang harus terus diperbarui. Di sisi lain, RAG mampu memberikan jawaban diamis dengan mengambil konteks dari dokumen eksternal serta mengurangi risiko halusinasi pada LLM (Elysia & Herianto, 2024; Hidayat dkk., 2025; Husain dkk., 2025). Meskipun begitu, penggunaan RAG membutuhkan sumber daya komputasi lebih besar dan berpotensi menimbulkan latensi maupun biaya tambahan ketika volume permintaan meningkat. Ketidaksempurnaan masing-masing pendekatan ini menunjukkan perlunya solusi arsitektur yang mampu menyeimbangkan akurasi, fleksibilitas, dan efisiensi sumber daya.

Penelitian ini mengusulkan arsitektur *chatbot hybrid* yang menggabungkan keunggulan Rasa dalam menangani pertanyaan berpola tetap dengan kemampuan RAG menjawab pertanyaan kompleks berbasis dokumen, sehingga menghasilkan sistem yang lebih akurat, fleksibel, dan efisien. Pengembangan arsitektur *hybrid* yang menggabungkan penanganan *intent* terstruktur (Rasa) dan penelusuran dokumen dinamis (RAG) memiliki kebutuhan dokumentasi yang lengkap untuk mengetahi bagaimana pendekatan alur logika pada sistem ini, sehingga pemilihan metode pengembangan menjadi krusial. Oleh karena itu, penelitian ini mengadopsi ICONIX *Process* yang menawarkan keseimbangan antara analisis kebutuhan dan desain arsitektur secara terstruktur (Yulianta & Mursanto, 2008). Keunggulan utamanya terletak pada tahapan *Robustness Analysis* yang menjembatani kebutuhan pengguna dan desain teknis (Rosenberg dkk., 2005), sekaligus memberikan *traceability* yang tinggi dari kebutuhan hingga implementasi. Dengan demikian, ICONIX *Process* dipilih sebagai strategi mitigasi risiko untuk memastikan *chatbot hybrid* yang dikembangkan akurat, efisien, dan selaras dengan SOP akademik FSM UNDIP.

1.2 Rumusan Masalah

Berdasarkan latar belakang dan permasalahan yang telah dibahas, maka rumusan masalah dalam penelitian ini adalah bagaimana merancang dan membangun *Chatbot Hybrid* berbasis *web* menggunakan NLU-Rasa dan LLM-RAG dengan menerapkan metodologi ICONIX *Process* untuk mendukung layanan informasi akademik di Fakultas Sains dan Matematika Universitas Diponegoro.

1.3 Tujuan dan Manfaat

Penelitian ini disusun dengan arah target capaian yang jelas serta diharapkan dapat memberikan kontribusi nyata bagi lingkungan akademik. Bagian ini memaparkan mengenai tujuan utama yang ingin dicapai dari pengembangan sistem, serta manfaat praktis yang diharapkan dapat diperoleh oleh instansi terkait setelah sistem berhasil diimplementasikan.

1.3.1 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk merancang dan membangun *Chatbot Hybrid* berbasis *web* menggunakan NLU-Rasa dan LLM-RAG dengan menerapkan metodologi *ICONIX Process* untuk mendukung layanan informasi akademik di Fakultas Sains dan Matematika Universitas Diponegoro.

1.3.2 Manfaat Penelitian

Berdasarkan tujuan yang telah ditetapkan, hasil penelitian ini diharapkan dapat memberikan dampak positif terhadap optimalisasi layanan akademik. Adapun manfaat yang diharapkan dari pengembangan sistem *chatbot* ini difokuskan bagi institusi, yaitu Fakultas Sains dan Matematika Universitas Diponegoro, antara lain:

1. Menyediakan sistem *Chatbot* yang dapat meningkatkan efisiensi pelayanan informasi akademik dengan memberikan akses cepat kepada mahasiswa.
2. Otomatisasi jawaban atas pertanyaan yang sering diajukan mahasiswa.
3. Meningkatkan kepuasan mahasiswa karena informasi dapat diperoleh tanpa harus datang langsung ke kampus atau menunggu jam layanan.

1.4 Ruang Lingkup

Ruang lingkup dalam perancangan dan pembangunan *Chatbot Hybrid* berbasis Rasa dan LLM dengan menerapkan metodologi ICONIX *Process* untuk mendukung layanan informasi akademik di Fakultas Sains dan Matematika Universitas Diponegoro adalah sebagai berikut:

1. Sistem *chatbot* hanya mencakup layanan yang tercantum dalam tujuh SOP akademik, yaitu: SOP Permohonan Izin Aktif Setelah Cuti, SOP Pengisian IRS, SOP Legalisir Ijazah dan Transkrip, SOP Permohonan Izin Cuti Akademik, SOP Pengajuan Rekomendasi Beasiswa, SOP Pengajuan Proposal Kegiatan Organisasi Mahasiswa, dan SOP Permohonan Izin Keterlambatan Pembayaran UKT. Pemilihan ketujuh SOP ini didasarkan pada ketersediaan dokumen SOP di lingkungan FSM UNDIP yang telah mengalami pembaruan (*updated*) dan tervalidasi saat penelitian ini dilakukan.
2. Ruang lingkup tidak mencakup proses verifikasi administrasi, persetujuan berkas atau integrasi dengan sistem akademik fakultas. Evaluasi sistem difokuskan pada kemampuan sistem untuk memenuhi semua kebutuhan fungsional yang sudah ditentukan, tanpa membahas keamanan sistem.
3. Penelitian ini berfokus pada rancang bangun perangkat lunak dan arsitektur sistem menggunakan metodologi ICONIX *Process*. Evaluasi mendalam mengenai performa model (akurasi *intent* Rasa dan relevansi jawaban LLM) dilakukan pada penelitian terpisah.

1.5 Sistematika Penulisan

Untuk memberikan gambaran dari pembahasan-pembahasan pada masing-masing bab dari laporan tugas akhir ini, sistematika penulisan disusun sebagai berikut:

BAB I PENDAHULUAN

Bab ini berisi uraian mengenai latar belakang penelitian, perumusan masalah, tujuan, manfaat, ruang lingkup, serta sistematika penulisan. Bab ini bertujuan memberikan gambaran umum mengenai alasan penelitian dilakukan serta batasan masalah yang ditetapkan.

BAB II TINJAUAN PUSTAKA

Bab ini menyajikan landasan teori yang mendukung penelitian, antara lain teori tentang *Chatbot*, *Rasa Framework, Large Language Model (*LLM),

Retrieval-Augmented Generation (RAG), dan metodologi ICONIX *Process.* Selain itu, bab ini juga memuat kajian penelitian terdahulu yang relevan untuk menunjukkan posisi penelitian ini dibandingkan dengan penelitian sebelumnya.

BAB III METODE PENELITIAN

Bab ini menyajikan pembahasan mengenai metode penelitian yang digunakan oleh penulis dalam skripsi ini menggunakan ICONIX *Process*. Uraian meliputi tahap *requirements*, *Milestone* 1 : *requirements review*, tahap *analysis/preliminary design*, *Milestone* 2 : *preliminary design review*, tahap *detailed design*, *Milestone* 3 : *critical design review*, dan tahap *Implementation*

BAB IV IMPLEMENTASI DAN PENGUJIAN

Bab ini berisi implementasi sistem *Chatbot hybrid* berbasis web sesuai dengan rancangan yang telah dibuat. Selain itu, bab ini menjelaskan hasil pengujian sistem, termasuk pengujian terhadap fungsionalitas *Chatbot*, akurasi jawaban, serta kesesuaian dengan dokumen SOP FSM yang digunakan sebagai *knowledge base*.

BAB V KESIMPULAN DAN SARAN

Bab ini menyajikan mengenai kesimpulan dan saran berdasarkan dari hasil dan saran kegiatan penelitian dengan judul Pengembangan *Chatbot Hybrid* Berbasis Web Menggunakan Rasa dan LLM Dengan Metodologi ICONIX (Studi Kasus FSM UNDIP) yang telah dilakukan oleh penulis untuk pengembangan penelitian selanjutnya.

BAB II
TINJAUAN PUSTAKA

Bab ini menyajikan tinjauan pustaka yang mencakup tentang penelitian terdahulu dan konsep-konsep yang menjadi landasan dalam Pengembangan Academic Chatbot Hybrid Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX Process. Teori-teori yang dibahas meliputi pengertian *Chatbot*, ICONIX *Process,* Aplikasi berbasis web, dan teknologi yang digunakan.

2.1 Penelitian Terdahulu

Penelitian terdahulu ditinjau untuk memahami perkembangan riset terkait *chatbot* akademik, pemanfaatan Rasa dan RAG, serta penerapan meodologi ICONIX dalam pengembangan sistem. Studi literatur ini dilakukan untuk mengidentifikasi pendekatan, kelebihan, dan keterbatasan dari penelitian sebelumnya sehingga dapat menjadi dasar dalam merumuskan kebutuhan, arah pengembangan, serta kontribusi penelitian ini.

Tabel 2.1 Studi Literatur

No	Penulis	Judul Literatur	Hasil/Kesimpulan
1	(Anindyati, 2023)	Analisis dan Perancangan Aplikasi *Chatbot* Menggunakan *Framework* Rasa dan Sistem Informasi Pemeliharaan Aplikasi (Studi Kasus: *Chatbot* Penerimaan Mahasiswa Baru Politeknik Astra)	Penelitian ini menunjukkan Rasa memiliki keunggulan yang membuat interaksi lebih nyata dan mengatur dialog. Rasa bersifat open *source* sehingga memiliki adaptabilitas tinggi untuk modifikasi sesuai kebutuhan spesifik.
2	(Ruindungan & Jacobus, 2021)	*Chatbot* Development for an Interactive Academic Information Services using the Rasa Open *Source Framework*	Penelitian ini menunjukkan hasil evaluasi NLU memperlihatkan *chatbot* dapat memahami maksud pesan teks pengguna dengan baik, ditunjukkan dengan nilai rata-rata tertimbang untuk precision 0,995, recall 0,995, dan F1-Score 0,995 .

No	Penulis	Judul Literatur	Hasil/Kesimpulan
3	(Husain dkk., 2025)	Development of an Academic Services *Chatbot Based* on Retrieval-Augmented Generation (RAG)	Penelitian ini menunjukkan bahwa LLM standar memiliki risiko halusinasi, sehingga RAG *Framework* adalah solusi yang terbukti untuk menciptakan *chatbot* yang faktual dan terpercaya Sistem pada penelitian ini mencegah halusinasi dengan menolak menjawab 81,25% pertanyaan di luar konteks.
4	(Hidayat dkk., 2025)	Optimalisasi Layanan Sistem Informasi Mahasiswa dengan Integrasi Telegram: *Chatbot* Retrieval-Augmented-Generation berbasis Large Language Model	Penelitian ini menunjukkan bahwa *chatbot* berbasis RAG dan LLM untuk menjawab pertanyaan akademik yang informatif, akurat, dan ringkas dengan mencari jawaban relevan dari dokumen teks
5	(Yulianta & Mursanto, 2008)	Pengembangan Aplikasi Web dengan ICONIX *Process* dan UML Studi Kasus: Sistem Manajemen Isi	Penelitian ini menunjukkan kelebihan ICONIX *Process* yang dapat menawarkan keseimbangan dengan tahap *Robustness* Analysis berfungsi menjembatani celah analisis kebutuhan (what) dan desain teknis (how).
6	(Agustha dkk., 2024)	Pengembangan Aplikasi Monitoring Informasi Proyek Berbasis Web Menggunakan Metode Iconix *Process* (Studi Kasus: PT Telkom Akses Witel Semarang)	Penelitian ini menggunakan ICONIX *Process* dan mengatakan bahwa metodologi ini berperan penting dalam memastikan bahwa desain dan pengembangan aplikasi dilakukan secara terstruktur dan terdokumentasi dengan baik.

Berdasarkan studi literatur pada Tabel 2.1, dapat disimpulkan bahwa *chatbot* telah banyak digunakan sebagai solusi untuk meningkatkan interaksi layanan informasi akademik, baik melalui pendekatan *intent based* menggunakan Rasa maupun pendekatan generatif

berbasis RAG. Penelitian-penelitian yang memanfaatkan Rasa menunjukkan keunggulan pada pengelolaan dialog terstruktur dan akurasi identifikasi *intent* yang tinggi, namun masih terbatas pada pola pertanyaan tertentu. Sebaliknya, penelitian menggunakan RAG menunjukkan kemampuan *chatbot* dalam menghasilkan jawaban yang lebih dinamis dan faktual, namun memiliki tantangan pada kebutuhan komputasi yang besar dan risiko latensi.

Selain itu, dua penelitian mengenai ICONIX *Process* menunjukkan bahwa metodologi ini mampu memberikan proses pengembangan yang lebih terstruktur dengan dukungan *Robustness* Analysis dan dokumentasi yang kuat. Dari keseluruhan studi literatur, terlihat adanya celah penelitian berupa kurangnya *hybrid* kedua pendekatan *chatbot* tersebut dalam konteks layanan akademik, serta minimnya penerapan metodologi ICONIX pada pengembangan *chatbot hybrid*. Oleh karena itu, penelitian ini berkontribusi dengan mengembangkan *chatbot hybrid* Rasa dan RAG berbasis web menggunakan ICONIX *Process* sebagai metodologi perancangan yang terstruktur.

2.2 Dasar Teori

Pada bagian ini dijelaskan berbagai konsep, istilah serta landasan ilmiah yang menjadi dasar dalam penelitian ini. Dasar teori disusun untuk memberikan pemahaman menyeluruh mengenai teknologi dan metode yang digunakan, sehingga proses perancangan dan implementasi sistem dapat dipahami secara terstruktur. Pembahasan pada subbab ini meliputi teori terkait *chatbot* dan komponen pendukungnya seperti Rasa, RAG, dan *Large Language Model* (LLM), metodologi pengembangan perangkat lunak ICONIX *Process*, pemodelan menggunakan *Unified Modeling Language* (UML), teknologi yang digunakan dalam implementasi sistem, serta konsep pengujian sistem yang diterapkan untuk memastikan fungsionalitas berjalan sesuai kebutuhan.

2.2.1 Teori Terkait Sistem *Chatbot*

Subbab ini membahas konsep dasar yang berkaitan dengan sistem *chatbot* dalam penelitian ini. Pemahaman mengenai *chatbot* diperlukan untuk menjelaskan bagaimana sistem mampu menerima, memproses, dan menghasilkan *respons* terhadap pertanyaan pengguna. Selain itu, dijelaskan pula teknologi pendukung yang digunakan dalam pengembangan *chatbot hybrid*, yaitu Rasa sebagai *Framework intent-based*, *Retrieval-Augmented Generation* (RAG) sebagai pendekatan pencarian berbasis pengetahuan, serta

Large Language Model (LLM) yang berperan dalam menghasilkan *respons* yang lebih natural dan kontekstual. Dengan memahami teori-teori ini, proses desain dan implementasi *chatbot* dalam penelitian dapat dipahami secara komprehensif.

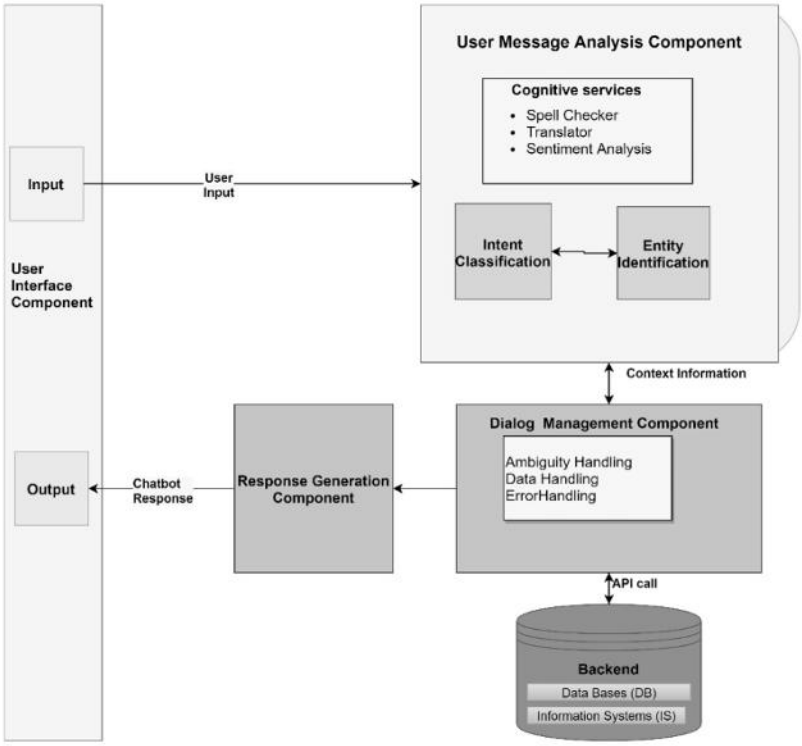
2.2.1.1 Sistem *Chatbot*

Chatbot merupakan program berbasis kecerdasan buatan yang dirancang untuk menyimulasikan percakapan manusia melalui teks atau suara (Adamopoulou & Moussiades, 2020). Sebagai bagian dari model interaksi manusia dan komputer (HCI), *chatbot* bekerja dengan menangkap pesan pengguna, memahami maksudnya dan menghasilkan *respons* otomatis yang relevan. Teknologi ini dapat memahami bahasa manusia dan dapat diintegrasikan ke berbagai platform seperti aplikasi *web*, perangkat *mobile*, maupun layanan pesan instan (Muhammad dkk., 2022).

Chatbot banyak digunakan pada sektor bisnis, pendidikan, layanan pelanggan dan kesehatan kerena mampu memberikan informasi secara cepat dan *respons*if. Fungsinya meliputi penyedia layanan informasi, perantara kepada pengguna, dan alat untuk mengumpulkan data pelanggan potensial dalam strategi pemasaran digital (Adamopoulou & Moussiades, 2020; Muhammad dkk., 2022).

Secara historis, pengembangan *chatbot* berakar pada gagasan mesin cerdas Alan Turing pada tahun 1950-an yang melahirkan tes turing. *Chatbot* modern bermula dari ELIZA (1966), diikuti PARRY (1972), Jabberwacky (1988), dan ALICE (1995) yang menggunakan AIML. Evolusi ini berlanjut hingga hadirnya asisten virtual seperti Siri, Watson, Google *Assistant*, dan Alexa (Adamopoulou & Moussiades, 2020).

Arsitektur umum *chatbot* terdiri atas beberapa komponen utama yang bekerja secara berurutan untuk memproses pesan pengguna dan menghasilkan *respons* yang sesuai. Secara garis besar, alur kerja *chatbot* mencakup empat komponen utama, yaitu *User Interface Component*, *User Message Analysis Component*, *Dialog Management Component*, dan *Response Generation Component.* Keempat komponen ini membentuk *pipeline* pemrosesan pesan yang bekerja secara berurutan mulai dari penerimaan *input* hingga pengiriman *respons*. Struktur lengkap alur tersebut dijelaskan pada Gambar 2.1, yang menggambarkan bagaimana pesan pengguna dianalisis, dipahami, dikelola, dan akhirnya dibalas oleh sistem *chatbot*.



Gambar 2.1 *General Chatbot architecture* (Adamopoulou & Moussiades, 2020)

Pada tahap awal, pesan pengguna diterima melalui komponen antarmuka dan diteruskan ke komponen analisis pesan. Pada bagian ini dilakukan pemrosesan bahasa alami, seperti pemeriksaan ejaan, penerjemahan, atau analisis sentimen, serta proses inti berupa *intent classification* dan *entity identification* untuk memahami maksud dan informasi yang relevan dari input pengguna. Hasil analisis ini kemudian digunakan oleh Dialog *Management Component* untuk menangani konteks percakapan, mengelola ambiguitas, mengambil data yang dibutuhkan, serta menangani error. Jika diperlukan, komponen ini dapat melakukan panggilan API ke sistem *back*end atau basis data.

Tahap akhir dilakukan oleh *Response Generation Component*, yaitu pembentukan *respons* berdasarkan keputusan yang dihasilkan oleh dialog *management*. Respons tersebut kemudian dikembalikan kepada pengguna melalui komponen *output*. Dengan arsitektur ini, *chatbot* dapat bekerja secara sistematis, mempertahankan konteks percakapan, serta memastikan *respons* yang relevan dan konsisten.

Dengan memahami arsitektur dasar dan alur kerja *chatbot*, pengembangan *chatbot* akademik pada penelitian ini dapat dilakukan secara lebih sistematis menggunakan teknologi pendukung seperti Rasa, RAG, dan LLM yang akan dijelaskan pada subbab berikutnya.

2.2.1.2 Rasa *Framework*

Rasa merupakan *Framework open source* untuk membangun *chatbot* dan asisten percakapan yang cerdas berbasis *machine learning*. *Framework* ini menyediakan arsitektur modular yang memungkinkan pengembang untuk mengekstraksi *intent* dan entitas pengguna, mengelola dialog *multi-turn*, serta menghubungkan *chatbot* dengan layanan *back*end atau API eksternal (Kong & Wang, 2021). Rasa menjadi salah satu plat*form* pengembangan *chatbot* paling populer karena fleksibilitas, dan komunitasnya yang besar.

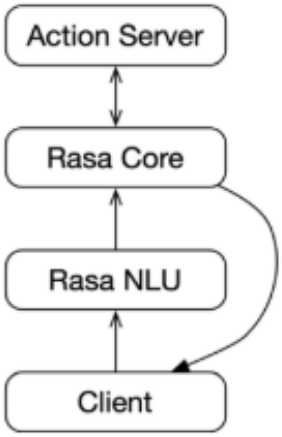
Secara umum, Rasa terdiri atas dua komponen utama, yaitu Rasa NLU dan Rasa *Core*, serta satu komponen tambahan berupa Rasa SDK untuk menangani aksi kustom (*custom actions*). Ketiga komponen ini bekerja sama untuk memahami pesan, menentukan tindakan, dan menghasilkan *respons* kepada pengguna.

Rasa NLU merupakan modul yang bertanggung jawab dalam proses pemahaman bahasa alami. Tidak seperti arsitektur NLU tradisional yang kompleks dan memerlukan banyak tahap pra-pemrosesan, rekayasa fitur, dan pelatihan model secara manual, Rasa NLU menawarkan pendekatan yang lebih modern dan terintegrasi. Modul ini secara langsung menggabungkan proses *intent classification* dan *entity extraction* dalam satu *pipeline machine learning* yang dapat dikonfigurasi. Rasa NLU bekerja pada level kalimat untuk mengidentifikasi maksud pengguna (*intent*), dan pada level kata untuk mengekstrasi entitas penting. Selain itu, Rasa NLU mampu melakukan slot filling untuk melengkapi informasi yang belum lengkap.

Setelah *intent* dan entitas dikenali oleh NLU, proses selanjutnya ditangani oleh Rasa *Core*, yaitu modul yang menentukan aksi terbaik yang harus dilakukan *chatbot* berdasarkan riwayat percakapan. Rasa *Core* mengelola percakapan *multi-turn*, mengelola konteks dialog, menangani ambiguitas, dan menentukan langkah percakapan berikutnya.

Rasa SDK digunakan untuk membuat aksi kustom (*custom actions*) yang dijalankan pada *Action* Server. *Custom action* digunakan ketika *chatbot* harus melakukan fungsi tertentu yang bergantung pada logika bisnis, seperti mengambil data dari *database*, mengirim notifikasi, melakukan perhitungan tertentu, atau menghubungkan ke layanan *back*end lainnya. *Action* Server berjalan sebagai proses terpisah dan berkomunikasi dengan

Rasa *Core* melalui HTTP. Arsitektur kerja keseluruhan Rasa yang melibatkan NLU, *Core*, dan *Action* Server ditunjukkan pada Gambar 2.2.



Gambar 2.2 Alur Proses Kerja Rasa (Kong & Wang, 2021)

Desain modular ini memberikan fleksibilitas tinggi, memisahkan logika percakapan dari logika bisnis, sehingga memudahkan pengembangan dan pemeliharaan sistem *chatbot* skala besar. (Kong & Wang, 2021) mencatat beberapa keunggulan utama Rasa diantaranya *open source* sehingga fleksibel untuk diakses dan dikustomisasi, mudah diintegrasikan dengan API dan layanan *back*end, memiliki komunitas dan dokumentasi yang kuat, serta cocok untuk domain spesifik seperti *chatbot* akademik.

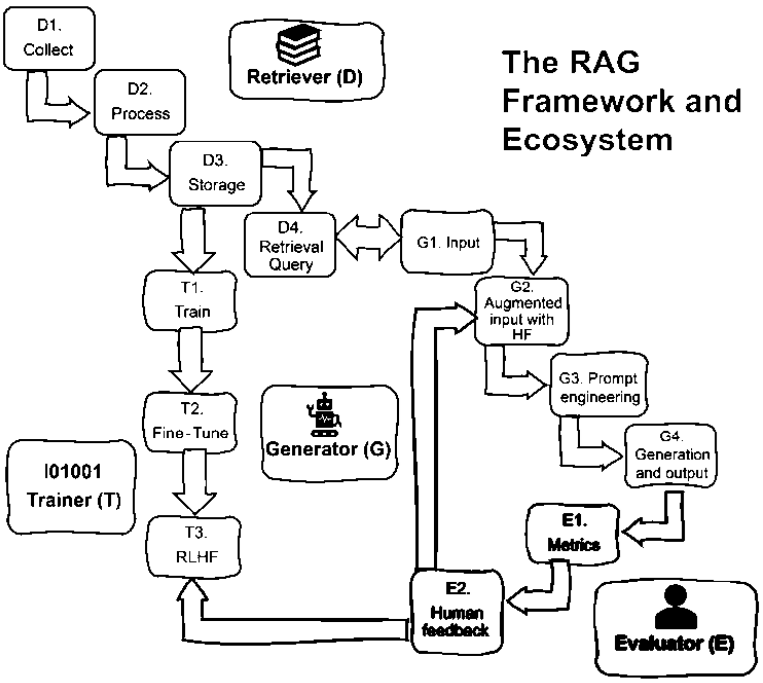
2.2.1.3 *Retrieval-Augmented Generation* (RAG)

Retrieval Augmented Generation (RAG) adalah pendekatan yang menggunakan kemampuan model generatif dengan proses pencarian informasi (*retrieval*) untuk menghasilkan *respons* yang lebih akurat dan berbasis data. RAG dikembangkan sebagai solusi terhadap keterbatasan LLM yang sering mengalami halusinasi, yaitu kondisi ketika model menghasilkan jawaban yang tidak akurat karena tidak memiliki informasi yang cukup dalam data latihnya (Rothman, 2024).

Menurut (Rothman, 2024), ketika sebuah model tidak memiliki pengetahuan mengenai topik tertentu, ia akan tetap berusaha menghasilkan jawaban berdasarkan *token* yang paling mungkin muncul, bukan jawaban yang benar secara faktual, sehingga menyebabkan kesalahan informasi. RAG bekerja dengan menyediakan informasi eksternal yang relevan kepada model generatif sebelum pembuatan jawaban dilakukan. RAG dapat

memberikan *knowledge* untuk LLM dengan dokumen yang diambil melalui proses *retrieval*, sehingga model dapat menghasilkan *respons* yang lebih tepat dan sesuai konteks (Rothman, 2024). Pendekatan ini sangat bermanfaat ketika domain informasi bersifat khusus, dinamis, atau memiliki aturan yang harus dipatuhi seperti SOP akademik di FSM UNDIP. Secara konseptual, RAG terdiri atas dua komponen utama, yaitu *retriever* dan Generator. *Retriever* bertugas mengumpulkan, memproses, menyimpan, dan mengambil informasi yang diperlukan untuk melengkapi masukan pengguna, sementara Generator menggunakan informasi tersebut untuk menghasilkan *respons* yang lebih akurat. Bersama dengan *evaluator* dan *trainer*, keduanya membentuk ekosistem RAG yang terdiri atas empat domain utama yaitu *data, storage, retrieval*, dan *generation* (Rothman, 2024).

Pada gambar 2.3, dapat dilihat arsitektur RAG, proses mengumpulkan, memproses, dan menyimpan dikelompokkan dalam domain *retriever*. Hal ini dikarenakan *retriever* tidak hanya melakukan pencarian dokumen, tetapi juga memastikan data terkumpul, diproses, dan disimpan dalam format yang dapat dicari. Data akan disimpan dalam bentuk terstruktur dan memungkinkan *retriever* memilih informasi yang paling relevan terhadap pertanyaan pengguna. Setelah dokumen relevan diambil oleh *retriever,* domain generator bertugas menggabungkan *input* pengguna dengan hasil *retrieval* tersebut sebagai konteks tambahan bagi model generatif. Generator kemudian melakukan *prompt Engineering*, menjalankan model LLM, dan menghasilkan keluaran akhir berupa jawaban yang lebih informatif dan akurat. Dengan demikian, generator menggabungkan pengetahuan internal model dengan pengetahuan eksternal yang diambil melalui proses retrieval, sehingga menghasilkan *respons* yang lebih akurat dan kontekstual. *Evaluator* menilai kualitas *output* yang dihasilkan oleh generator. Evaluasi ini dilakukan melalui dua pendekatan yaitu metrik otomatis (*metrics*) dan umpan balik manusia (*human feedback*). Evaluasi ini sangat penting untuk mengidentifikasi kesalahan, bias, maupun ketidaksesuaian konteks pada jawaban yang dihasilkan. Hasil evaluasi kemudian digunakan untuk memperbaiki model atau menyesuaikan *pipeline* RAG secara keseluruhan. *Trainer* bertanggung jawab terhadap proses pelatihan awal model generatif, termasuk *pre-training*, *fine-tuning*, dan reinforcement learning from human feed*back* (RLHF) apabila diperlukan. Pada implementasi tertentu, *Trainer* berfungsi menyesuaikan model agar lebih *respons*if terhadap data domain spesifik. Dengan adanya pelatihan yang berulang, kinerja generator dapat meningkat secara berkelanjutan sesuai kebutuhan sistem.



Gambar 2.3 RAG *Framework (Rothman, 2024)*

Dengan struktur tersebut RAG mampu menyatukan proses *retrieval* dan *generative* dalam satu alur kerja yang efisien. Integrasi antara retriever dan generator memastikan bahwa model tidak hanya mengandalkan pengetahuan internal LLM, tetapi juga selalu merujuk pada sumber informasi yang valid, sehingga kualitas *respons* dapat dipertahankan dan ditingkatkan secara berkelanjutan.

2.2.1.4 *Large Language Model* (LLM)

Large Language Model (LLM) dapat dipahami sebagai sebuah layanan yang memproses teks dan menghasilkan teks baru sebagai *respons*. Pengguna memberikan sebuah *string* atau teks masukan yang disebut *prompt*, yaitu instruksi atau pertanyaan yang ingin dijawab model. LLM kemudian menghasilkan keluaran berupa *completion* atau *response*, yaitu teks hasil prediksi *token* demi *token* berdasarkan pola yang telah dipelajari selama proses pelatihan. Dengan prinsip *text in*, *text out* ini, LLM mampu menyelesaikan berbagai tugas berbasis bahasa, seperti menjawab pertanyaan, menyusun ringkasan, maupun menghasilkan percakapan (Berryman & Ziegler, 2025).

Model yang termasuk dalam kategori LLM antara lain GPT (OpenAI), Gemini (Google/DeepMind), Claude (Anthropic), dan LLaMA (Meta). Pada penelitian ini,

digunakan model Gemini sebagai komponen generatif untuk menghasilkan jawaban yang bersifat natural dan mendukung proses *hybrid chatbot* berbasis Rasa dan RAG.

2.2.1.5 Pendekatan *Hybrid* pada Rasa dan RAG

Framework Rasa mendukung penerapan arsitektur hibrida modern untuk menangani kompleksitas percakapan pengguna. Berdasarkan dokumentasi resmi (Rasa Technologies, 2026), interaksi pengguna dikategorikan dalam dua jenis utama yaitu percakapan transaksional yang terstruktur dan percakapan informasional yang dinamis.

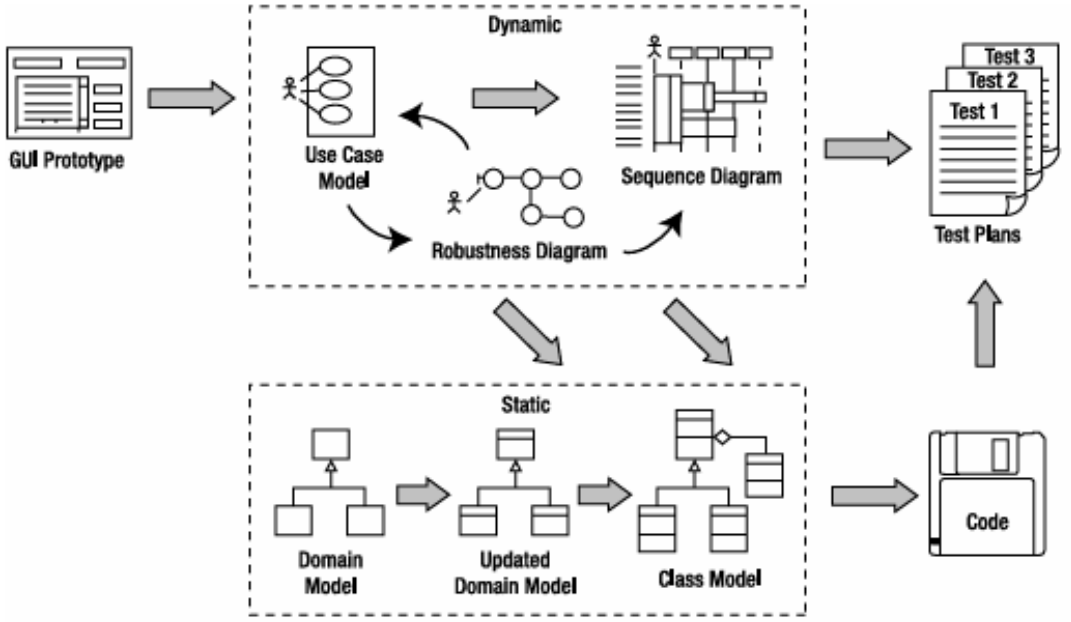
Untuk mengakomodasi kedua kebutuhan tersebut (Zangiabadi, 2025) menjelaskan bahwa kombinasi (*hybrid*) diperlukan untuk menjembatani kesenjangan antara kontrol terprediksi yang dimiliki Rasa dan kelancaran bahasa dari LLM. (Luo, 2025) dan (Rasa Technologies, 2026) lebih lanjut merinci mekanisme teknisnya, NLU digunakan untuk alur dialog yang diketahui sedangkan LLM digunakan untuk mekanisme *fallback* untuk menangani pertanyaan kompleks yang tidak terpola. Pendekatan ini menghasilkan sistem hibrida yang fleksibel dalam berbahasa namun dapat diandalkan keakuratan datanya (Kite Metric, 2025).

2.2.2 ICONIX *Process*

ICONIX *Process* adalah metode pengembangan perangkat lunak berbasis *use case* driven object modeling. ICONIX *Process* berfokus pada area yang berada di antara *use case* dan kode program, dilengkapi dengan tahap analisis dan perancangan. Menurut (Rosenberg dkk., 2007) ICONIX *Process* memiliki karakteristik minimalis, *use case driven*, dan *iteratif*. ICONIX *Process* terdiri dari empat tahap utama, yaitu *Requirement*, *Analysis/Preliminary Design*, *Detailed Design*, dan *Implementation* serta dilengkapi juga dengan tahap *review* seperti *Requirements Review*, *Preliminary Design Review*, dan *Critical Design Review*.

Pada gambar 2.4 dijelaskan, proses dibagi menjadi alur kerja dinamis dan statis yang sangat iteratif, dimana setiap tahapan memiliki umpan balik untuk menyempurnakan tahapan sebelumnya. Pada alur kerja dinamis, bertujuan untuk menghasilkan *use case diagram*, *robustness diagram* , dan *sequence diagram*. Lalu pada alur kerja statis bertujuan

untuk menghasilkan domain model yang akan mengalami iterasi secara berulang dan berkembang menjadi model yang lebih *update*.



Gambar 2.4 ICONIX *Process (Rosenberg dkk., 2007)*

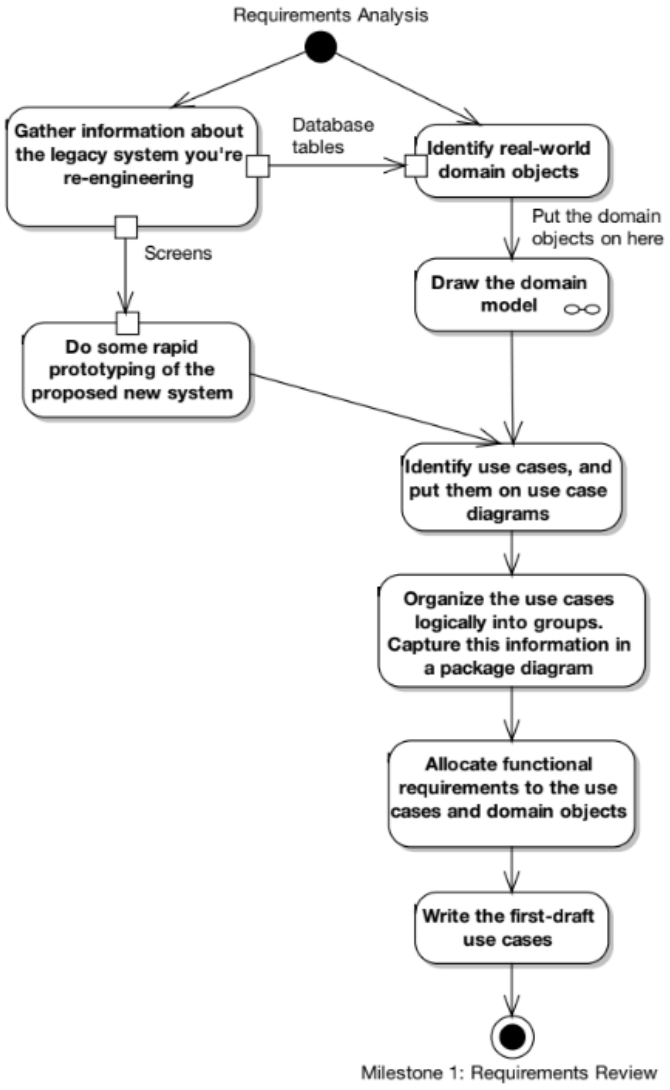
2.2.2.1 Tahap *Requirements*

Tahap *requirements* adalah tahap awal pada ICONIX *Process* yang bertujuan untuk menjelaskan apa saja kemampuan yang harus dimiliki sistem, memahami ruang masalah, dan menjelaskan bagaimana pengguna dan sistem saling berinteraksi. Tahap awal yaitu *functional requirements* yang dilakukan pada saat projek dimulai. Menurut (Rosenberg dkk., 2007), penyusunan *functional requirements* sangat bergantung pada bagaimana proyek kita dikelola. *Functional Requirements* dapat disusun sendiri oleh tim pengembang melalui analisis kebutuhan, atau dapat pula diberikan oleh *stakeholder* maupun analis bisnis. ICONIX *Process* tidak mewajibkan satu metode tunggal untuk mengumpulkan kebutuhan. Tekniknya dapat berupa analisis sistem berjalan, survei, studi dokumen, prototipe, maupun masukan langsung dari pengguna atau pihak terkait. Hal terpenting adalah *functional requirements* memberikan daftar yang jelas mengenai kemampuan apa saja yang harus dimiliki sistem.

Setelah *functional requirements* ditentukan, ICONIX *Process* melanjutkan tahap *requirements* dengan melakukan *domain modeling*. Tujuan dari pemodelan domain adalah untuk memahami ruang masalah secara jelas dan tidak ambigu. Pada buku yang ditulis oleh

Rosenberg ditekankan bahwa domain model awalnya belum sempurna, dan akan disempurnakan seiring eksplorasi *use case* lebih lanjut. Selanjutnya, tahap *requirements* mencakup penyusunan *behavioral requirements,* yaitu kebutuhan yang menjelaskan bagaimana pengguna dan sistem berinteraksi dalam skenario nyata.

Untuk menyusun behavioral requirements yang akurat, (Rosenberg dkk., 2007) menyarankan penggunaan GUI atau prototipe antarmuka sebagai alat bantu visual untuk mempermudah memahami alur interaksi yang terjadi pada sistem. Setelah daftar *use case* sudah teridentifikasi dibuatlah *use case* diagram dan kemudian draft pertama *use cases* dibuat. Diagram alir yang menggambarkan tahap *requirements analysis* pada ICONIX *Process* dapat dilihat pada Gambar 2.5.



Gambar 2.5 Tahap *Requirements (Rosenberg dkk., 2007)*

2.2.2.2 *Milestone* 1: *Requirements Review*

Menurut (Rosenberg dkk., 2007) tahap *requirements review* merupakan langkah penting untuk memastikan bahwa kebutuhan yang sudah didefinisikan pada tahap *requiremets* mudah dipahami. Pada bukunya ia memberikan beberapa langkah untuk melakukan *review* seperti memastikan setidaknya 80% permasalahan utama dalam bahasa yang mudah dipahami, memastikan bahwa domain model menunjukkan hubungan antar objek, memastikan *use case* mendeskripsikan skenario utama dan alternatif, dan memastikan bahwa *use case*, domain model, dan prototipe antarmuka sesuai dengan kebutuhan *user* dan stakeholder.

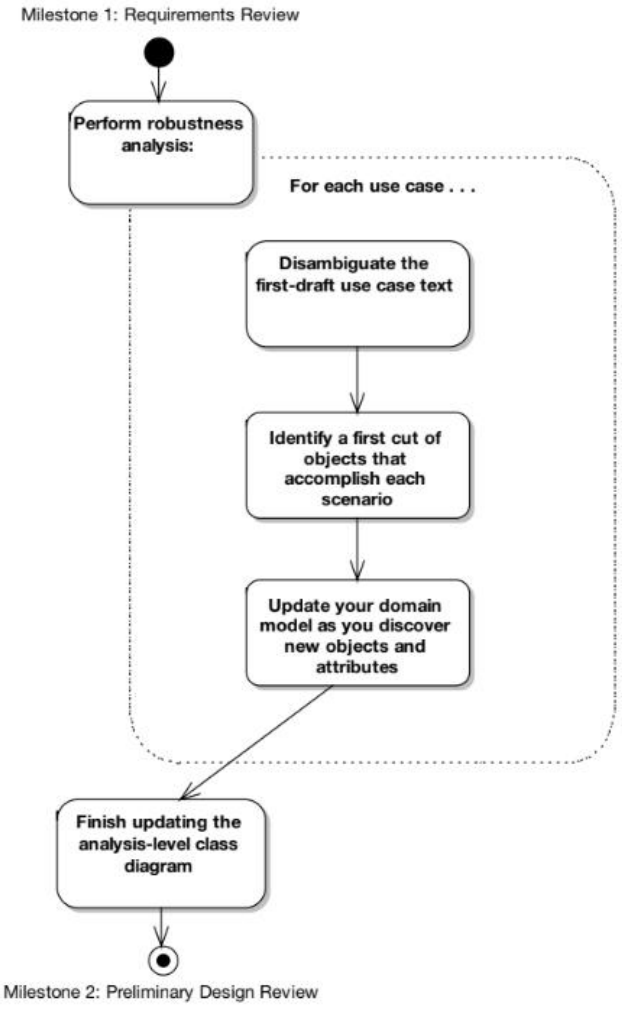
2.2.2.3 Analysis/*Preliminary Design*

Tahap analisis berfokus pada membuat sistem yang tepat, sedangkan desain berfokus pada membuat sistem dengan tepat. *Preliminary Design* berada di antara keduanya sebagai langkah yang menjembatani kebutuhan dengan desain rinci. Menurut (Rosenberg dkk., 2007), pengembang biasanya tidak dapat memahami kebutuhan secara tuntas tanpa melakukan eksplorasi desain terlebih dahulu. Tahap inilaih yang difasilitasi oleh *analysis / preliminary design* melalui teknik *robustness analysis.*

Pada Gambar 2.6 dijelaskan bahwa tahap *Analysis/Preliminary Design* pada ICONIX *Process* dilakukan secara iteratif untuk setiap *use case* guna menjembatani pemahaman antara kebutuhan (*requirements*) dan desain sistem. Tahap ini dimulai dengan menghilangkan ambiguitas dalam *use case* awal dengan memperjelas skenario dan detail teknis yang tidak sesuai dengan level analisis. Selanjutnya dilakukan *robustness analysis*, yaitu proses menggambar *robustness diagram* sebagai representasi visual dari langkah-langkah pada *use case* dalam bentuk interaksi antar objek.

Melalui diagram ini, pengembang kemudian mengidentifikasi objek yang terlibat pada skenario, baik berupa *boundary, entity,* maupun *Controller*. Selama proses tersebut, domain model diperbarui untuk menambahkan kelas atau atribut baru yang ditemukan sehingga model semakin lengkap dan konsisten. Tahap *preliminary design* kemudian diakhiri dengan penyelesaian *analysis-level class diagram,* yaitu diagram kelas pada level analisis yang telah memasukkan *boundary, entity*, dan *Controller* yang relevan untuk mewujudkan setiap *use case*. Seluruh rangkaian aktivitas ini menjadi dasar untuk mencapai

Milestone 2: Preliminary Design Review, yaitu kondisi ketika *robustness diagram*, domain model, dan *class diagram* analisis telah konsisten satu sama lain serta siap digunakan pada tahap desain rinci berikutnya.



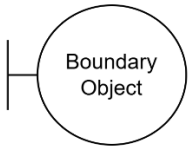
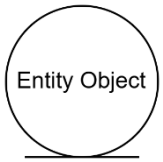
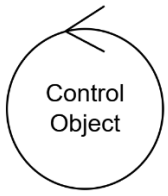
Gambar 2.6 *Preliminary Design (Rosenberg dkk., 2007)*

2.2.2.4 *Robustness* Diagram

Robustness Diagram merupakan artefak penting dalam ICONIX *Process* yang berfungsi menjembatani tahap analisis dengan tahap desain mendetail. Menurut (Rosenberg dkk., 2007), *robustness* diagram adalah representasi visual dari perilaku yang dideskripsikan dalam sebuah *use case.* Diagram ini berbentuk hibrida antara class diagram dan *activity* diagram, karena di satu sisi menggambarkan objek yang berpartisipasi, dan di sisi lain menggambarkan aliran aksi antar objek tersebut. *Robustness* diagram menunjukkan bagaimana langkah-langkah pada *use case* direalisasikan oleh objek-objek di dalam sistem, sehingga terdapat korelasi antara teks *use case* dengan alur pada diagram.

Pada *Robustness* diagram, terdapat tiga jenis objek utama yang digunakan, yaitu *boundary object, entity object*, dan *Controller*. Secara konseptual, aturan dasar interaksi pada *robustness* diagram seperti *boundary* dapat berinteraksi dengan *Controller*, dan *Controller* dapat berinteraksi dengan *entity*, sedangkan *boundary* tidak boleh berinteraksi langsung dengan sesama *boundary* atau *entity*. Aturan ini menjaga agar desain tetap terstruktur, memisahkan antarmuka, logika pemrosesan, dan data secara jelas.

Tabel 2.2 Simbol *Robustness* Diagram (Rosenberg dkk., 2007)

Simbol	Deskripsi
Boundary Object 	Merepresentasikan antarmuka sistem yang berinteraksi langsung dengan aktor. Biasanya berupa halaman web, layar (UI), *form input*, atau end*point* API. Bertindak sebagai pintu masuk data dari pengguna.
Entity Object 	Mewakili objek-objek domain yang menyimpan data dan merepresentasikan konsep bisnis inti.
Control Object 	Mengatur logika aplikasi dan menjadi penghubung antara *boundary* dan *entity*. Mengimplementasikan aksi atau fungsi logis pada *use case*.

Tabel 2.2 menunjukkan simbol-simbol yang digunakan dalam *robustness* diagram pada ICONIX *Process*. Dengan demikian, *robustness* diagram menjadi langkah penting dalam *preliminary design* untuk memastikan bahwa setiap kebutuhan telah diterjemahkan secara konsisten ke dalam desain yang valid dan dapat diimplementasikan.

2.2.2.5 *Milestone* 2 : *preliminary Design Review*

*Preliminary Design Review (*PDR) merupakan tahap yang membantu memastikan konsistensi antara *robustness* diagram, domain model, dan *use case* sehingga seluruh artefak berhubungan dan siap diturunkan ke *Sequence Diagram* serta *class* diagram *(Rosenberg dkk., 2007)*. Komponen yang ditinjau meliputi *use case* harus menggambarkan dialog dua arah antara aktor dan sistem dengan jelas, domain model yang bertindak sebagai kamus konsep berisi entitas dan relasinya, dan *robustness diagram* yang harus tetap berada pada level desain konseptual dan tidak masuk ke detail teknis seperti yang terdapat pada tahap *detailed design*. Melalui PDR, kesalahan analisis dapat diidentifikasi lebih awal sehingga desain dapat dilanjutkan secara lebih terarah dan konsisten.

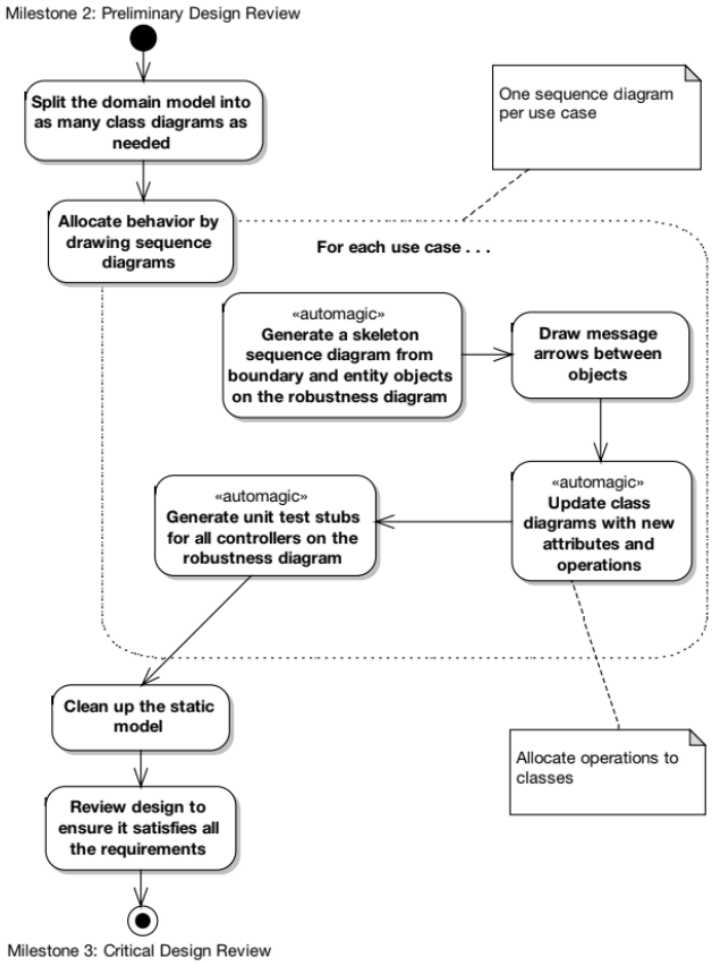
2.2.2.6 *Detailed Design*

Tahap *detailed design* adalah tahapan yang berfokus pada pembangunan sistem dengan cara yang benar (*building the system right*). Jika tahapan sebelumnya berfokus pada pemahaman kebutuhan ("membangun sistem yang benar"), tahap ini berfokus pada efisiensi eksekusi, beban jaringan, penggunaan memori, serta penggunaan kembali kode (*reusability*) (Rosenberg dkk., 2007). Pada tahap ini, *use case* yang telah didefinisikan harus sudah lengkap, benar, dan detail sehingga siap untuk dijadikan landasan desain teknis. Tujuan utamanya adalah menemukan alokasi fungsi yang optimal ke dalam kelas-kelas (*behavior allocation*).

Sequence Diagram digunakan sebagai kendaraan utama untuk mengeksplorasi desain detail sistem berdasarkan skenario per skenario. Fungsi utama *Sequence Diagram* adalah mengalokasikan perilaku ke dalam kelas-kelas (*allocate behavior to classes*). Dalam diagram ini, kontroler (*Controllers*) yang diidentifikasi pada tahap *Robustness Analysis* diubah menjadi pesan atau operasi pada *boundary objects* dan *entity objects*. *Sequence Diagram* menunjukkan secara detail bagaimana langkah-langkah dalam teks *use case* diimplementasikan oleh objek-objek sistem yang saling berinteraksi (Rosenberg dkk., 2007).

Pada Pada Gambar 2.7 proses *detailed design* menunjukkan bahwa terdapat langkah-langkah iteratif yaitu membuat satu *Sequence Diagram* untuk setiap *use case* guna menunjukkan detail implementasi, memperbarui model domain dengan menambahkan operasi (*operations*) ke objek domain saat menggambar *sequence diagram*. Pada tahap ini,

objek domain berevolusi menjadi kelas atau entitas yang sesungguhnya, dan merapikan model statis (*class diagram*) untuk memastikan desain terfaktor dengan baik sebelum masuk ke tahap pengkodean. Langkah pembuatan *Sequence Diagram* terdapat 4 langkah esensial yaitu, menempatkan teks *use case* (termasuk basic dan *alternate courses*) langsung ke dalam diagram sebagai referensi perilaku yang dibutuhkan, memasukkan objek entitas yang telah diidentifikasi dari *Robustness* Diagram, memasukkan elemen antarmuka (*boundary*) dan aktor yang terlibat, dan mengubah kontroler menjadi pesan antar objek. Langkah ini merupakan inti dari keputusan desain, di mana pengembang menentukan kelas mana yang bertanggung jawab atas operasi tertentu.



Gambar 2.7 *Detailed Design (Rosenberg dkk., 2007)*

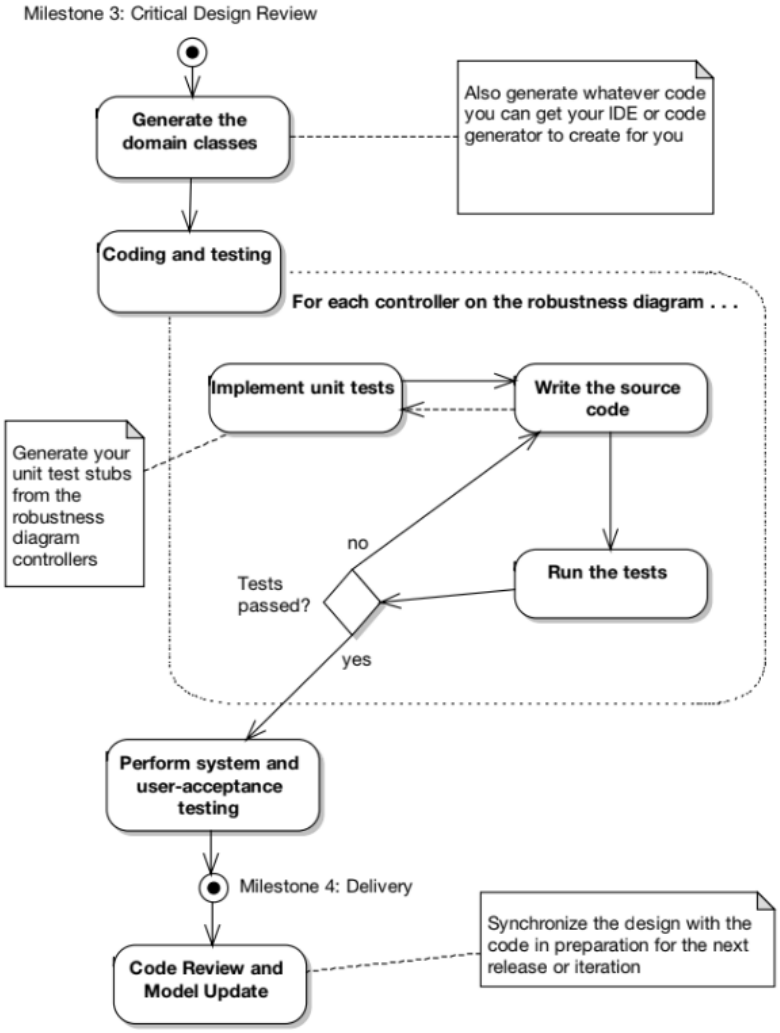
2.2.2.7 *Milestone* 3 :*Critical Design Review*

Pada taham *critical design review* tim pengembang memastikan bahwa cara sistem dibangun melalui *detailed design* sudah sesuai dengan apa yang ditentukan pada tahap *requirements*. Menurut (Rosenberg dkk., 2007) tahap ini bertujuan untuk meninjau kualitas

rancangan, memverifikasi konsistensi antara *use case, sequence diagram,* dan *class diagram,* serta menghilangkan celah logika yang dapat mengganggu implementasi.

2.2.2.8 Tahap *Implementation*

Tahap *Implementation* merupakan fase terakhir dalam ICONIX *Process,* yaitu di mana hasil desain yang telah disusun pada fase *detailed design* diubah menjadi kode program. Menurut (Rosenberg dkk., 2007), tahap ini befokus pada merepresentasikan rancangan dan kebutuhan sistem yang telah ditetapkan sebelumnya. ICONIX menekankan bahwa setelah usaha besar dilakukan untuk menghasilkan model yang konsisten dari *use case* hingga class diagram, akan menjadi kesalahan besar jika proses pengkodean dilakukan secara terpisah dari model yang telah dibuat.



Gambar 2.8 Critical Design *Review(Rosenberg dkk., 2007)*

2.2.3 Teori UML

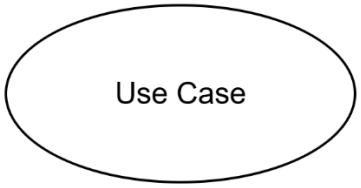
UML adalah bahasa standar untuk menulis cetak biru perangkat lunak yang digunakan untuk memvisualisasikan, menspesifikasi, membangun, dan mendokumentasikan sistem perangkat lunak. UML menyediakan seperangkat aturan sintaksis untuk merepresentasikan sistem. UML dikembangkan oleh *Object Management Group (OMG)* sebagai standar terbuka untuk mendukung sistem berorientasi objek(Booch dkk., 2005).

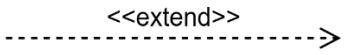
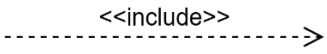
UML menurut (Fowler, 2004) bukanlah metode atau proses melainkan bahasa pemodelan yang dapat digunakan dalam berbagai pendekatan pengembangan perangkat lunak. Fowler juga menguraikan bahwa UML banyak digunakan untuk membuat ide desain secara cepat, merancang sistem secara rinci. UML digunakan untuk memfasilitasi komunikasi antara tim pengembang, *stakeholder,* dan pengguna, sehingga memastikan bahwa sistem memenuhi kebutuhan bisnis.

2.2.3.1 *Use case Diagram*

Menurut (Booch dkk., 2005) *use case* adalah deskripsi urutan aksi yang dilakukan sistem untuk menghasilkan hasil yang bernilai bagi seorang aktor. *Use case* menjelaskan implementasi sebuah sistem, tapi fokus pada apa yang dilakukan sistem dari sudut pandang pengguna. Sedangkan menurut (Fowler, 2004) *use case* adalah saranan utama untuk mengumpulkan kebutuhan fungsional, serta menjadi dasar untuk analisis, perancangan, dan pengujian. *Use case* juga digunakan sebagai alat komunikasi antara analis, developer, dan *stakeholder*. Daftar simbol pada *use case diagaram* dapat dilihat pada tabel 2.3.

Tabel 2.3 *Use case Diagaram (Rosenberg dkk., 2007)*

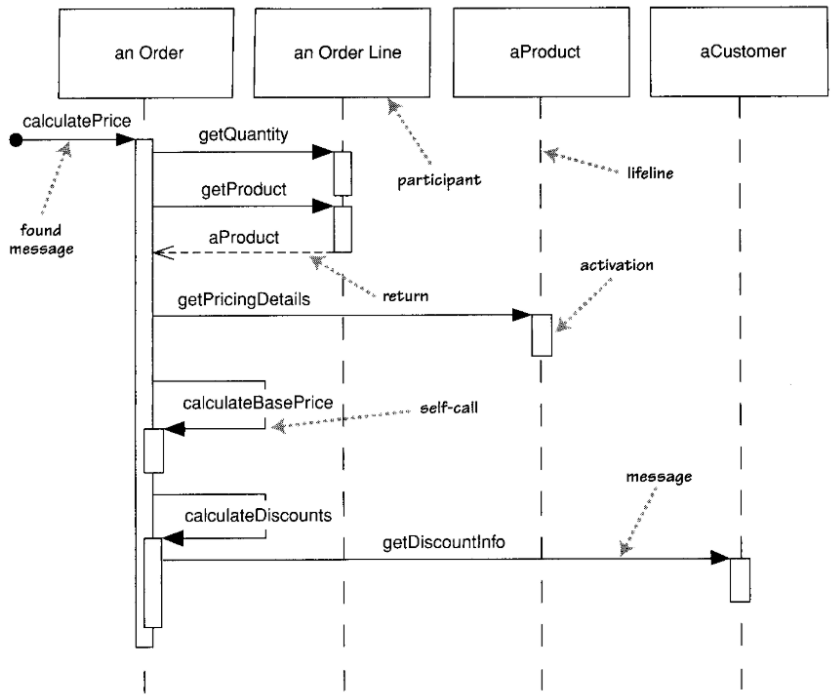
Simbol	Deskripsi
Use case 	*Use case* digambarkan sebagai elips yang merepresentasikan fungsi, layanan, atau tujuan yang ingin dicapai aktor melalui sistem.

Simbol	Deskripsi
Aktor 	Aktor adalah sesuatu yang berinteraksi dengan sistem dari luar batas sistem, aktor dapat berupa manusia, sistem eksternal, perangkat/sensor.
Asosiasi	Garis yang menghubungkan aktor dengan *use case*
Ekstensi 	Garis bertanda <<extend>>, menggambarkan bahwa satu *use case* dapat diperluas oleh *use case* lain
Include 	Garis bertanda <<include>>, menunjukkan bahwa satu *use case* memerlukan *use case* lain

2.2.3.2 *Sequence Diagram*

Menurut Fowler (2004), *Sequence Diagram* adalah diagram interaksi yang menekankan pada urutan waktu pengiriman pesan (*message*). Diagram ini digunakan untuk memodelkan logika dari sebuah skenario penggunaan (*use case*) dengan cara mendeskripsikan bagaimana objek-objek berinteraksi satu sama lain untuk menyelesaikan tugas tertentu.Contoh notasi dan elemen *Sequence Diagram* berdasarkan standar UML 2.0 dapat dilihat pada Gambar 2.9.

Seperti yang diilustrasikan pada Gambar 2.9, elemen dasar diagram ini terdiri dari partisipan atau objek yang diletakkan di bagian atas (misalnya *an Order* dan *an Order Line*), serta garis putus-putus vertikal yang disebut *lifeline*. *Lifeline* ini merepresentasikan keberadaan objek tersebut selama rentang waktu interaksi berlangsung, di mana waktu berjalan dari atas ke bawah.



Gambar 2.9 Simbol *Sequence Diagram* (Fowler, 2004)

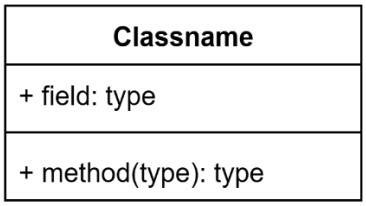
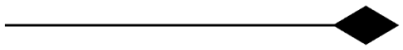
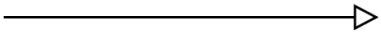
Dalam diagram ini, periode ketika sebuah objek sedang aktif melakukan pemrosesan atau memegang kendali eksekusi ditandai dengan kotak persegi panjang pada *lifeline* yang disebut sebagai *activation* atau *execution specification*. Interaksi antarobjek dilakukan melalui berbagai jenis pesan (*message*) yang dibedakan bentuk panahnya. Panah dengan kepala penuh (hitam) menandakan *synchronous message*, di mana pengirim menunggu hingga proses selesai (contoh pesan *getQuantity*), sedangkan garis putus-putus dengan panah terbuka menunjukkan *return message* atau pengembalian nilai. Selain itu, terdapat notasi *self-call* untuk menggambarkan objek yang memanggil metodenya sendiri (seperti *calculateBasePrice*), serta *found message* yang ditandai dengan lingkaran hitam kecil untuk menunjukkan pemicu awal interaksi yang sumbernya berada di luar cakupan diagram.

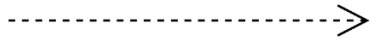
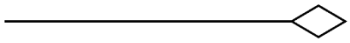
2.2.3.3 *Class Diagram*

Menurut (Booch dkk., 2005), *Class Diagram* menggambarkan struktur statis sistem dengan menampilkan kelas, atribut, metode, dan hubungan antar objek. Diagram ini menjadi inti dari model berorientasi objek karena mendefinisikan tipe-tipe objek yang membentuk sistem. *Class Diagram* disebut sebagai representasi paling penting dalam UML karena menjadi peta struktural sistem dan digunakan sejak analisis hingga implementasi (Fowler,

2004). Menurut (Rosenberg dkk., 2005) *class diagram* adalah hasil dari *robustness* dan *sequence diagram*. Daftar simbol pada *Class Diagram* dapat dilihat pada tabel 2.4.

Tabel 2.4 Simbol *Class Diagram (Booch dkk., 2005)*

Simbol	Deskripsi
Kelas / *Classes* 	Elemen utama dalam *class diagram* yang digambarkan sebagai persegi panjang terbagi tiga bagian: nama kelas (atas), atribut (tengah), dan operasi/metode (bawah). Mewakili *blueprint* untuk objek dalam sistem. Atribut adalah nama properti dari kelas yang mendeskripsikan rentang nilai yang dapat dimiliki oleh setiap *instance* kelas tersebut. Bersifat struktural dan menggambarkan data yang disimpan oleh objek. *Method* Adalah Implementasi dari layanan (*service*) yang dapat diminta dari objek untuk memengaruhi perilakunya. Merupakan abstraksi dari aksi yang dapat dilakukan terhadap objek dan ditulis di bagian bawah kelas.
Asosiasi/*association* 	Menghubungkan dua kelas yang memiliki relasi. Menunjukkan bahwa satu objek dari kelas pertama dapat berhubungan dengan satu atau lebih objek dari kelas kedua. Dapat dilengkapi dengan *multiplicity* (contoh: 1, 0..1, *).
Komposisi/*composition* 	Hubungan *whole–part* yang kuat, di mana bagian tidak dapat eksis tanpa keseluruhan. Jika objek *whole* dihapus, maka *part* juga ikut dihapus.
Generalisasi/*generalization* 	Menunjukkan hubungan pewarisan antar kelas *(superclass–subclass)*. *Subclass* mewarisi atribut dan operasi *superclass*.

Simbol	Deskripsi
Kebergantungan/*dependency* 	Menunjukkan bahwa perubahan pada satu kelas dapat memengaruhi kelas lain. Biasanya digunakan untuk hubungan sementara (misalnya pemanggilan metode).
Agregasi/*aggregation* 	Menunjukkan hubungan *whole–part* di mana bagian *(part)* dapat eksis secara independen dari keseluruhan *(whole)*.

2.2.4 *Framework* dan Teknologi

Pengembangan sistem *chatbot* berbasis web membutuhkan pemilihan teknologi yang tepat pada setiap lapisan arsitektur. Setiap komponen teknologi memiliki peran penting dalam mendukung proses komunikasi klien–server, pemrosesan logika aplikasi, penyajian antarmuka, serta pengelolaan data. Pada penelitian ini digunakan Web Server dan protokol HTTP sebagai dasar komunikasi web, FastAPI sebagai *Framework back*end, Next.js sebagai *Framework front*end, serta PostgreSQL sebagai basis data relasional. Uraian singkat mengenai masing-masing teknologi disajikan pada subbab berikut.

2.2.4.1 Web Server dan Protokol Hypertext Transfer (HTTP)

Web Server merupakan komponen utama dalam arsitektur *World Wide Web*, memiliki fungsi krusial sebagai host dan penyedia konten web (Gourley & Totty, 2002). Dalam operasionalnya, Web Server secara esensial dikenal sebagai HTTP Server karena inti komunikasinya didasarkan pada *Hypertext Transfer Protocol* (HTTP). Protokol ini adalah bahasa fundamental yang bertugas memindahkan berbagai jenis konten multimedia, mulai dari dokumen HTML, gambar, hingga data dinamis yang dihasilkan sesuai permintaan, dari server menuju klien web. Salah satu karakteristik penting dari HTTP adalah keandalannya dalam transmisi data, yang menjamin bahwa integritas informasi tetap terjaga selama pengiriman. Keefektifan dan kesederhanaan protokol ini tidak hanya terbatas pada aktivitas penjelajahan web, melainkan juga menjadikannya landasan bagi berbagai aplikasi jaringan modern, termasuk layanan web yang berfungsi sebagai mekanisme integrasi data.

Arsitektur komunikasi antara server dan klien ini beroperasi dalam sebuah model yang disebut Klien Server (*Client Server Model*). Transaksi dimulai ketika *Web Client*, seperti web, mengirimkan Permintaan HTTP (*HTTP Request*) yang secara spesifik menargetkan suatu Sumber Daya Web (*Web Resource*) pada server. Sumber daya tersebut bisa berupa file statis yang tersimpan dalam sistem atau konten dinamis yang dihasilkan oleh perangkat lunak sesuai kebutuhan. Untuk mengidentifikasi lokasi sumber daya secara unik dan tepat, klien menggunakan *Uniform Resource Locator* (URL), yang merupakan bentuk paling umum dari *Uniform Resource Identifier* (URI). Setelah memproses permintaan, Web Server akan mengirimkan Respons HTTP (*HTTP Response*) yang memuat data yang diminta(Gourley & Totty, 2002).

2.2.4.2 FAST API

Pengembangan web modern ditandai dengan pergeseran fokus ke layanan dan API *(Application Programming Interfaces)*. API menjadi kontrak yang sangat penting, mirip dengan skema basis data, untuk menghubungkan *back*end (mengurus akses basis data dan logika bisnis) dan *frontend* (antarmuka pengguna interaktif). FastAPI adalah *Framework* web modern, cepat (*high-per*form*ance*) untuk membangun API dengan Python 3.6+ berdasarkan *type hints* Python standar (Lubanovic, 2024). Keunggulan FastAPI adalah performanya yang cepat dan secara langsung menghasilkan spesifikasi API yang memudahkan dokumentasi dan pengujian *endpoint*.

(Lubanovic, 2024) menyarankan arsitektur berlapis untuk web skala sedang hingga kompleks. disarankan menggunakan model tiga tingkat yang jelas terpisah, yaitu Lapisan Web (*Web Layer*), Lapisan Layanan (*Service Layer*), dan Lapisan Data (*Data Layer*). Arsitektur ini sangat cocok dengan struktur *Controller* (API), *Service*, *Models*, dan *Repository* (Data). Model arsitektur tersebut sangat didukung oleh desain dan filosofi inti dari FastAPI.

2.2.4.3 Next JS

Next.js merupakan *open source* JavaScript *web Framework* yang dibangun di atas React, dirancang untuk membangun aplikasi web yang skalabel, berkinerja tinggi, dan modern (Riva, 2022). Kerangka kerja ini dikenal sebagai React *Framework for production* karena menyediakan serangkaian fitur penting secara *out of the box*, seperti *server-side*

rendering (SSR), *static site generation* (SSG), *incremental static generation*, dan sistem *routing* berbasis berkas (*file-based routing*). Next.js berfungsi sebagai kerangka kerja universal yang memungkinkan pengembang menulis kode untuk sisi klien maupun server dalam satu aplikasi.

Struktur proyek dasar Next.js memiliki dua direktori yang bersifat wajib dan dicadangkan, yaitu *pages* dan *public*. Direktori *pages* merupakan implementasi dari *file-based routing* Next.js, di mana setiap berkas JavaScript di dalamnya secara otomatis akan menjadi halaman atau route publik pada aplikasi. Direktori *public* berfungsi untuk menampung semua aset statis dan publik yang digunakan di situs web, seperti gambar, *font*, dan *stylesheet* yang telah dikompilasi. Selain kedua direktori wajib tersebut, pengembang dapat dengan bebas menambahkan direktori lain seperti *components*, *utilities*, atau *styles* untuk tujuan pengorganisasian kode tanpa mengganggu proses *build* atau pengembangan Next.js (Riva, 2022).

2.2.4.4 Postgre SQL

Buku The Art of PostgreSQL oleh (Fontaine, 2021; Maymala, 2015)secara khusus membahas PostgreSQL sebagai sistem manajemen basis data relasional obyek (*Object-Relational Database Management System*/ORDBMS) yang bersifat *open-source* dan menekankan pada ekstensibilitas serta kepatuhan terhadap standar SQL. PostgreSQL dikembangkan oleh Departemen Ilmu Komputer di Berkeley dan telah banyak diterapkan dalam berbagai jenis aplikasi, mulai dari aplikasi *mobile*, web, hingga sistem analitik. Salah satu keunggulan utama PostgreSQL dibandingkan dengan RDBMS lainnya adalah kemampuannya untuk digunakan dalam skala besar, terintegrasi dengan server, serta mendukung berbagai bahasa pemrograman populer seperti JavaScript, Java, PHP, dan Python.

2.2.5 *Software Testing*

Pengujian perangkat lunak (*software testing*) merupakan elemen fundamental dalam siklus hidup pengembangan sistem yang berfungsi untuk menjamin kualitas dan keandalan produk akhir. Menurut (Leloudas, 2023), pengujian perangkat lunak didefinisikan sebagai proses evaluasi terhadap sistem atau aplikasi untuk menemukan cacat (*defects*), kesalahan (*errors*), atau *bug*, sekaligus memverifikasi bahwa perangkat lunak tersebut telah memenuhi

persyaratan yang ditentukan dan mampu berfungsi dengan benar. Aktivitas ini bukanlah peristiwa satu kali, melainkan sebuah proses berkelanjutan yang dimulai dari tahap awal pengembangan hingga tahap pemeliharaan. (Desikan & Ramesh, 2006) menambahkan bahwa pengujian harus dilihat dalam konteks yang lebih luas daripada sekadar menemukan kesalahan kode, dengan prinsip utama menemukan cacat sedini mungkin sebelum produk sampai ke tangan pelanggan. Mengingat pengujian menyeluruh (*exhaustive testing*) hampir tidak mungkin dilakukan karena banyaknya kombinasi jalur logika, pengujian harus dilakukan dengan strategi yang efektif untuk meminimalkan risiko kegagalan sistem di lingkungan produksi. Selain itu, pelaksanaan pengujian sejak dini sangat krusial karena biaya perbaikan cacat akan meningkat secara signifikan seiring berjalannya waktu pengembangan.

Black-Box Testing adalah pendekatan pengujian perangkat lunak yang berfokus pada fungsionalitas eksternal sistem tanpa memerlukan pengetahuan mengenai struktur kode internal, logika, atau algoritma yang membangunnya. Dalam metode ini, penguji memperlakukan sistem sebagai "kotak hitam", di mana perhatian utama tertuju sepenuhnya pada input yang diberikan dan kesesuaian output yang dihasilkan terhadap spesifikasi. Sebagaimana dijelaskan oleh (Leloudas, 2023), teknik ini memungkinkan pengujian dilakukan berdasarkan spesifikasi persyaratan, sehingga penguji dapat memvalidasi apakah perangkat lunak memenuhi kebutuhan pengguna tanpa terganggu oleh detail teknis implementasi. Teknik yang paling fundamental dan komprehensif dalam pendekatan ini untuk memvalidasi kebutuhan pengguna adalah *Requirements Based Testing*.

Requirements Based Testing merupakan teknik pengujian yang dirancang secara spesifik untuk memvalidasi bahwa sistem telah memenuhi seluruh persyaratan yang didokumentasikan dalam *Software Requirements Specification* (SRS). Menurut (Desikan & Ramesh, 2006) teknik ini bertujuan untuk memastikan bahwa semua persyaratan, baik yang eksplisit tertulis dalam dokumen maupun yang implisit diasumsikan ada, telah teruji dengan baik melalui tinjauan mendetail terhadap SRS untuk memastikan konsistensi dan kelengkapannya . Inti dari pelaksanaan teknik ini adalah penggunaan alat bantu yang disebut *Requirements Traceability Matrix* (RTM). RTM berfungsi sebagai jembatan vital yang menghubungkan dokumen persyaratan dengan kasus uji (*test cases*), melacak setiap persyaratan dari awal hingga tahap pengujian. Melalui penggunaan RTM, tim penguji dapat memetakan setiap ID persyaratan ke dalam kondisi pengujian spesifik, sehingga memastikan

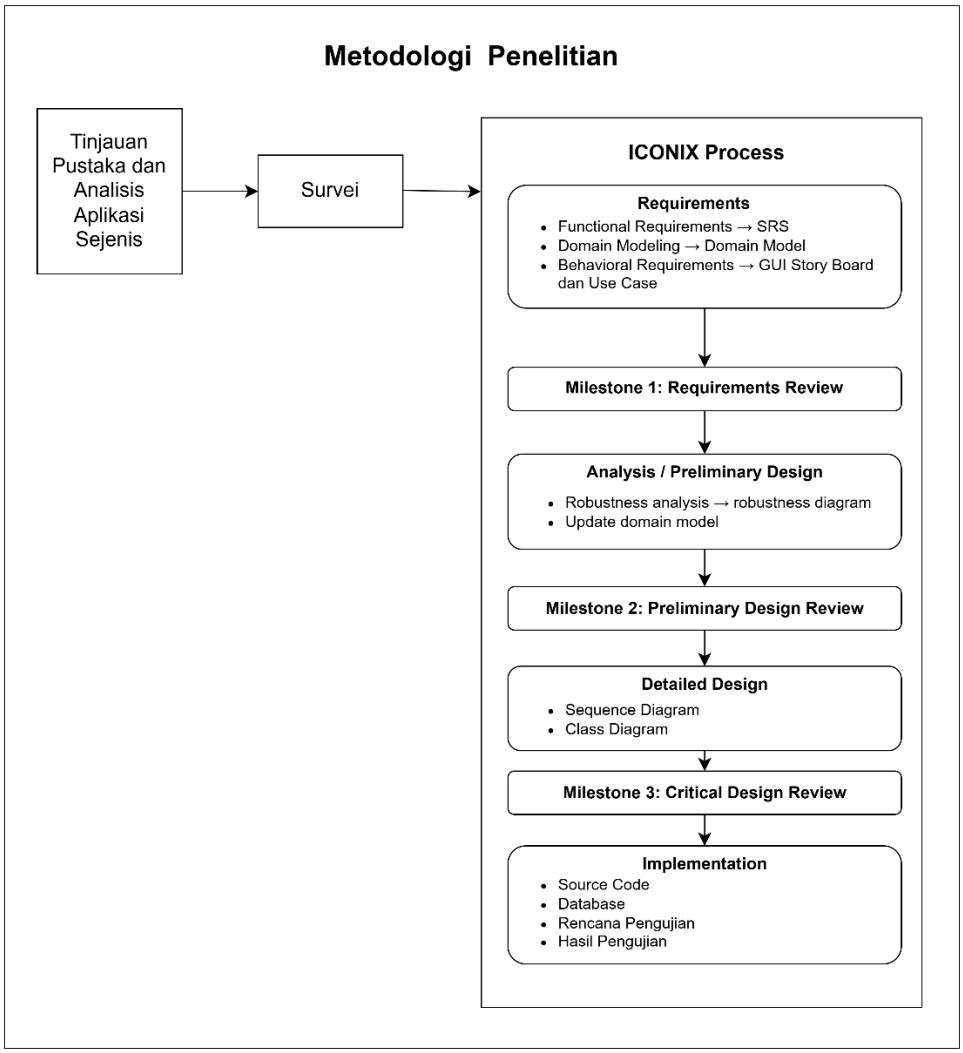
tidak ada fitur yang terlewat untuk diuji. Selain itu, RTM memungkinkan tim untuk memprioritaskan eksekusi pengujian pada fitur-fitur yang memiliki risiko tinggi atau dampak besar bagi pengguna, serta membantu mengidentifikasi dampak cacat dengan cepat jika sebuah tes gagal. Dengan demikian, pendekatan ini memberikan jaminan bahwa produk akhir benar-benar sesuai dengan spesifikasi yang disepakati dan meminimalkan risiko kegagalan fitur di lingkungan pengguna akhir.

2.2.6 Metode Kuisioner

Kuesioner merupakan teknik pengumpulan data yang dilakukan dengan cara memberi seperangkat pertanyaan atau pernyataan tertulis kepada responden untuk dijawab. Penyebaran kuesioner pada penelitian ini dilakukan menggunakan media elektronik (online survey) untuk menjangkau responden secara efisien. Untuk mengukur pendapat responden, digunakan Skala Likert. Skala Likert digunakan untuk mengukur sikap, pendapat, dan persepsi seseorang atau sekelompok orang tentang fenomena sosial (Sugiyono, 2022).

BAB III
METODE PENELITIAN

Bab ini berisi metode penelitian yang diimplementasikan oleh penulis dalam tugas akhir mengenai Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*. Penelitian dimulai dengan tinjauan pustaka, kemudian dilakukan survei dan masuk ke tahap ICONIX *Process*. Metodologi utama penelitian yang dapat dilihat pada Gambar 3.1



Gambar 3. 1 Metodologi Penelitian

3.1 Tinjauan Pustaka dan Analisis Aplikasi Sejenis

Tahapan awal dalam penelitian ini adalah melakukan tinjauan pustaka dan analisis aplikasi sejenis. Kedua kegiatan ini dilakukan untuk memperoleh pemahaman mendalam

mengenai teori, konsep, serta sistem yang relevan dengan topik penelitian. Tinjauan pustaka dilakukan untuk mengetahui lebih dalam pendekatan dan metode serta teknologi yang paling sesuai untuk diterapkan dalam Pengembangan *Chatbot Hybrid* Berbasis Web Menggunakan Rasa dan LLM Dengan Metodologi ICONIX (Studi Kasus FSM UNDIP). Hasil dari tinjauan pustaka ini digunakan sebagai landasan teori yang dijelaskan pada Bab II.

Selain mempelajari teori, dilakukan pula analisis terhadap beberapa aplikasi *Chatbot* akademik sejenis yang telah dikembangkan sebelumnya di lingkungan perguruan tinggi lain. Tahapan ini bertujuan untuk mengidentifikasi fitur, antarmuka, serta teknologi yang digunakan oleh sistem *Chatbot* sejenis. Selain itu tahapan ini juga dilakukan untuk menilai kelebihan dan kekurangan dari sistem yang ada, sehingga dapat diketahui gap yang bisa diperbaiki dan ditingkatkan serta menentukan inovasi dan keunikan dari sistem yang akan dikembangkan. Analisis aplikasi sejenis ini menghasilkan perbandingan yang akan disajikan pada Bab IV.

3.2 Tahap *Requirements*

Pada penelitian ini dilakukan pengumpulan kebutuhan fungsional dengan melakukan survei kepada mahasiswa untuk mengidentifikasi masalah yang dialami pengguna dalam mencari informasi akademik, serta fitur yang mereka anggap penting pada layanan *chatbot*. Selain itu juga dilakukan analisis *chatbot* akademik dari institusi lain sebagai referensi fitur dan praktik terbaik. Hasil pengumpulan kebutuhan tersebut dirumuskan menjadi *functional requirements* dan dituangkan ke dalam dokumen *Software Requirements Spesification* (SRS) yang berisi daftar kemampuan yang harus dapat dilakukan oleh sistem.

Setelah itu dilakukan pemodelan domain untuk memahami ruang masalah secara jelas. Domain model yang dibuat pada tahap ini adalah tahap awal yang nantinya akan disempurnakan setelah dilakukan eksplorasi *use case*.

Langkah berikutnya adalah menyusun *behavioral requirements*. Identifikasi *use case* dilakukan dengan bantuan prototipe untuk mengetahui skenario yang menggambarkan bagaimana pengguna dan sistem akan berinteraksi. Setelah daftar *use case* diper*oleh*, kemudian dibuat *use case* diagram. Tahap terakhir adalah dengan mengalokasikan kebutuhan fungsional yang sudah tercantum pada SRS dengan *use case* dan objek domain sehingga setiap kebutuhan dapat ditelusuri ke skenario interaksi dan objek sistem sesuai pendekatan ICONIX *Process*.

3.3 *Milestone* 1: *Requirements Review*

Pada penelitian ini, proses *requirements review* dilakukan untuk memastikan bahwa kebutuhan yang telah ditetapkan sesuai dengan kebutuhan pengguna sebelum masuk tahap desain. *Review* dilakukan dengan melakukan *review* bersama *stakeholder* Fakultas (Validasi GUI *Storyboard*).

Pada sesi ini, *stakeholder* memeriksa alur penggunaan dan antarmuka yang direpresentasikan pada SRS, *use case* dan prototipe antarmuka. Validasi GUI merupakan metode yang direkomendasikan oleh ICONIX *Process* karena prototipe mampu memvisualisasikan skenario *use case* yang mudah dipahami sehingga stakeholder mudah untuk mengidentifikasi alur utama, alur alternatif, dan potensi kebutuhan tambahan. Selain itu juga dilakukan validasi GUI pada pengguna akhir yaitu mahasiswa menggunakan kuisioner pada 5 mahasiswa di FSM UNDIP.

Setelah melakukan validasi GUI, dilakukan *cross-artifact Checking* untuk memastikan keterkaitan dan konsistensi antara seluruh artefak *requirements,* yaitu SRS, domain model, *use case*, dan prototipe antarmuka. *Review* ini memastikan bahwa setiap kebutuhan pada SRS memiliki representasi pada *use case*, bahwa domain model telah mencakup entitas yang digunakan pada skenario, serta bahwa seluruh *use case* telah divisualisasikan dengan tepat melalui storyboard antarmuka.

3.4 Tahap Analysis/*Preliminary Design*

Pada penelitian ini, tahap *analysis / preliminary design* diadopsi dari ICONIX *Process* dan berfungsi untuk menjembatani kebutuhan dengan desain rinci. Tahap ini difokuskan untuk memastikan bahwa setiap *use case* yang telah ditetapkan pada tahap sebelumnya dapat direalisasikan secara logis melalui pemetaan objek dan alur interaksi.

Kegiatan utama pada tahap ini adalah melakukan *robustness analysis* dan pembaruan domain model. *Robustness Analysis*, yaitu memvalidasi setiap *use case* dengan menyusun *robustness* diagram yang memetakan interaksi antara *boundary, control,* dan *entity*. Tahapan ini digunakan untuk mengidentifikasi objek-objek yang benar-benar diperlukan dan memastikan tidak ada ambiguitas sebelum masuk desain rinci. Pembaruan Domain Model, yaitu memperbarui domain model awal berdasarkan temuan dari *robustness analysis*. Pada tahap ini ditambahkan entitas baru, atribut, atau relasi yang muncul dari skenario *use case* sehingga model menggambarkan kebutuhan fungsional secara lebih lengkap.

3.5 *Milestone* 2: *Preliminary Design Review*

Pada penelitian ini, *Prelimenary Design Review* dilakukan sebagai tahap evaluasi internal untuk memastikan konsistensi dan keterhubungan antara *use case, domain model*, dan *robustness diagram* sebelum melanjutkan ke perancangan rinci seperti *Sequence Diagram* dan *class diagram*. *Prelimenary Design Review* dilakukan dengan metode *cross artifacy Checking* oleh peneliti, yaitu meninjau setiap langkah pada *use case* agar sesuai dengan elemen pada *robustness* diagram, memastikan bahwa seluruh *entity* yang muncul telah terdefinisi pada domain model, serta memeriksa bahwa *robustness* diagram tetap berada pada tingkat desain konseptual. Proses ini mengikuti pedoman ICONIX *Process* yang menekankan konsistensi, kelengkapan alur, serta kepatuhan terhadap aturan *boundary,control,* dan *entity*.

3.6 Tahap *Detailed Design*

Tahap *detailed design* merupakan fase di mana model analisis yang telah dibuat pada tahap sebelumnya diterjemahkan ke dalam desain teknis yang lebih mendetail. Fokus utama tahap ini adalah menyusun *Sequence Diagram* dan *class diagram* berdasarkan hasil *robustness analysis*.

Sequence Diagram digunakan untuk menggambarkan interaksi dinamis antar objek dalam sistem berdasarkan skenario dari setiap *use case.* Pada tahap ini dilakukan pemetaan antara *boundary, control,* dan *entity* yang telah diidentifikasi sebelumnya ke dalam alur pesan dan urutan eksekusi logika sistem. *Class Diagram* dihasilkan untuk menunjukkan struktur statis sistem secara lengkap, mencakup atribut, operasi, dan relasi antar kelas.

3.7 *Milestone* 3: *Critical Design Review*

Milestone ketiga atau *Critical Design Review* (CDR) dilakukan untuk memastikan bahwa desain sistem sudah matang dan siap diimplementasikan ke dalam kode program. Pada penelitian ini, CDR dilakukan dengan metode *cross-artifact Checking* yang berfokus pada sinkronisasi antara desain dinamis (*Sequence Diagram*) dan desain statis (*Class Diagram*).

Proses evaluasi ini melibatkan peninjauan kembali apakah setiap langkah dalam teks *use case* telah terwakili secara visual dalam pesan-pesan di *Sequence Diagram*. Selain itu, peneliti memeriksa apakah setiap pesan (*message*) yang dikirimkan antar objek dalam *Sequence Diagram* telah didefinisikan sebagai operasi (metode) yang valid dalam *Class Diagram*. Tujuannya adalah untuk menjamin tidak ada celah logika (*logic gaps*) dan memastikan bahwa seluruh atribut serta metode yang diperlukan telah dialokasikan dengan tepat ke dalam kelas-kelas terkait sebelum tahap implementasi dimulai.

3.8 Tahap *Implementation*

Tahap *Implementation* merupakan fase di mana seluruh rancangan sistem yang telah disusun dan divalidasi pada tahap sebelumnya diterjemahkan ke dalam bentuk kode program. Pada tahap ini, pengembangan sistem dilakukan dengan pendekatan *modular* menggunakan *Framework* FastAPI (Python) untuk sisi *back*end dan Next.js (TypeScript) untuk sisi *front*end. Arsitektur sistem dirancang mengikuti pola *Controller*–Service–Repository (CSR) guna memastikan adanya pemisahan tanggung jawab yang jelas antar komponen serta mempermudah proses pengujian, pemeliharaan, dan pengembangan di masa mendatang.

Pada sisi *back*end, sistem dikembangkan untuk menangani proses autentikasi pengguna, pengelolaan percakapan, integrasi dengan mesin *Chatbot hybrid* Rasa NLU dan Retrieval-Augmented Generation (RAG) berbasis LLM, serta penyimpanan data ke dalam basis data PostgreSQL. Lapisan ini juga mengimplementasikan logika bisnis utama dan menyediakan *Application Programming Interface (API)* yang dapat diakses oleh *front*end.

Sementara itu, sisi *front*end dikembangkan menggunakan Next.js untuk membangun antarmuka pengguna yang interaktif dan *respons*if. Antarmuka ini mencakup halaman *login*, percakapan *Chatbot*, *dashboard* analitik, serta fitur manajemen log percakapan bagi pengguna berperan sebagai admin.

Komponen utama sistem, yaitu *Chatbot Engine*, berfungsi sebagai pusat pemrosesan bahasa alami. Arsitektur *Chatbot* ini menggabungkan kekuatan Rasa untuk mendeteksi *intent* dengan tingkat kepercayaan tinggi dan LLM untuk menghasilkan jawaban kontekstual ketika sistem mendeteksi *fallback*. Pendekatan *hybrid retrieval* ini memungkinkan *Chatbot* menjawab pertanyaan pengguna dengan akurat berdasarkan SOP akademik FSM UNDIP sekaligus memberikan *respons* adaptif untuk pertanyaan terbuka namun tetap dapat menghindari masalah halusinasi.

Setelah seluruh komponen selesai diimplementasikan, dilakukan proses integrasi dan pengujian sistem menggunakan *blackbox testing*. Pengujian ini bertujuan untuk memastikan setiap modul berfungsi sesuai rancangan, API terintegrasi dengan baik, dan *Chatbot* mampu memberikan jawaban yang relevan.

Pada tahap ini juga dilakukan pengaturan parameter untuk mengatur keseimbangan antara akurasi dengan fleksibilitas jawaban dari model *hybrid*. Dilakukan percobaan menggunakan beberapa variasi nilai *threshold*.

Hasil akhir dari tahap implementasi ini adalah sistem *Chatbot Hybrid* Berbasis Web Fakultas Sains dan Matematika Universitas Diponegoro. Sistem ini telah mengintegrasikan teknologi Rasa dan RAG secara fungsional serta memenuhi kebutuhan layanan akademik yang telah didefinisikan pada tahap awal penelitian.

BAB IV
HASIL DAN PEMBAHASAN

Bab ini membahas hasil dari proses analisis dan perancangan yang telah dilakukan dalam penelitian Pengembangan *Academic Chatbot Hybrid* Berbasis Web Menggunakan *Framework* NLU-Rasa dan LLM-RAG dengan Metodologi ICONIX *Process*. Pembahasan meliputi analisis aplikasi sejenis sebagai bahan pembanding, hasil survei kebutuhan pengguna, serta analisis kebutuhan sistem berdasarkan metode ICONIX *Process*. Hasil-hasil ini menjadi dasar dalam perancangan dan implementasi sistem pada bab berikutnya.

4.1 Analisis Aplikasi Sejenis

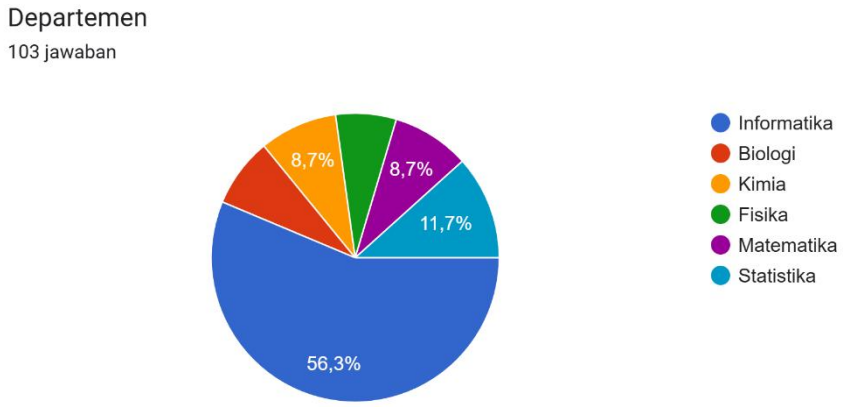
Analisis aplikasi sejenis dilakukan untuk memahami bagaimana sistem *Chatbot* akademik telah dikembangkan pada institusi lain, serta mengidentifikasi fitur-fitur yang dapat diadopsi atau disesuaikan dengan kebutuhan Fakultas Sains dan Matematika UNDIP. Dua aplikasi dijadikan pembanding utama, yaitu AKASHA – *Chatbot* AI Undiksha dan *Chatbot* Akademik UPN 'Veteran' Jakarta.

Tabel 4.1 Analisis Aplikasi Sejenis

Aplikasi / Penelitian	Deskripsi Singkat	Fitur yang Diadopsi
AKASHA – *Chatbot* AI Undiksha (Sudiartika, 2025) http://akasha.ganeshaku.my.id/	*Chatbot* berbasis *Corrective* RAG untuk pencarian informasi akademik di Undiksha. Menggunakan LangChain, LangGraph, FAISS, Flask, dan Vue.js. Memiliki *dashboard* admin lengkap dan pengelolaan model AI.	- Penggunaan RAG untuk pencarian kontekstual. - *Dashboard* admin untuk monitoring. - Riwayat percakapan untuk audit.
Chatbot Akademik UPN 'Veteran' Jakarta (Notokoesoemo, 2023) https://repository.upnvj.ac.id/25035/	*Chatbot* berbasis Rasa Open *Source* untuk menjawab pertanyaan akademik (mahasiswa, dosen, tendik). Admin dapat CRUD *intent*, *entity*, dan utterance.	- *Intent* & *response* management Rasa. - Web admin untuk melihat *fallback* log.

Hasil analisis perbandingan tersebut memberikan rekomendasi bahwa *chatbot* FSM perlu melakukan *hybrid* sistem RAG dan Rasa untuk memperluas kemampuan pencarian kontekstual, menyediakan *dashboard* admin untuk memantau aktivitas *Chatbot*, serta mempertahankan fitur penyimpanan riwayat percakapan sebagai fungsi penting dalam proses audit dan peningkatan model.

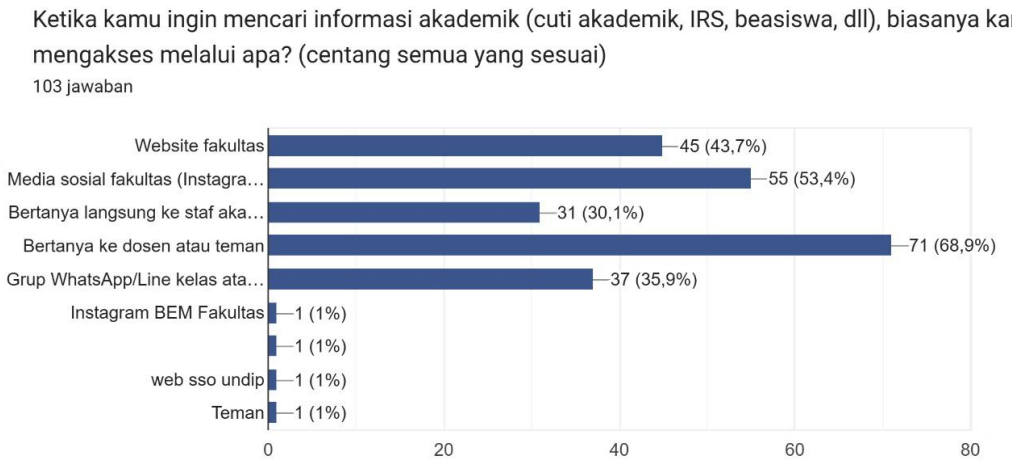
4.2 Survei Kebutuhan Pengguna



Gambar 4. 1 Diagram Sebaran Departemen Responeden

Survei dilakukan secara daring menggunakan formulir *Google Form* dan diikuti oleh 103 responden dari enam departemen di FSM. Mayoritas responden berasal dari Departemen Informatika (58 responden; 56,3%), diikuti oleh Statistika (12 responden; 11,7%), serta proporsi yang merata dari departemen Kimia, Matematika, Biologi, dan Fisika (masing-masing di bawah 10%). Partisipasi dari berbagai departemen ini diharapkan memberikan gambaran komprehensif mengenai kebutuhan informasi akademik di lingkungan fakultas.

Kemudian diberikan pertanyaan kepada para mahasiswa untuk mengetauhi perilaku saat ini mahasiswa dalam mecari informasi akademik. Dapat dilihat pada gambar 4.2 bahwa metode pencarian informasi saat ini yang tersebar di website dan media sosial dinilai kurang efektif, dengan kendala utama sulit menemukan halaman relevan (53,4%) dan informasi tidak diperbarui (55,3%).

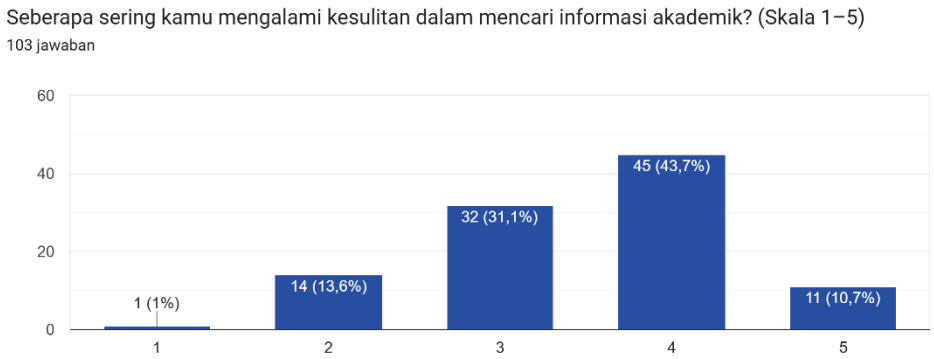


Gambar 4.2 Diagram Jawaban Kebiasaan Akses Mahasiswa terhadap Informasi Akademik di FSM Undip

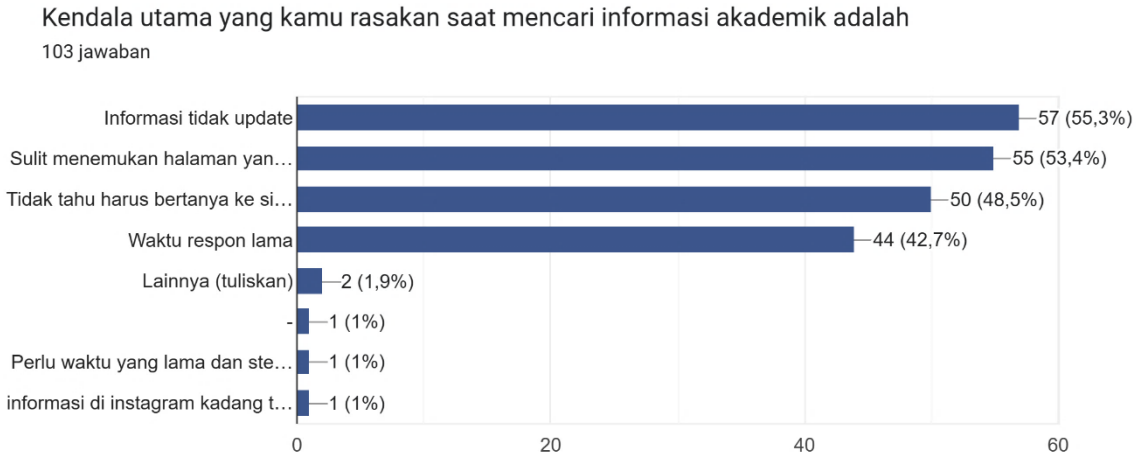
Analisis Kebiasaan Akses Informasi Mahasiswa Berdasarkan hasil survei yang ditunjukkan pada Gambar 4.2, dari total 103 responden mahasiswa FSM UNDIP, terlihat bahwa metode pencarian informasi akademik masih didominasi oleh cara-cara manual dan informal. Mayoritas mahasiswa, sebesar 68,9% (71 responden), lebih memilih untuk "Bertanya ke dosen atau teman" ketika membutuhkan informasi. Angka ini jauh lebih tinggi dibandingkan penggunaan "Website fakultas" yang hanya dipilih oleh 43,7% (45 responden) mahasiswa. Selain itu, penggunaan media sosial fakultas (Instagram) juga cukup tinggi yaitu sebesar 53,4%. Data ini mengindikasikan bahwa mahasiswa cenderung lebih menyukai saluran komunikasi yang bersifat interaktif dan personal dibandingkan menelusuri informasi secara mandiri melalui website statis. Kecenderungan ini memvalidasi perlunya pengembangan sistem *chatbot* yang dapat meniru interaksi percakapan (seperti bertanya pada teman) namun dengan basis data resmi yang akurat.

Analisis Tingkat Kesulitan Pencarian Informasi Selanjutnya, Gambar 4.3 memperlihatkan persepsi mahasiswa terhadap tingkat kesulitan dalam mencari informasi akademik menggunakan Skala Likert 1 (Sangat Mudah) hingga 5 (Sangat Sulit). Hasil survei menunjukkan bahwa mayoritas responden merasa kesulitan dalam mengakses informasi yang valid. Sebanyak 43,7% (45 responden) memilih skala 4 (Sulit), dan 31,1% (32 responden) memilih skala 3 (Netral/Cukup Sulit). Hanya sebagian kecil mahasiswa (1%) yang merasa pencarian informasi sangat mudah. Jika diakumulasikan, lebih dari 85% responden merasakan adanya hambatan (skala 3, 4, dan 5) dalam mendapatkan informasi

akademik. Tingginya persentase kesulitan ini menjadi landasan kuat mengenai urgensi pengembangan Academic Chatbot sebagai solusi untuk mempermudah aksesibilitas informasi di lingkungan FSM UNDIP.



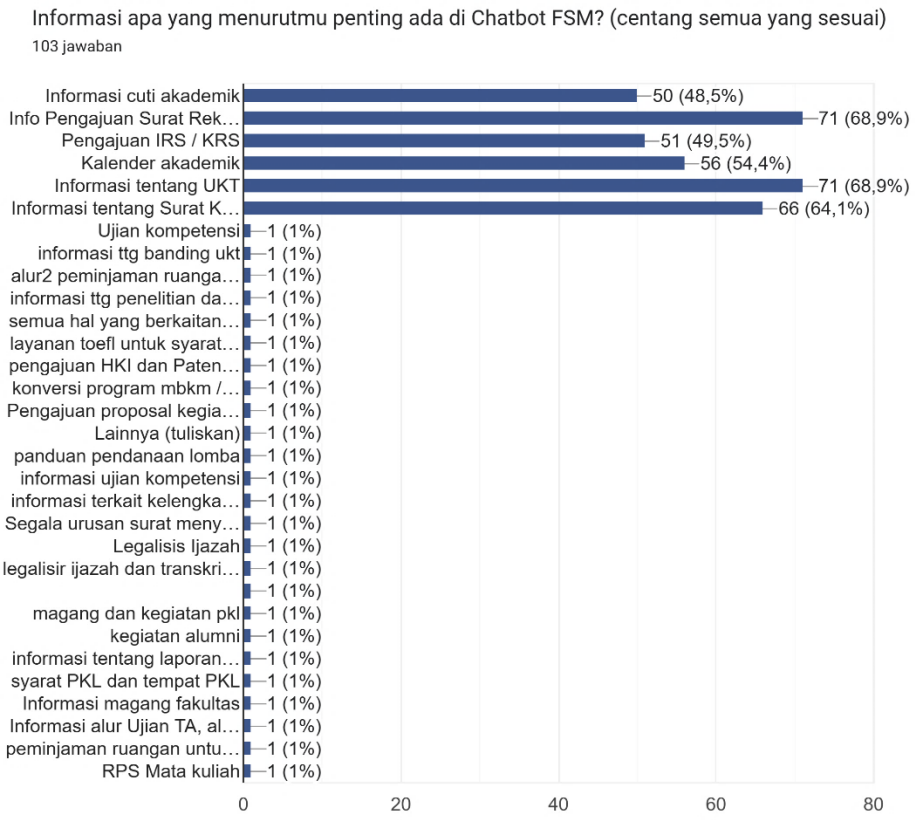
Gambar 4. 3 Skala Likert Seberapa Sulit mecari informasi akademik



Gambar 4.4 Diagram Kendala Utama Saat Mencari Informasi Akademik

Analisis Kendala Utama Pencarian Informasi Berdasarkan data pada Gambar 4.4, teridentifikasi empat kendala utama yang paling sering dialami mahasiswa. Kendala yang paling dominan adalah "Informasi tidak update", yang dikeluhkan oleh 55,3% (57 responden). Hal ini menunjukkan bahwa kanal informasi yang ada saat ini sering kali memuat data kedaluwarsa, yang memvalidasi urgensi penggunaan metode RAG agar jawaban chatbot selalu berbasis pada dokumen SOP terbaru. Selain itu, 53,4% (55 responden) merasa "Sulit menemukan halaman yang dituju" dan 48,5% (50 responden) mengaku "Tidak tahu harus bertanya ke siapa". Data ini mengindikasikan tingginya fragmentasi informasi di FSM UNDIP. Oleh karena itu, pengembangan chatbot sebagai

pintu gerbang informasi terpusat menjadi solusi krusial untuk meniadakan kebingungan mahasiswa dalam menavigasi birokrasi akademik.



Gambar 4.5 Diagram Survey Harapan Infromasi yang terrsedi pada *Chatbot FSM*

Terkait basis pengetahuan (*knowledge base*), mayoritas responden memprioritaskan informasi administratif rutin seperti Surat Keterangan Mahasiswa Aktif (64,1%), Informasi terkait UKT (68,9%), dan Info Beasiswa (68,9%). Selain kebutuhan dasar tersebut, responden juga secara spesifik meminta informasi mendalam mengenai alur Tugas Akhir (TA) dan Yudisium, prosedur Magang/PKL/MBKM, legalisir ijazah, hingga peminjaman ruangan dan pengajuan HKI. Keragaman kebutuhan informasi ini, ditambah dengan tingginya permintaan fitur riwayat percakapan (94,1%), menegaskan perlunya *chat*bot cerdas yang mampu menangani variasi topik akademik secara fleksibel.

4.3 Tahap *Requirements*

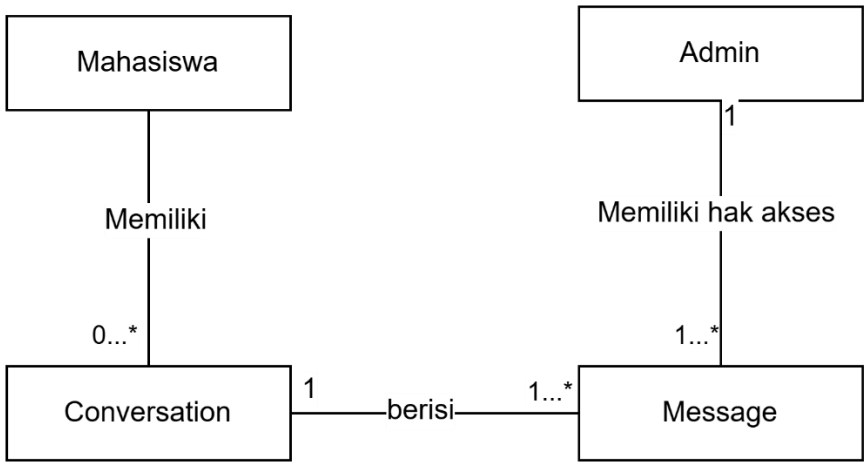
Tahap *requirements* dalam ICONIX *Process* bertujuan untuk mendefinisikan kebutuhan sistem secara rinci serta mengidentifikasi entitas utama dalam domain *Chatbot* FSM. Kebutuhan fungsional (*functional requirements*) mendefinisikan kemampuan yang harus dimiliki sistem, yaitu bagaimana sistem me*respons* tindakan pengguna dan menyediakan layanan sesuai fungsinya. Pada tahap ini dihasilkan *output* berupa *Software Requirements Spesification* (SRS) yang disajikan pada Tabel 4.2 dan merangkum seluruh kebutuhan fungsional utama *Chatbot* FSM.

Tabel 4.2 *Software Requirements Spesification* (SRS) *Chatbot* FSM

No	SRS-ID	Deskripsi
1	SRS-CB-F-01	Sistem harus menyediakan mengautentikasi pengguna (mahasiswa/ admin) menggunakan email dan kata sandi yang valid.
2	SRS-CB-F-02	Sistem harus menyediakan pengambilan *role* pengguna yang sedang *login* untuk keperluan personalisasi dan kontrol akses.
3	SRS-CB-F-03	Sistem harus menerapkan *role-based access control* (RBAC) untuk membedakan hak akses Mahasiswa dan Admin pada rute/fitur.
4	SRS-CB-F-04	Sistem harus memfasilitasi pengiriman pesan ke *Chatbot* dan menampilkan jawaban, dengan alur normal via Rasa dan *fallback* ke RAG saat *intent* tidak dikenali.
5	SRS-CB-F-05	Sistem harus menyimpan setiap pesan dan percakapan beserta metadata (sumber Rasa/RAG, *intent*, *confidence*, *latency*, *timestamp*) untuk audit dan analitik.
6	SRS-CB-F-06	Sistem harus menampilkan daftar percakapan milik pengguna yang sedang *login* (dengan dukungan *limit*/sortir terbaru).
7	SRS-CB-F-07	Sistem harus menampilkan riwayat pesan suatu percakapan yang dipilih pengguna.
8	SRS-CB-F-08	Sistem harus menyajikan *dashboard* analitik untuk Admin berisi total percakapan hari ini, total pesan, distribusi sumber (Rasa/RAG), dan tiga *intent* teratas.

9	SRS-CB-F-09	Sistem harus menyediakan *chat log* untuk Admin dengan kemampuan filter berdasarkan tanggal, pengguna, sumber (Rasa/RAG), dan *intent*.
10	SRS-CB-F-10	Sistem harus menyediakan pengelolaan data pengguna (CRUD) yang dapat diakses Admin.

4.3.1 *Domain Modeling*



Gambar 4. 6 Domain Model

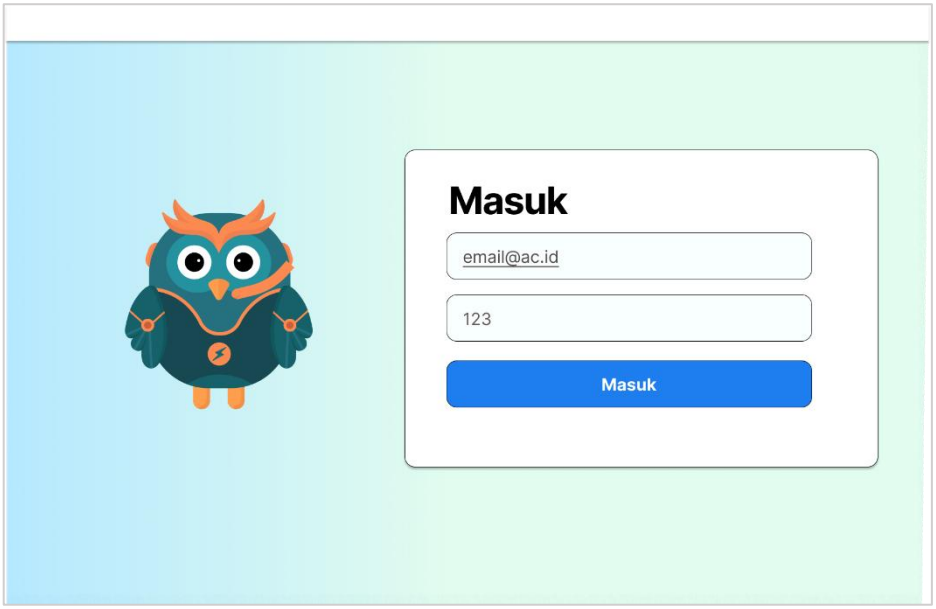
Berdasarkan analisis kebutuhan fungsional, diperoleh empat entitas utama dalam domain sistem. Mahasiswa mewakili aktor yang berinteraksi langsung dengan sistem, yaitu mahasiswa FSM UNDIP yang mengajukan pertanyaan atau mencari informasi akademik. *Conversation* merupakan representasi satu sesi percakapan antara pengguna dan sistem yang berfungsi menyimpan konteks alur dialog. *Message* adalah unit dasar dari percakapan yang berisi konten pesan, baik yang dikirim oleh pengguna maupun balasan dari *chatbot*. Selain itu, terdapat entitas Admin yang merepresentasikan pengelola sistem dengan wewenang hak akses terhadap data pesan. Hubungan antar entitas menunjukkan bahwa satu Mahasiswa dapat memiliki banyak *Conversation*, yang berarti sistem menyimpan riwayat sesi percakapan pengguna (relasi *one-to-many*). Selanjutnya, satu *Conversation* terdiri dari banyak *Message* yang membentuk struktur tanya-jawab. Terakhir, entitas Admin memiliki relasi hak akses terhadap *Message*, yang menggambarkan wewenang administrator untuk memantau log data pesan di *database* untuk keperluan evaluasi, tanpa terlibat langsung sebagai partisipan dalam percakapan tersebut.

4.3.2 Prototipe Antarmuka

Pada tahap ini dirancang prototipe antarmuka pengguna untuk menggambarkan interaksi utama antara pengguna dengan sistem. Prototipe antarmuka disusun menggunakan Figma dan terdiri dari dua peran pengguna. Prototipe ini menunjukkan penggambaran sederhana berupa desain sistem yang dibuat. Berikut adalah desain *Chatbot* FSM yang dibuat.

4.3.2.1 Desain Antarmuka Halaman *Login* (DA-CB-01)

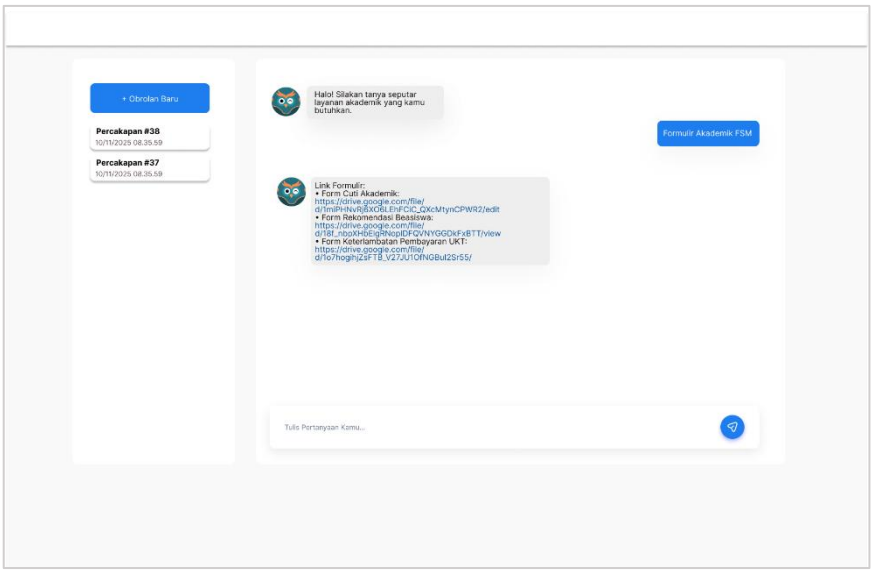
Desain antarmuka halaman *login* memberikan gambaran ketika pengguna mengakses halaman *login* untuk masuk ke sistem untuk melakukan autentikasi pengguna.



Gambar 4. 7 Desain Antarmuka Halaman Login

4.3.2.2 Desain Antarmuka Halaman Percakapan (DA-CB-02)

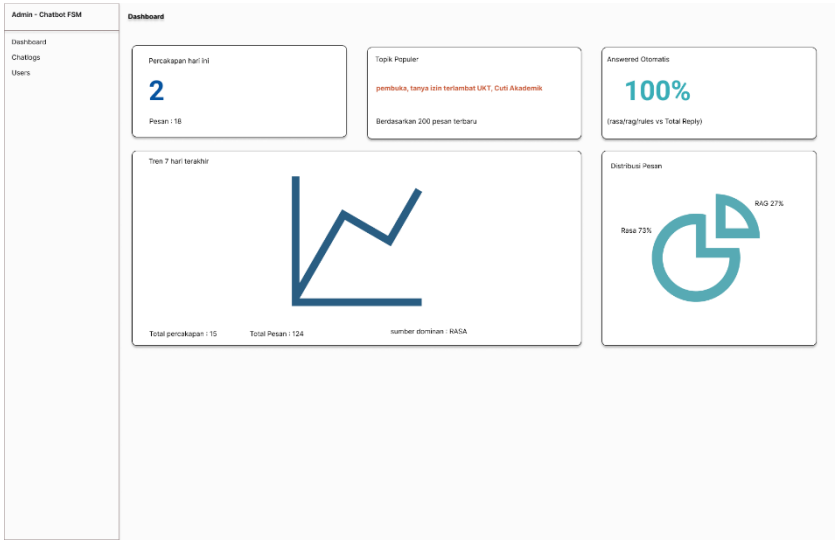
Desain antarmuka halaman percakapan menunjukkan gambaran ketika mahasiswa berhasil masuk kedalam sistem akan diarahkan ke halaman percakapan. Halaman ini terdiri dari beberapa bagian. Pada sebelah kiri terdapat *sidebar* yang menunjukkan panel histori percakapan yang dimiliki *user* dan terdapat juga tombol untuk memulai percakapan baru. Kemudian disebelah kanan terdapat area *chat* dimana adalah tempat mahasiswa berinteraksi dengan *Chatbot* untuk mengirim pertanyaan dan mendapatkan jawaban.



Gambar 4. 8 Desain Antarmuka Halaman Percakapan

4.3.2.3 Desain Antarmuka Halaman *Dashboard* Admin (DA-CB-03)

Desain antarmuka halaman *dashboard* admin menggambarkan ketika admin masuk kedalam sistem dan membuka menu *dashboard*. Pada halaman ini ditampilkan rangkuman analisa pesan dan percakapan yang berfungsi untuk memantau ada berapa pesan/percakapan, *top intent*, dan distribusi pesan.



Gambar 4. 9 Desain Antarmuka Halaman Dashboard Admin

4.3.2.4 Desain Antarmuka Halaman Menu *Chat*logs (DA-CB-04)

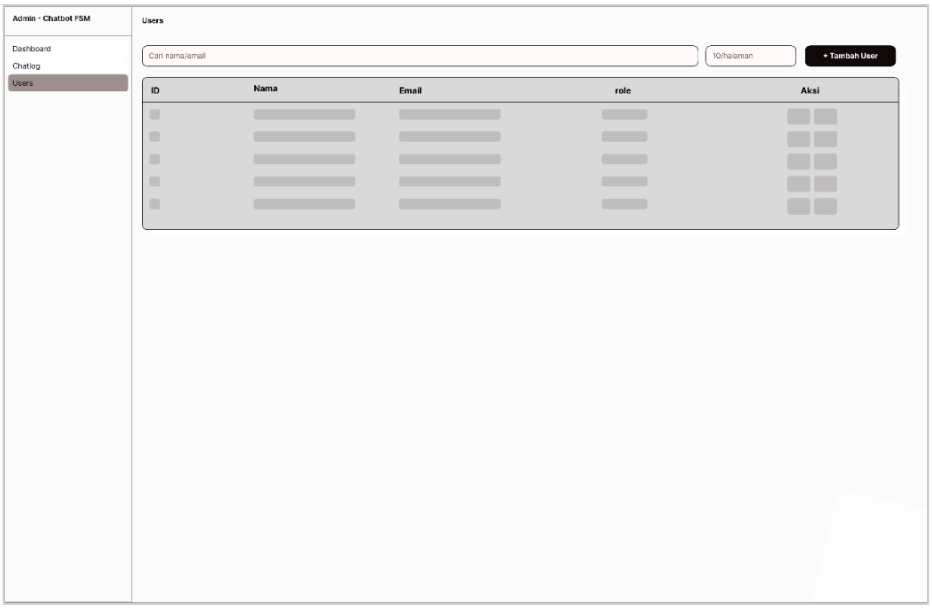
Desain antarmuka menu *chatlog* menggambarkan saat admin ingin melihat dan memfilter *chatlog* yang ada. Berisi Riwayat pesan dari *user* maupun bot beserta filter *chatlog* seperti *role*, sumber dan lain-lain.



Gambar 4. 10 Desain Antarmuka Halaman Menu Chatlogs

4.3.2.5 Desain Antarmuka Halaman Pengelolaan *User* (DA-CB-05)

Desain antarmuka halaman pengelolaan *user* menggambarkan ketika admin ingin melihat daftar *user*, menambahkan *user* baru, memperbarui *user* dan menghapus *user*.



Gambar 4.11 Desain Antarmuka Halaman Pengelolaan *User*

4.3.3 Daftar *Use case*

Peneliti melakukan identifikasi terhadap daftar *use case* pada sistem *Chatbot* yang dikembankan. Proses identidikasi tersebut melibatkan 2 aktor yaitu admin dan mahasiswa. Rincian lengkap daftar *use case* dapat dilihat pada tabel 4.3

Tabel 4.3 Daftar *Use case*

No	UC-ID	Nama *Use case*	Aktor
1	UC-CB-01	Autentikasi Pengguna	Mahasiswa / Admin
2	UC-CB-02	Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot*	Mahasiswa
3	UC-CB-03	Melihat Riwayat Pesan	Mahasiswa
4	UC-CB-04	Melihat *Dashboard* Analitik	Admin
5	UC-CB-05	Melihat dan Memfilter *Chat Logs*	Admin
6	UC-CB-06	Melihat Daftar Pengguna	Admin
7	UC-CB-07	Menambah Pengguna Baru	Admin
8	UC-CB-08	Memperbarui Pengguna	Admin
9	UC-CB-09	Menghapus Pengguna	Admin

4.3.4 *Use case* Diagram

Seluruh *use case* yang telah diidentifikasi ke dalam *use case* diagram setelah proses pendefinisian *use case* selesai dilakukan. *Use case* diagram disusun untuk memberikan gambaran visual mengenai interaksi antara masing-masing aktor dengan sistem. Diagram ini membantu dalam memahami batasan sistem, peran masing-masing aktor, serta keterkaitan antarproses yang terjadi di dalam *Chatbot* FSM. Visualisasi lengkap dari *use case* diagram dapat dilihat pada Gambar 4.11.



Gambar 4. 12 *Use Case* Diagram

4.3.5 *Use case Scenario*

Use case scenario digunakan untuk mengetauhi penjelasan naratif yang detail mengenai interaksi antara aktor dan sistem untuk setiap *use case* yang telah didefinisikan pada *use case* diagram.

1. *Use case Scenario* Autentikasi Pengguna

   Use case scenario autentikasi pengguna merupakan gambaran bagaimana aktor dapat mengakses *Chatbot* dengan memasukkan *user*name dan password.

Tabel 4.4 *Use case Scenario* Autentikasi Pengguna

Nama	Deskripsi
ID *Use case*	UC-CB-01
Mencakup SRS	SRS-CB-F-01 , SRS-CB-F-02, SRS-CB-F-03
Deskripsi	Aktor mendapatkan token autentikasi dan akses ke sistem sesuai peran.
Aktor	Admin , Mahasiswa
Kondisi Awal	Aktor berada pada halaman *login*
Skenario Utama	1. Pengguna memasukkan email dan password ke halaman *login* 2. Pengguna menekan ‘Masuk’ 3. Sistem memvalidasi email dan password 4. Sistem mengembalikan pesan sukses dan halaman *login* mengambil detail *user* 5. Jika *role* dari *user* adalah “admin”, sistem akan mengarahkan pengguna ke halaman *dashboard* admin 6. Jika *role* dari *user* adalah “mahasiswa” sistem akan mengarahkan pengguna ke halaman *chat*
Skenario Alternatif	1. Sistem gagal melakukan validasi email dan password 2. Endpoint mengembalikan status error 401 *Unauthorized* 3. Menampilkan email atau password salah

2. *Use case Scenario* Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot*

   Use case ini menjelaskan proses ketika mahasiswa melakukan interaksi inti dengan *Chatbot*. Mahasiswa yang sudah melakukan *login* dapat mengetikkan pesan pada halaman *chat* dan menekan tombol kirim. Sistem menampilkan pesan tersebut sebagai pesan dengan peran *user* pada area percakapan, kemudian memprosesnya menggunakan layanan Rasa untuk mendeteksi *intent*. Apabila *intent* tidak dikenal atau *confidence score* rendah, sistem melakukan *fallback* ke layanan RAG untuk menghasilkan jawaban berbasis pencarian pengetahuan. Setelah jawaban dihasilkan, sistem menyimpan *respons* beserta metadata seperti sumber jawaban, *intent*, *confidence*, dan *role*, kemudian menampilkan jawaban tersebut kepada mahasiswa. Pada skenario alternatif, apabila layanan Rasa maupun RAG tidak dapat diakses, sistem memberikan *respons default* berupa pesan "Maaf, layanan sedang bermasalah".

Tabel 4.5 *Use case Scenario* Mengirim Pesan *Chatbot*

Nama	**Deskripsi**
ID *Use case*	UC-CB-02
Deskripsi	Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot*
Mencakup SRS	SRS-CB-F-04, SRS-CB-F-05
Aktor	Mahasiswa
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Mahasiswa mengetikkan pesan ke input pesan di halam *chat* 2. Mahasiswa menekan tombol kirim 3. Halaman *chat* menampilkan pesan mahasiswa dengan *role user* di area *chat* 4. Sistem memproses pesan melalui Rasa. 5. Jika *intent* tidak dikenal, sistem melakukan *fallback* ke RAG.

Nama	Deskripsi
	6. Sistem menyimpan balasan bot beserta metadata (*source*, *intent*, *confidence*,*role*) 7. Halaman *chat* menampilkan jawaban di area *chat*.
Skenario Alternatif	Layanan Rasa dan RAG Tidak Tersedia 1. Sistem gagal memanggil Rasa dan RAG. 2. Sistem menghasilkan balasan default: "Maaf, layanan sedang bermasalah." 3. Sistem menampilkan balasan tersebut di area *chat*.

3. *Use case Scenario* Melihat Riwayat Pesan

Use case ini menggambarkan bagaimana mahasiswa dapat melihat riwayat pesan sebelumnya setelah berhasil *login*. Ketika berada pada halaman *chat*, sistem secara otomatis mengambil maksimal sepuluh percakapan terakhir milik mahasiswa. Daftar *Conversation* tersebut kemudian ditampilkan pada area sidebar agar mahasiswa dapat memilih dan melanjutkan percakapan tertentu. Pada skenario alternatif, apabila belum ada riwayat percakapan, sistem mengembalikan daftar kosong dan menampilkan pesan "Belum ada percakapan."

Tabel 4.6 *Use case Scenario* Melihat Riwayat Pesan

Nama	Deskripsi
ID *Use case*	UC-CB-03
Deskripsi	Melihat Riwayat Pesan
Mencakup SRS	SRS-CB-F-06 dan SRS-CB-F-07
Aktor	Mahasiswa
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Mahasiswa berada di halaman *chat*

Nama	Deskripsi
	2. Sistem mengambil maksimal 10 percakapan terakhir milik pengguna. 3. Halaman *chat* menampilkan daftar *Conversations* di sidebar pada halaman *chat* 4. Mahasiswa memilih salah satu item percakapan di sidebar riwayat pada halaman *chat* 5. Sistem mengambil semua pesan dalam percakapan tersebut. 6. Halaman *chat* menampilkan riwayat pesan pada area *chat*
Skenario Alternatif	Apa bila belum ada riwayat percakapan 1. Sistem mengembalikan daftar kosong. 2. Sistem menampilkan pesan "Belum ada percakapan."

4. *Use case Scenario* Melihat *Dashboard* Analitik

   Use case ini menjelaskan bagaimana admin mengakses *dashboard* analitik setelah *login*. Admin memilih menu *dashboard* melalui navigasi, kemudian sistem mengarahkan admin ke halaman *dashboard*. *Backend* memproses data percakapan dan pesan untuk menghasilkan informasi statistik, seperti jumlah percakapan hari ini, total pesan, tren tujuh hari terakhir, topik populer, persentase jenis balasan, serta distribusi pesan berdasarkan *role* dan sumber. Sistem mengembalikan data tersebut ke antarmuka dan *dashboard* menampilkannya dalam bentuk kartu statistik dan grafik interaktif. *Use case* ini tidak memiliki skenario alternatif.

Tabel 4.7 *Use case Scenario* Melihat *Dashboard* Analitik

Nama	Deskripsi
ID *Use case*	UC-CB-04
Deskripsi	Melihat *Dashboard* Analitik
Mencakup SRS	SRS-CB-F-08

Nama	Deskripsi
Aktor	Admin
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin menekan menu navigasi *dashboard* 2. Sistem mengarahkan ke halaman *dashboard* 3. Sistem memproses data dari messages dan *Conversations* untuk menghasilkan analitik 4. Sistem mengembalikan data analitik 5. Halaman *dashboard* merender data analitik ke kartu statistik percakapan hari ini, grafik tren 7 hari, Topik populer, persentasi total balasan otomatis, dan distribusi pesan
Skenario Alternatif	Tidak Ada

5. *Use case Scenario* Melihat dan Memfilter *Chat Logs*

   Use case ini menjelaskan proses admin dalam meninjau dan memfilter seluruh log percakapan. Setelah membuka halaman *chat* logs, admin dapat memasukkan kriteria filter seperti isi pesan (*query*), *role* (*user*/*assistant*), sumber jawaban (Rasa atau RAG), atau rentang tanggal tertentu. Sistem kemudian mengambil data message yang sesuai dengan filter tersebut dan memperbarui tampilan tabel pada halaman untuk menampilkan hasilnya. Pada skenario alternatif, apabila tidak ditemukan log yang sesuai, sistem mengembalikan daftar kosong tanpa error.

Tabel 4.8 *Use case Scenario* Memfilter *Chat Logs*

Nama	Deskripsi
ID *Use case*	UC-CB-05
Deskripsi	Melihat dan Memfilter *Chat Logs*

Nama	Deskripsi
Mencakup SRS	SRS-CB-F-09
Aktor	Admin
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin membuka halaman *chat* logs 2. Admin memasukkan kriteria filter misal isi pesan, *role*, *source*, atau rentang tanggal 3. Sistem mengambil messages yang cocok dengan filter criteria 4. Halaman *Chat* Log memperbarui tabel untuk menampilkan pesan yang telah difilter
Skenario Alternatif	Log Tidak Ditemukan 1. Sistem (*back*end) mengembalikan daftar *Chat*LogItem kosong.

6. *Use case Scenario* Melihat Daftar Pengguna

   Use case ini menjelaskan bagaimana admin mengakses daftar seluruh pengguna yang terdaftar. Ketika admin menekan menu navigasi "*User*s", sistem mengarahkan ke halaman pengguna lalu mengambil seluruh data *user*s dari database. Halaman kemudian merender data tersebut ke dalam tabel pengguna. Jika database tidak memiliki data pengguna, sistem mengembalikan daftar kosong.

Tabel 4.9 *Use case Scenario* Melihat Daftar Pengguna

Nama	Deskripsi
ID *Use case*	UC-CB-06
Deskripsi	Melihat Daftar Pengguna
Mencakup SRS	SRS-CB-F-10
Aktor	Admin

Nama	Deskripsi
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin menekan menu navigasi '*User*s'. 2. Sistem mengarahkan ke Halaman*User*s. 3. Sistem mengambil daftar *user*s dari database. 4. Halaman*User*s menampilkan daftar *user*s di Tabel *User*s.
Skenario Alternatif	Tidak Ada Pengguna 1. Sistem mengembalikan daftar kosong.

7. *Use case Scenario* Menambah Pengguna Baru

   Use case ini menggambarkan proses admin dalam menambahkan akun pengguna baru. Admin menekan tombol Tambah *User* sehingga sistem menampilkan FormTambah*User* berbentuk modal. Admin mengisi data seperti nama, email, password, dan *role*, kemudian menekan tombol Simpan. Sistem menyimpan data tersebut ke database dan menutup modal setelah proses berhasil. Pada skenario alternatif, apabila email yang dimasukkan sudah terdaftar, *back*end mengembalikan error 400 dan antarmuka menampilkan pesan “Email sudah terdaftar.”

Tabel 4.10 *Use case Scenario* Menambah Pengguna Baru

Nama	Deskripsi
ID *Use case*	UC-CB-07
Deskripsi	Menambah Pengguna Baru
Mencakup SRS	SRS-CB-F-10
Aktor	Admin
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin menekan Tombol Tambah*User* di Halaman*User*s.

Nama	Deskripsi
	2. Sistem menampilkan FormTambah*User* (Modal). 3. Admin memasukkan data *UserCreate*In (Nama, Email, Password, *Role*) ke FormTambah*User*. 4. Admin menekan tombol 'Simpan'. 5. Sistem membuat *record users* baru di database. 6. Sistem menutup *Form*
Skenario Alternatif	Email Sudah Terdaftar 1. Sistem (*back*end) mendeteksi duplikasi email. 2. *Endpoint* POST /*user*s mengembalikan *error* (misal: *400 Bad Request*). 3. FormTambah*User* menampilkan pesan "Email sudah terdaftar".

8. *Use case Scenario* Memperbarui Pengguna

   Use case ini menjelaskan proses admin dalam memperbarui informasi akun pengguna. Admin memilih ikon edit pada baris pengguna tertentu dan sistem menampilkan FormEdit*User* dengan data yang sudah terisi sebelumnya. Admin dapat mengubah data seperti email atau password, kemudian menekan tombol *Update*. Sistem memperbarui data pada database dan memuat ulang tabel pengguna. Pada skenario alternatif, apabila terjadi kesalahan validasi seperti format email tidak valid atau password terlalu pendek, GUI menampilkan pesan kesalahan validasi.

Tabel 4.11 *Use case Scenario* Memperbarui Pengguna

Nama	Deskripsi
ID *Use case*	UC-CB-08
Deskripsi	Memperbarui Pengguna
Mencakup SRS	SRS-CB-F-10
Aktor	Admin

Nama	Deskripsi
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin menekan TombolEdit pada baris *user*s di Tabel*User*s. 2. Sistem (GUI) menampilkan FormEdit*User* (Modal) yang sudah terisi data *user*s. 3. Admin mengubah data *User* (misal: mengubah 'email' atau 'password') 4. Admin menekan tombol '*Update*'. 5. Sistem memperbarui *record users* di database. 6. Sistem (GUI) menutup FormEdit*User* dan memuat ulang Tabel*User*s
Skenario Alternatif	Validasi Gagal 1. Sistem menemukan field tidak valid (email salah format, password terlalu pendek). 2. Sistem menampilkan pesan validasi gagal.

9. *Use case Scenario* Menghapus Pengguna

   Use case ini mendeskripsikan bagaimana admin menghapus akun pengguna. Admin menekan ikon hapus pada daftar pengguna, kemudian sistem menampilkan dialog konfirmasi. Setelah admin menekan tombol Hapus, sistem menghapus data pengguna dari database dan baris pengguna pada tabel juga dihapus. Pada skenario alternatif, jika admin memilih batal, sistem menutup dialog tanpa melakukan perubahan.

Tabel 4.12 *Use case Scenario* Menghapus Pengguna

Nama	Deskripsi
ID *Use case*	UC-CB-09
Deskripsi	Menghapus Pengguna
Mencakup SRS	SRS-CB-F-10

Nama	Deskripsi
Aktor	Admin
Kondisi Awal	Aktor sudah melakukan *login*
Skenario Utama	1. Admin menekan TombolHapus (ikon tempat sampah) pada baris *user*s di Tabel*User*s. 2. Sistem menampilkan dialog konfirmasi "Aksi ini tidak bisa dibatalkan. Yakin ingin menghapus *user*?". 3. Admin menekan 'Hapus'. 4. Sistem menghapus *record users* dari database. 5. Sistem (GUI) menghapus baris *user*s dari Tabel*User*s.
Skenario Alternatif	Admin Membatalkan Aksi 1. Admin menekan “Batal” pada dialog. 2. Sistem menutup dialog tanpa melakukan perubahan.

4.4 *Milestone* 1: *Requirements Review*

Requirements review pada ICONIX *Process* bertujuan untuk memastikan bahwa kebutuhan sistem yang telah didefinisikan pada tahap *requirements* telah dipahami dengan jelas dan konsisten oleh seluruh *stakeholder*. Pada penelitian ini, Requirements *Review* dilakukan melalui dua aktivitas utama, yaitu validasi GUI storyboard bersama stakeholder dan *cross-artifact Checking* terhadap seluruh artefak kebutuhan.

Validasi dilakukan pada tanggal 10 November 2025 dimana hasilnya dilampirkan pada Lampiran 1. Dilakukan bersama supervisor subbagian akademik dan kemahasiswaan FSM UNDIP. Pemilihan supervisor sebagai validator didasarkan pada peran strategis beliau sebagai *stakeholder* yang memahami secara mendalam kebutuhan layanan akademik serta jenis pertanyaan yang paling sering diajukan oleh mahasiswa sehari-hari. Proses ini bertujuan memastikan bahwa prototipe antarmuka sesuai kebutuhan pengguna dan menggambarkan alur kerja sistem dengan benar. *Stakeholder* meninjau beberapa hal meliputi alur percakapan pada halaman *chatbot*, tampilan pesan dan interaksi pengguna, kejelasan alur navigasi, kesesuaian antarmuka dengan *usecase,* dan kesesuaian fitur dengan

kebutuhan bagian akademik FSM UNDIP. Hasil validasi menunjukkan bahwa tampilan antarmuka secara umum dianggap sudah memenuhi kebutuhan, dan mudah dipahami oleh pengguna. Hasil validasi dapat dilihat pada Lampiran 1.

Selanjutnya, dilakukan validasi desain antarmuka yang melibatkan pengguna akhir, yaitu mahasiswa, guna memverifikasi aspek kenyamanan (*usability*) dan pengalaman pengguna (*user experience*) sebelum masuk ke tahap implementasi. Validasi dilakukan dengan menyebarkan kuesioner berbasis Skala Likert (1-5) kepada 5 (lima) responden mahasiswa FSM UNDIP yang mencakup 8 (delapan) indikator utama, yaitu: keterbacaan teks, tata letak (*layout*), desain ikon, kemudahan navigasi *sidebar*, aksesibilitas riwayat percakapan, kemudahan akses *input*, kenyamanan visual, serta kesesuaian dengan ekspektasi layanan setelah mereka melakukan uji coba pada purwarupa interaktif di Figma.

Berdasarkan hasil rekapitulasi penilaian dari 5 (lima) responden mahasiswa FSM UNDIP, desain antarmuka memperoleh skor rata-rata keseluruhan sebesar 4,48 yang masuk dalam kategori Sangat Baik. Indikator fungsional seperti navigasi dan aksesibilitas fitur utama memperoleh skor tertinggi (4,80), sedangkan aspek kenyamanan visual berada pada kategori Baik (4,00 – 4,20).Detail hasil rekapitulasi penilaian validasi mahasiswa selengkapnya dapat dilihat pada Lampiran 3.

Selain penilaian kuantitatif, responden memberikan masukan kualitatif yang konstruktif. Terkait saran mengenai ketersediaan mode gelap (*Dark Mode*) untuk kenyamanan mata, fitur tersebut sesungguhnya telah diakomodasi dalam rancangan sistem dan siap digunakan. Adapun masukan visual lainnya penyatuan elemen maskot dan formulir pada halaman *login* agar lebih padat (*compact*), penambahan *padding* pada balon pesan (*chat bubble*) agar teks lebih nyaman dibaca, serta standarisasi jarak antar elemen (*spacing*) telah dirangkum dan dijadikan acuan standar visual (*style guide*) untuk diterapkan langsung pada tahap implementasi kode guna meningkatkan estetika dan konsistensi tampilan aplikasi

Setelah proses validasi GUI dilakukan, dilanjutkan dengan *cross-artifact-Checking* sesuai rekomendasi ICONIX *Process*. Artefak yang diperiksa mencakup SRS, domain model, *use case,* dan desain antarmuka. Tujuan dari pemeriksaan ini untuk memastikan bahwa setiap kebutuhan memiliki representasi pada *use case,* setiap *use case* telah divisualisasikan pada desain antarmuka, dan domain model mencerminkan objek yang

terlibat dalam *use case*. Untuk memperjelas keterlacakan antar artefak, dibuat tabel yang menunjukkan *cross-artifact-Checking* pada Tabel 4.13.

Tabel 4.13 *Cross-artifact-Checking* Tahap *Requirements Review*

No.	SRS	*Use case*	Domain Model	Desain antarmuka
1	SRS-CB-F-01 Autentikasi pengguna	UC-CB-01 Autentikasi Pengguna	*User*	DA-CB-01 Halaman *Login*
2	SRS-CB-F-02 Pengambilan *role* pengguna	UC-CB-01 Autentikasi Pengguna	*User*	DA-CB-01 Halaman *Login* (*role* digunakan untuk *redirect* mahasiswa/admin)
3	SRS-CB-F-03 *Role-Based Access Control* (RBAC)	UC-CB-01 Autentikasi Pengguna	*User*	DA-CB-02 Halaman Percakapan (Mahasiswa) & DA-CB-03 Dashboard Admin
4	SRS-CB-F-04 Pengiriman pesan *chatbot* dengan Rasa dan *fallback* RAG	UC-CB-02 Mengirim Pesan *Chatbot* dan Menerima Jawaban	*User, Conversation, Message*	DA-CB-02 Halaman Percakapan
5	SRS-CB-F-05 Penyimpanan pesan dan metadata	UC-CB-02 Mengirim Pesan Chatbot dan Menerima Jawaban	*Conversation, Message*	DA-CB-02 Halaman Percakapan (Riwayat Chat)
6	SRS-CB-F-06 Menampilkan daftar percakapan pengguna	UC-CB-03 Melihat Riwayat Pesan	*User, Conversation*	DA-CB-02 Sidebar Riwayat Percakapan

No.	SRS	*Use case*	Domain Model	Desain antarmuka
7	SRS-CB-F-07 Menampilkan riwayat pesan suatu percakapan	UC-CB-03 Melihat Riwayat Pesan	*User, Conversation, Message*	DA-CB-02 Area Percakapan
8	SRS-CB-F-08 Dashboard analitik admin	UC-CB-04 Melihat Dashboard Analitik	*Conversation, Message*	DA-CB-03 Dashboard Admin
9	SRS-CB-F-09 *Chat logs* dan filter	UC-CB-05 Melihat dan Memfilter *Chat Logs*	*User, Message, Conversation*	DA-CB-04 Halaman *Chat Logs*
10	SRS-CB-F-10 Pengelolaan pengguna (CRUD)	UC-CB-06, UC-CB-07, UC-CB-08, UC-CB-09	User	DA-CB-05 Halaman Pengelolaan Pengguna

Tabel 4.13. menunjukkan *cross-artifact-Checking* yang digunakan untuk menunjukkan keterlacakan antara kebutuhan fungsional (SRS), *use case*, domain model, dan rancangan antarmuka pada *chatbot* FSM. Berdasarkan tabel tersebut, seluruh kebutuhan fungsional SRS-CB-F-01 hingga SRS-CB-F-10 telah dipetakan ke *use case* yang relevan (UC-CB-01 sampai UC-CB-09). Setiap *use case* memanfaatkan objek domain yang sesuai, seperti *User*, *Conversation* , dan Message, yang memastikan bahwa model konseptual sistem mendukung kebutuhan yang ditetapkan. Selain itu, setiap kebutuhan juga telah divisualisasikan pada desain antarmuka, seperti halaman *login*, halaman percakapan, *dashboard* admin, *chat* logs, serta modul pengelolaan pengguna. Dengan demikian, tahap ini membuktikan bahwa artefak *requirements* telah konsisten satu sama lain dan tidak terdapat kebutuhan yang terlewat (*no missing requirement*). Tahap ini juga memastikan bahwa proses analisis telah memenuhi prinsip ICONIX *Process*, yaitu menjaga konsistensi dan keterlacakan kebutuhan melalui hubungan yang kuat antara SRS, *use case*, domain model, dan rancangan antarmuka sebelum memasuki tahap desain selanjutnya.

4.5 Tahap Analysis/*Preliminary Design*

Tahap *analysis and design* merupakan inti dari ICONIX *Process*. Pada tahap ini, kebutuha sistem yang sebelumnya telah diidentifikasi melalui *domain modeling* dan *use case* diterjemahkan ke dalam desain yang lebih rinci dan siap diimplementasikan. Menurut (Rosenberg dkk., 2007) analisis dan desain harus tetap berorientasi pada *use case,* sehingga setiap artefak desain memiliki keterhubungan yang jelas dengan fungsionalitas yang benar-benar dibutuhkan oleh pengguna. Tahap ini berfungsi sebagai jembatan antara spesifikasi kebutuhan apa yang harus dilakukan oleh sistem dan bagaimana kita membangun sistem. Langkah-langkah yang dapat dilakukan adalah melakukan *robustness analysis* untuk memvalidasi alur setiap *use case*. Domain model juga diperbarui berdasarkan hasil validasi kebutuhan dan pemahaman yang lebih lengkap terhadap sistem. Domain Model yang diperbarui berfungsi untuk memetakan konsep-konsep inti, entitas penting, serta hubungan antarobjek yang menggambarkan ruang permasalahan.

4.5.1 *Robustness Analysis*

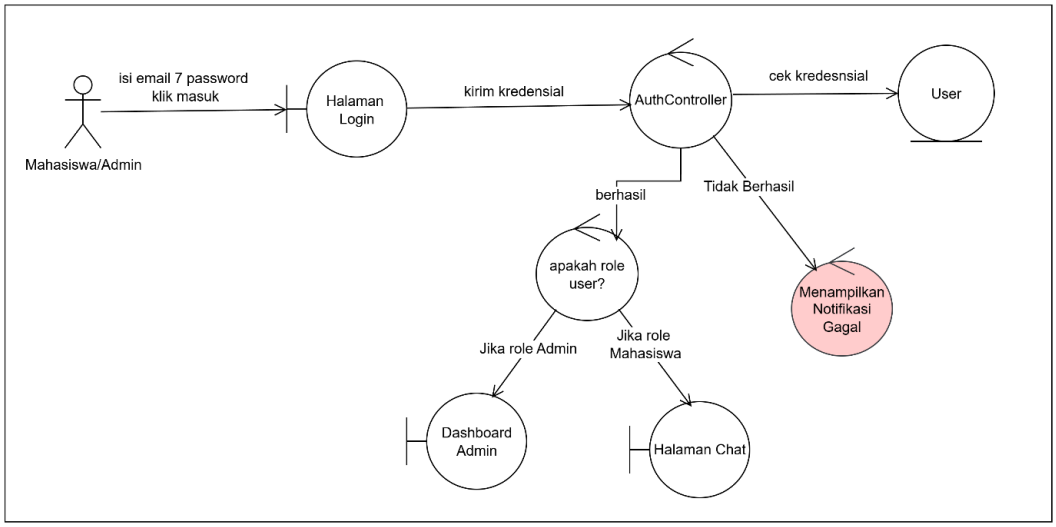
Robustness Analysis merupakan tahapan khas dalam ICONIX *Process* yang berfungsi untuk memvalidasi alur setiap *use case* dengan memetakan interaksi antara *boundary, control*, dan *entity*. Tahap ini memastikan bahwa setiap langkah pada *use case* dapat direalisasikan secara logis oleh komponen sistem, sekaligus mencegah terjadinya ambiguitas sebelum masuk ke tahap desain rinci. Berikut adalah penjelasan masing-masing robustness diagram pada sistem *chatbot* FSM.

4.5.1.1 *Robustness Diagram* Autentikasi Pengguna (RD-CB-01)

Robustness diagram Autentikasi Pengguna menggambarkan alur interaksi antara aktor, *boundary, control*, dan *entity* ketika pengguna melakukan proses *login*. Aktor (Mahasiswa/Admin) mengisi email dan password pada *Halaman Login* sebagai *boundary*, kemudian halaman ini meneruskan kredensial ke *AuthController* sebagai objek *control* yang menangani logika autentikasi. *AuthController* berinteraksi dengan entitas *User* untuk melakukan pengecekan email dan *password*.

Jika validasi gagal, *AuthController* mengembalikan *respons error* dan *boundary* menampilkan notifikasi gagal *login*. Jika validasi berhasil, *AuthController* memeriksa peran

(*role*) pengguna untuk menentukan halaman tujuan. Pengguna dengan *role* admin diarahkan ke *Dashboard* Admin, sedangkan pengguna dengan *role mahasiswa* diarahkan ke Halaman *Chat*. Diagram ini memastikan alur *login* tetap berada pada level konseptual dan mengikuti aturan *boundary–control–entity* sesuai metode ICONIX, sebagaimana ditunjukkan pada Gambar 4.13.



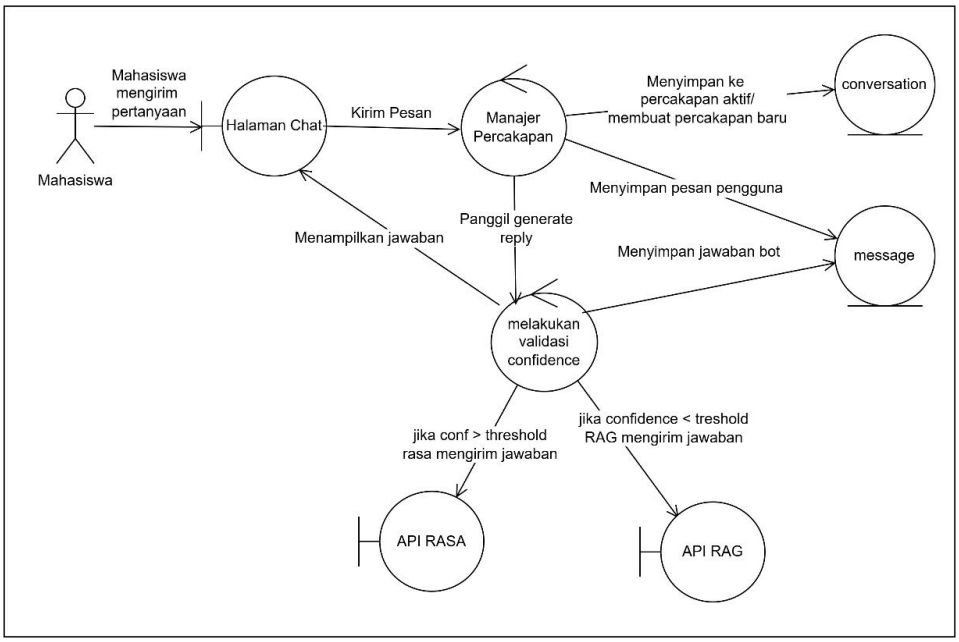
Gambar 4. 13 Robustness Diagram Autentikasi Pengguna (RD-CB-01)

4.5.1.2 *Robustness Diagram* Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot* (RD-CB-02)

Robustness diagram Mengirim Pesan *Chatbot* dan Menerima Jawaban dari *Chatbot* menggambarkan alur interaksi antar actor, *boundary, control,* dan *entity* ketika mahasiswa berkomunikasi dengan *chatbot* melalui halaman *chat*. Mahasiswa sebagai actor mengetikkan pesan pada halaman *chat* dan mengirimkannya, kemudian *boundary* meneruskan pesan tersebut kepada manajer percakapan, sebagai *control* yang bertanggung jawab mengelola percakapan. *Control* ini menyimpan pesan pengguna ke entitas *Conversation* dan *message*, serta meneruskan isi pesan ke pengolah balasan sebagai *control* yang menangani proses pemanggilan layanan Rasa dan RAG.

Pengolah balasan kemudian memanggil API RASA sebagai *boundary* eksternal untuk menganalisis *intent* dan tingkat *confidence* dari pesan pengguna. Jika proses pemanggilan Rasa berhasil dan *confidence* melebihi ambang batas, sistem menggunakan jawaban dari Rasa sebagai *respons*. Namun, apabila *confidence* rendah, *intent* tidak dikenali, atau Rasa tidak dapat diakses, *control* akan melakukan mekanisme *fallback* dengan

memanggil API RAG untuk mengambil jawaban berdasarkan dokumen-dokumen akademik yang tersedia. Jika RAG berhasil merespons, jawaban tersebut dikirim kembali ke *control* untuk diproses lebih lanjut. Selanjutnya, jawaban tersebut diteruskan kembali ke halaman *chat* sebagai *boundary* untuk ditampilkan kepada pengguna. Diagram ini memperlihatkan alur utama maupun alur alternatif secara konseptual sesuai dengan aturan *boundary–control–entity* dari ICONIX, sebagaimana ditunjukkan pada Gambar 4.14.



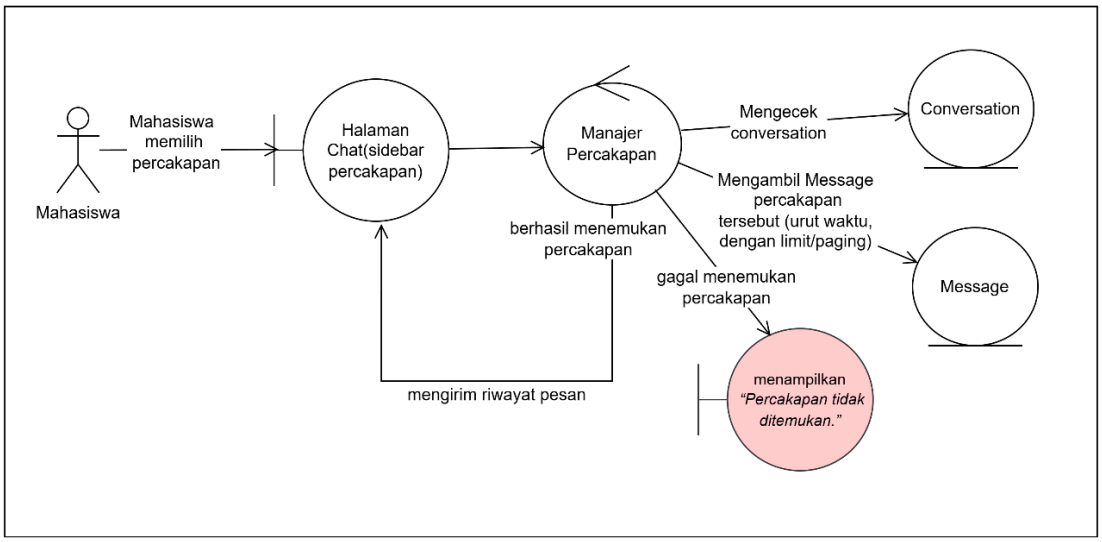
Gambar 4. 14 Robustness Diagram Proses Mengirim Pesan Chatbot dan Menerima Jawaban dari Chatbot (RD-CB-02)

4.5.1.3 *Robustness Diagram* Melihat Riwayat Pesan (RD-CB-03)

Robustness diagram melihat riwayat pesan menggambarkan alur interaksi ketika mahasiswa membuka halaman *chat* untuk melihat riwayat percakapan yang pernah dilakukan. Proses dimulai ketika mahasiswa mengakses halaman *chat*, kemudian *boundary* Halaman Chat meneruskan permintaan kepada Manajer Percakapan sebagai *control* untuk mengambil daftar percakapan milik pengguna melalui entitas Conversation dengan batasan jumlah tertentu. Selanjutnya, *control* juga mengambil riwayat pesan dari percakapan yang dipilih melalui entitas *Message* untuk dikembalikan ke halaman *chat* dan ditampilkan pada area percakapan.

Pada skenario alternatif, apabila tidak ditemukan percakapan yang sesuai, *control* mengembalikan *respons* ke *boundary* untuk menampilkan pesan “Percakapan tidak ditemukan.” kepada pengguna. *Robustness* diagram ini menunjukkan pemisahan peran

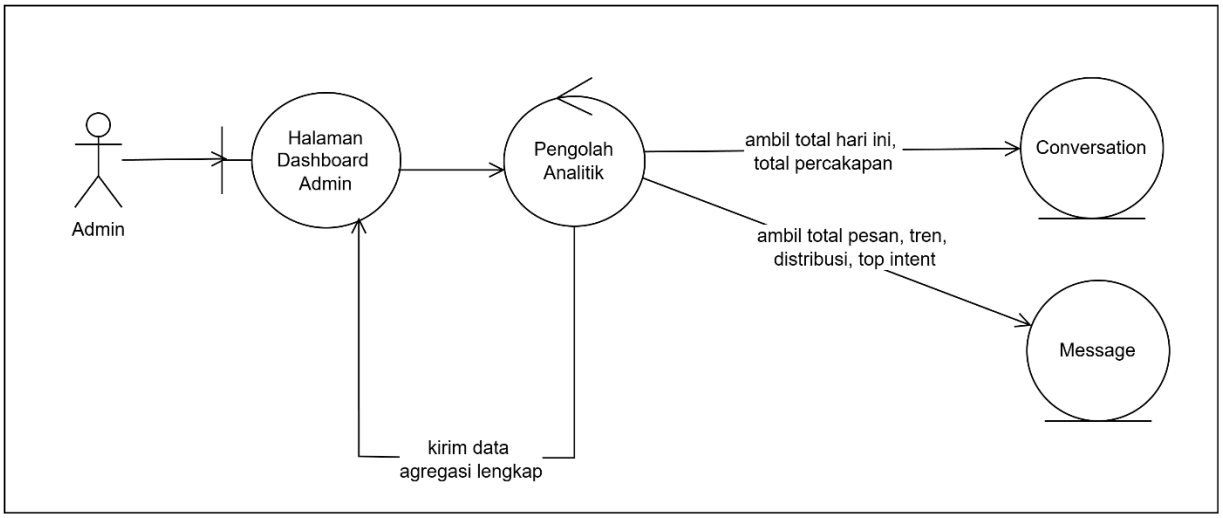
antara *boundary, control*, dan *entity* secara jelas sesuai dengan prinsip ICONIX Process, serta memastikan bahwa seluruh alur dalam use case Melihat Riwayat Pesan telah dimodelkan secara konsisten, sebagaimana ditunjukkan pada Gambar 4.15.



Gambar 4.15 *Robustness* Diagram Melihat Riwayat Pesan (RD-CB-03)

4.5.1.4 *Robustness Diagram* Melihat *Dashboard* Analitik (RD-CB-04)

Robustness diagram melihat *dashboard* analitik menggambarkan alur ketika admin mengakses halaman *dashboard* untuk melihat ringkasan statistik percakapan. Setelah admin memilih menu *dashboard*, halaman *dashboard* admin sebagai *boundary* mengirim permintaan pemuatan data ke pengolah analitik sebagai *control*. *Control* ini melakukan pengolahan data dengan mengambil informasi dari entitas *Conversation* dan *message*, seperti jumlah percakapan hari ini, total pesan, tren tujuh hari terakhir, topik populer, serta distribusi berdasarkan jenis balasan dan sumber jawaban. Hasil agregasi tersebut kemudian dikembalikan ke halaman *dashboard* dalam bentuk data siap tampil yang akan dirender sebagai kartu statistik dan grafik analitik. *Use case* ini tidak memiliki skenario alternatif sehingga proses berjalan sepenuhnya pada alur utama, sebagaimana ditunjukkan pada Gambar 4.16.

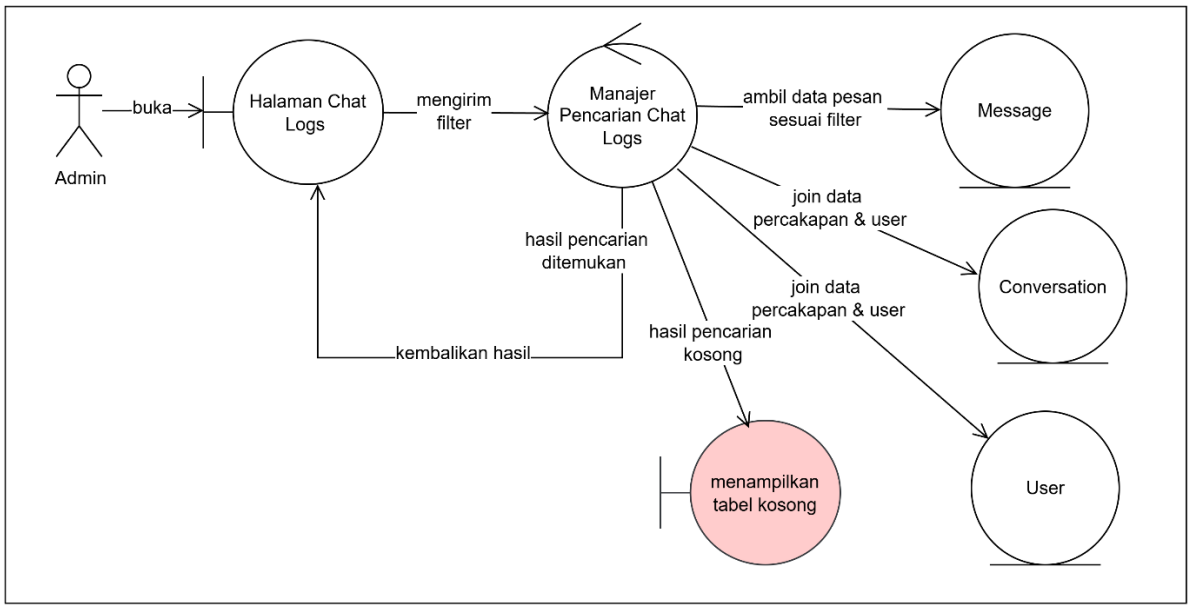


Gambar 4.16 *Robustness* Diagram Melihat *Dashboard* Analitik (RD-CB-04)

4.5.1.5 *Robustness Diagram* Melihat & Memfilter *Chat Logs* (RD-CB-05)

Robustness diagram melihat dan memfilter *chat logs* menggambarkan alur ketika admin membuka halaman log percakapan dan menerapkan filter pencarian. Setelah admin memasukkan kriteria seperti isi pesan, *role*, sumber jawaban, atau rentang tanggal, halaman *chat logs* sebagai *boundary* meneruskan permintaan tersebut kepada manajer pencarian *chat logs* sebagai *control*. *Control* kemudian mengambil data pesan yang sesuai filter melalui entitas *message*, serta melakukan *join* dengan entitas *Conversation* dan *user* untuk memperoleh informasi lengkap. Hasil pencarian dikembalikan ke halaman *chat logs* untuk ditampilkan dalam tabel.

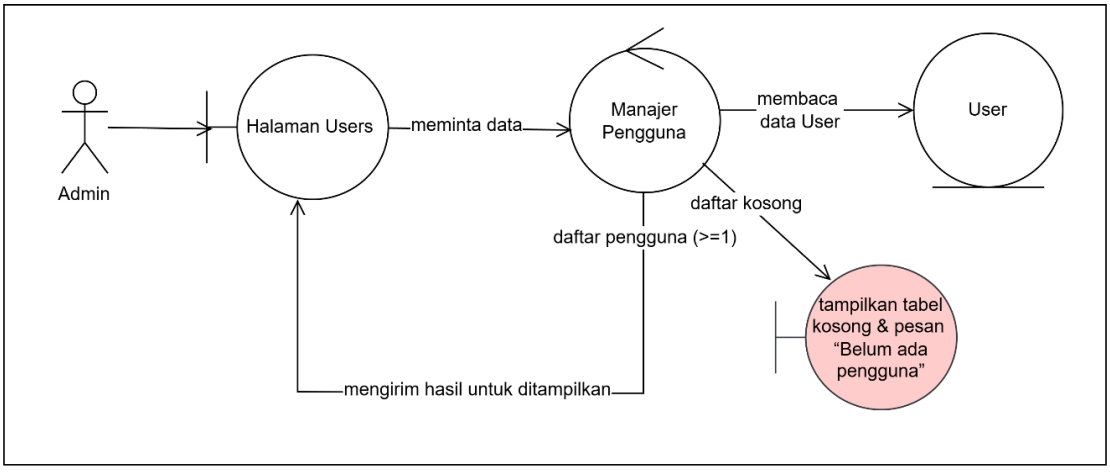
Pada skenario alternatif, apabila tidak ditemukan pesan yang sesuai, manajer pencarian *chat logs* mengembalikan daftar kosong, dan halaman *chat logs* menampilkan tabel kosong. Diagram ini mengikuti struktur *boundary–control–entity* sesuai prinsip ICONIX, sebagaimana ditunjukkan pada Gambar 4.17.



Gambar 4.17 *Robustness* Diagram Melihat & Memfilter *Chat Logs* (RD-CB-05)

4.5.1.6 *Robustness Diagram* Melihat Daftar Pengguna (RD-CB-06)

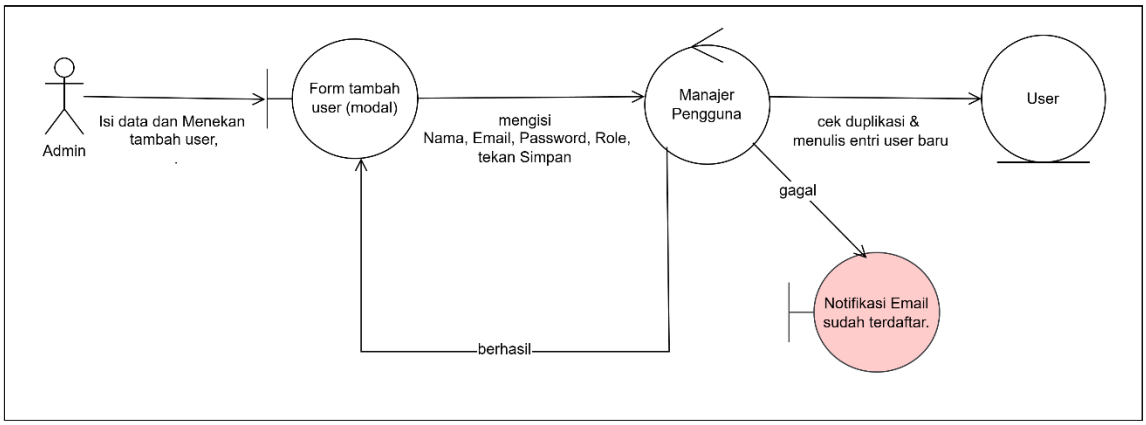
Robustness diagram ini menunjukkan halaman *user*s meminta daftar pengguna ke manajer pengguna, yang membaca data dari entitas *user* dan mengembalikan hasil berupa daftar pengguna atau tabel kosong jika tidak ada data, Diagram ini mengikuti struktur *boundary–control–entity* sesuai prinsip ICONIX, ditunjukkan pada Gambar 4.18.



Gambar 4.18 *Robustness* Diagram Melihat Daftar Pengguna (RD-CB-06)

4.5.1.7 *Robustness Diagram* Menambah Pengguna Baru (RD-CB-07)

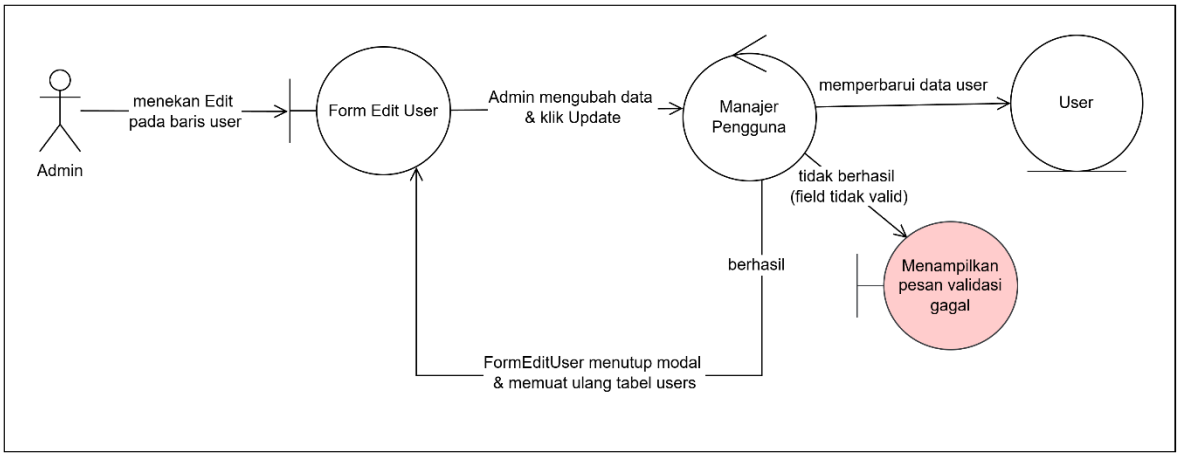
Robustness diagram ini menggambarkan alur ketika admin menambahkan pengguna baru melalui *form* tambah *user*. Setelah admin mengisi nama, email, *password*, dan *role* lalu menekan tombol simpan, *boundary* meneruskan data tersebut ke manajer pengguna sebagai kontrol utama. Manajer pengguna kemudian memeriksa entitas *User* untuk mendeteksi duplikasi email sekaligus membuat entri baru bila data valid. Jika proses berhasil, kontrol mengembalikan status sukses dan modal ditutup, sedangkan jika email sudah terdaftar, kontrol mengarahkan ke *boundary* notifikasi untuk menampilkan pesan "Email sudah terdaftar". Diagram ini mengikuti struktur *boundary–control–entity* sesuai prinsip ICONIX, ditunjukkan pada Gambar 4.19.



Gambar 4. 19 Robustness Diagram Menambah Pengguna Baru (RD-CB-07)

4.5.1.8 *Robustness Diagram* Memperbarui Pengguna (RD-CB-08)

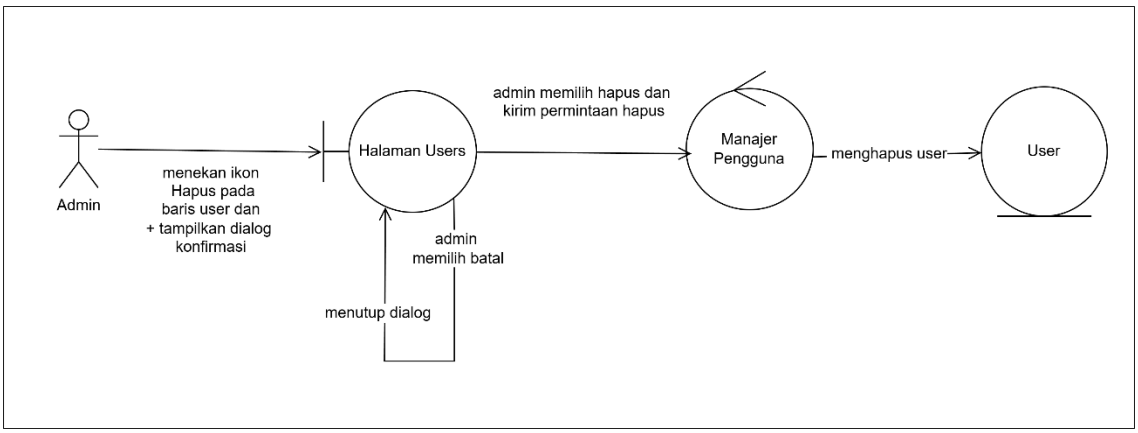
Robustness diagram memperbarui pengguna menggambarkan alur ketika admin menekan tombol *edit* pada baris pengguna sehingga sistem menampilkan *form* edit *user* dalam bentuk modal yang sudah terisi data awal. Setelah admin mengubah atribut seperti email atau *password* dan menekan tombol *update*, *Form edit user* meneruskan data tersebut ke manajer pengguna sebagai kontrol utama. Manajer pengguna memvalidasi *input* dan, apabila valid, memperbarui entitas *user* di *database* lalu mengembalikan status berhasil sehingga modal dapat ditutup dan tabel pengguna dimuat ulang. Pada skenario alternatif ketika ditemukan *field* tidak valid, manajer pengguna mengirim *respons* ke *boundary* untuk menampilkan pesan validasi gagal tanpa melakukan perubahan data. Diagram ini mengikuti struktur *boundary–control–entity* sesuai prinsip ICONIX, ditunjukkan pada Gambar Gambar 4.20.



Gambar 4.20 *Robustness* Diagram Memperbarui Pengguna (RD-CB-08)

4.5.1.9 *Robustness Diagram* Menghapus Pengguna (RD-CB-09)

Robustness diagram ini memperlihatkan alur ketika admin menghapus pengguna, dimulai dari halaman *users* yang menampilkan dialog konfirmasi dan meneruskan permintaan hapus ke manajer pengguna untuk menghapus entitas *user* di *database*. Jika proses berhasil, *boundary* memperbarui tampilan tabel, sedangkan pada alur alternatif ketika admin menekan batal, *boundary* hanya menutup dialog tanpa perubahan data. Diagram ini mengikuti struktur *boundary–control–entity* sesuai prinsip ICONIX dan divisualisasikan pada Gambar 4.21.



Gambar 4.21 *Robustness* Diagram Menghapus Pengguna (RD-CB-09)

4.5.2 *Updated Domain Model*

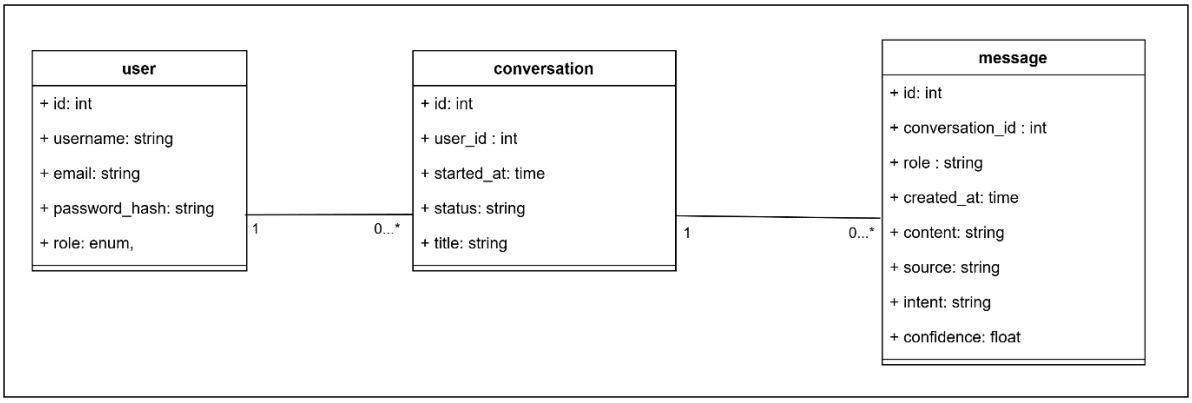
Updated domain model pada Gambar 4.22 merupakan hasil dari tahap *analysis / preliminary design* berdasarkan metode ICONIX *Process*. Seluruh robustness diagram yang

telah disusun pada tahap sebelumnya memetakan *boundary–control–entity* untuk setiap *use case*, dan temuan dari pemetaan tersebut menjadi dasar penyempurnaan struktur entitas dalam domain model.

Model yang telah diperbarui ini mencakup tiga entitas inti yang muncul secara konsisten dalam hasil *robustness analysis*, yaitu *User*, *Conversation* , dan *Message*. Entitas *User* direvisi untuk memastikan sistem dapat menangani autentikasi dan manajemen pengguna, dengan atribut seperti id, *username*, email, *password_hash*, dan *role* (enum). Berdasarkan analisis *use case*, setiap *user* dapat memiliki banyak percakapan sehingga terbentuk relasi *one-to-many* dengan entitas *Conversation* .

Entitas *Conversation* diperbarui untuk menampung atribut yang diperlukan dalam pengelolaan riwayat percakapan, seperti *user_id*, *started_at*, *status*, dan *title*. Selanjutnya, *robustness analysis* pada *use case* pengiriman pesan dan log percakapan menunjukkan bahwa setiap percakapan berisi banyak pesan, sehingga entitas *Message* tetap dipertahankan namun diperluas dengan atribut seperti *role*, *created_at*, *source*, *intent*, serta *confidence* (*float*) sebagai hasil analisis dari modul Rasa atau RAG.

Dengan pembaruan ini, domain model telah sepenuhnya mencerminkan objek dan relasi yang dibutuhkan untuk merealisasikan seluruh *use case* yang sebelumnya dianalisis. Model ini lebih lengkap dan bebas ambiguitas, sesuai dengan tujuan *tahapan preliminary design* dalam ICONIX *Process*, dan menjadi landasan utama untuk desain rinci pada tahap selanjutnya.



Gambar 4.22 *Updated Domain Model*

4.6 *Milestone* 2: *Preliminary Design Review*

Tahap *preliminary design review* (PDR), dilakukan setelah seluruh *robustness* diagram (RD-CB-01 s.d. RD-CB-09) dan *updated domain model* disusun. Sesuai penjelasan pada Bab 3, kegiatan utama pada PDR adalah *cross-artifact Checking*, yaitu memeriksa konsistensi dan keterlacakan antara artefak *requirements* dan artefak *analysis/preliminary design*. Pemeriksaan dilakukan dengan mencocokkan empat artefak utama, yaitu kebutuhan fungsional SRS (SRS-CB-F-01 s.d. SRS-CB-F-10), *use case* UC-CB-01 sampai UC-CB-09, *robustness* diagram RD-CB-01 sampai RD-CB-09, serta entitas dan relasi pada *updated domain model* (*User, Conversation , Message*). Hasil pemetaan tersebut dirangkum dalam Tabel 4.14 yang berfungsi sebagai matriks keterlacakan untuk membuktikan bahwa setiap kebutuhan fungsional telah memiliki representasi desain logis yang valid.

Secara spesifik, proses pemeriksaan ini memvalidasi tiga kelompok fungsionalitas utama dalam sistem. Pertama, pada aspek autentikasi dan keamanan sistem yang mencakup SRS-CB-F-01 hingga SRS-CB-F-03, pemeriksaan menunjukkan bahwa logika verifikasi kredensial dan kontrol akses (RBAC) telah terpetakan dengan tepat pada *Use case* Autentikasi (UC-CB-01) dan *Logout* (UC-CB-11). Validasi desain pada *Robustness* Diagram RD-CB-01 mempertegas bahwa interaksi antara *boundary* halaman *login* dan *control* autentikasi selalu bermuara pada pengecekan entitas *User*, sehingga memastikan pemisahan hak akses antara mahasiswa dan admin dapat berjalan sesuai desain. Hal ini menjamin bahwa atribut *role* pada *updated* domain model benar-benar dimanfaatkan dalam alur logika sistem

Kedua, pemeriksaan dilakukan pada fungsionalitas inti *chat*bot yang meliputi pengiriman pesan, mekanisme *fallback* dari Rasa ke RAG, serta pengelolaan riwayat percakapan (SRS-CB-F-04 sampai SRS-CB-F-07). Berdasarkan Tabel 4.14, kebutuhan ini direpresentasikan melalui *Use Case* Mengirim Pesan *Chatbot* dan Menerima Jawaban (UC-CB-02) serta *Use Case* Melihat Riwayat Pesan (UC-CB-03). *Robustness* Diagram RD-CB-02 dan RD-CB-03 menunjukkan bahwa entitas *Conversation* dan *Message* telah dirancang untuk mendukung penyimpanan *metadata* pesan, relasi *one-to-many* antar entitas, serta penampilan riwayat pesan per sesi percakapan, sebagaimana dituntut oleh kebutuhan fungsional sistem.

Ketiga, validasi dilakukan pada fitur administratif yang mencakup dashboard analitik dan pengelolaan data pengguna (SRS-CB-F-08 sampai SRS-CB-F-10). Fungsionalitas ini dipetakan pada *use case* UC-CB-04 hingga UC-CB-09 dan divisualisasikan melalui *Robustness* Diagram RD-CB-04 sampai RD-CB-09. Pada fitur dashboard analitik, desain menunjukkan bagaimana *control* pengolah analitik mengagregasi data dari entitas *Conversation* dan *Message* untuk menghasilkan statistik percakapan. Sementara itu, pada fitur pengelolaan pengguna, robustness diagram memastikan bahwa setiap operasi *create*, *read, update*, dan *delete* berinteraksi secara langsung dan aman dengan entitas *User*.

Berdasarkan hasil *cross-artifact checking* tersebut, dapat disimpulkan bahwa tidak terdapat kebutuhan yang terlewat (*no missing requirements*), serta seluruh artefak analisis dan desain awal telah saling terhubung secara konsisten. Dengan demikian, tahap *Preliminary Design Review* ini menyatakan bahwa desain sistem telah matang dan siap dilanjutkan ke tahap *Detailed Design* untuk penyusunan *Sequence Diagram* dan *Class Diagram*.

Tabel 4.14 *Cross-Artifact Checking* Tahap PDR

No	SRS	*Use Case*	*Robustness* Diagram	Domain Model	Desain Antarmuka
1	SRS-CB-F-01 Autentikasi Pengguna	UC-CB-01 Autentikasi Pengguna	RD-CB-01 Autentikasi Pengguna	*User*	DA-CB-01 Halaman *Login*
2	SRS-CB-F-02 Pengambilan *Role* Pengguna	UC-CB-01 Autentikasi Pengguna	RD-CB-01 Autentikasi Pengguna	*User*	DA-CB-01 Halaman *Login*
3	SRS-CB-F-03 RBAC	UC-CB-01 Autentikasi Pengguna	RD-CB-01 Autentikasi Pengguna	*User*	DA-CB-02 Halaman *Chat* & DA-CB-03 *Dashboard* Admin

No	SRS	*Use Case*	*Robustness* Diagram	Domain Model	Desain Antarmuka
4	SRS-CB-F-04 Pengiriman Pesan (Rasa & RAG)	UC-CB-02 Mengirim Pesan *Chatbot*	RD-CB-02 Mengirim Pesan *Chatbot*	*User, Conversation, Message*	DA-CB-02 Halaman *Chat*
5	SRS-CB-F-05 Penyimpanan Pesan & *Metadata*	UC-CB-02 Mengirim Pesan Chatbot	RD-CB-02 Mengirim Pesan Chatbot	*Conversation, Message*	DA-CB-02 Halaman *Chat*
6	SRS-CB-F-06 Daftar Percakapan Pengguna	UC-CB-03 Melihat Riwayat Pesan	RD-CB-03 Melihat Riwayat Pesan	*User, Conversation*	DA-CB-02 *Sidebar* Percakapan
7	SRS-CB-F-07 Riwayat Pesan Percakapan	UC-CB-03 Melihat Riwayat Pesan	RD-CB-03 Melihat Riwayat Pesan	*Conversation, Message*	DA-CB-02 Area Percakapan
8	SRS-CB-F-08 *Dashboard* Analitik	UC-CB-04 Melihat Dashboard Analitik	RD-CB-04 Melihat Dashboard Analitik	*Conversation, Message*	DA-CB-03 *Dashboard* Admin
9	SRS-CB-F-09 *Chat Logs* & Filter	UC-CB-05 Melihat & Memfilter *Chat Logs*	RD-CB-05 *Chat Logs*	*User, Conversation, Message*	DA-CB-04 Halaman *Chat Logs*
10	SRS-CB-F-10 Pengelolaan *User* (CRUD)	UC-CB-06 s.d. UC-CB-09	RD-CB-06 s.d. RD-CB-09	*User*	DA-CB-05 Halaman Pengelolaan *User*

4.7 Tahap *Detailed Design*

Tahap *Detailed Design* merupakan fase lanjutan dalam metodologi ICONIX *Process* yang dilakukan setelah proses *Preliminary Design Review*. Pada tahap ini desain sistem disempurnakan. Dapat dilihat pada sistem *Chatbot* FSM, *detailed design* diaplikasikan melalui beberapa langkah utama. Pertama, dibuat *Sequence Diagram* untuk setiap *use case*. *Sequence Diagram* ini menggambarkan alur interaksi antara *boundary*, *Controller*, dan *entity* class secara lebih rinci, termasuk urutan pemanggilan metode, pemrosesan data, dan alur logika untuk kedua sumber balasan *Chatbot*, yaitu Rasa dan RAG. *Sequensce diagram* ini sangat penting untuk memvisualisasikan proses teknis sistem, seperti pengambilan percakapan, pemrosesan pesan pengguna, *fallback* mekanisme, hingga penyimpanan pesan dalam *database*.

Melalui *detailed design* ini, seluruh model desain telah mencapai tahap yang konsisten, komprehensif, dan langsung dapat diterjemahkan menjadi kode program. Setiap keputusan desain yang diambil telah divalidasi melalui keterkaitan yang kuat dengan *use case* dan *robustness* diagram, sehingga meminimalkan risiko perubahan besar pada saat implementasi. Tahap ini juga menjadi jembatan yang memastikan bahwa pengembangan sistem berjalan terstruktur sesuai kebutuhan fungsional yang telah ditetapkan.

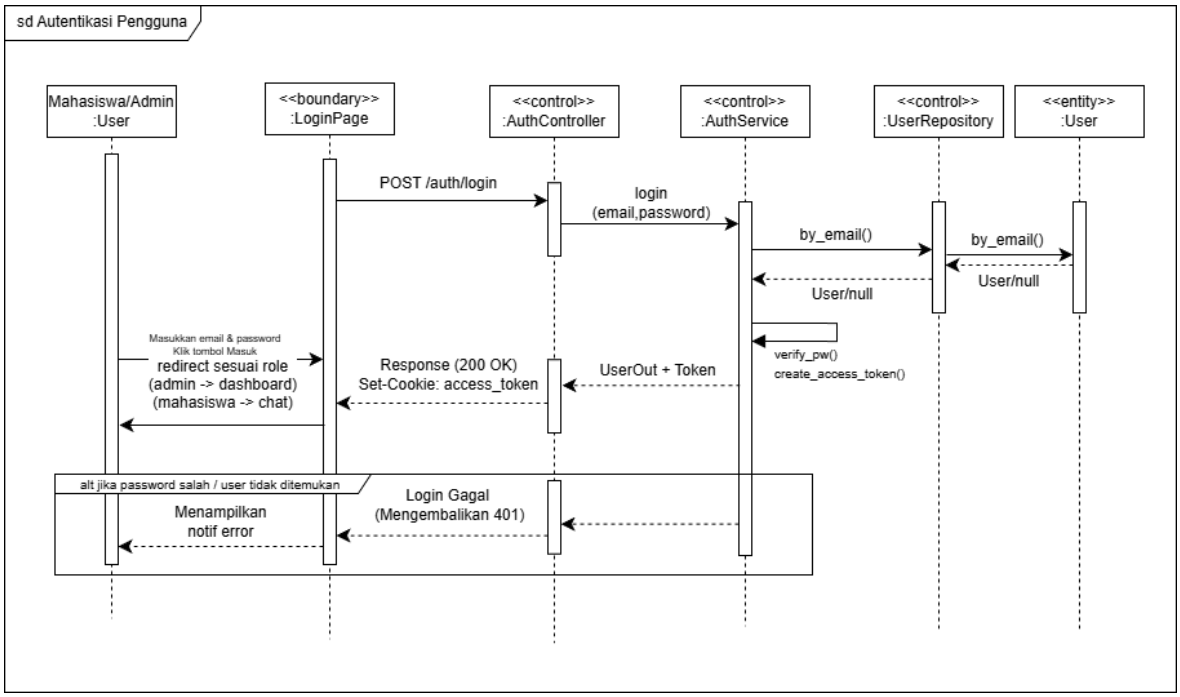
4.7.1 *Sequence Diagram*

Sequence Diagram menggambarkan interaksi dinamis antarobjek dalam sistem berdasarkan urutan pesan yang terjadi dari awal hingga akhir suatu skenario. Pada tahap *detailed design*, *Sequence Diagram* dibuat untuk seluruh *use case* utama yang telah melewati proses *robustness analysis*, sehingga menghasilkan representasi perilaku sistem yang lebih terperinci dan siap untuk diimplementasikan. Setiap diagram memperlihatkan keterlibatan *boundary object*, *Controller*, dan *entity* sesuai prinsip ICONIX *Process*.

4.7.1.1 *zSequence Diagram* Autentikasi Pengguna (SD-CB-01)

Sequence Diagram pada Gambar 4.23 ini menunjukkan proses autentikasi ketika mahasiswa atau admin melakukan *login* ke sistem. Pengguna memasukkan email dan *password* melalui antarmuka *login*, kemudian data tersebut dikirim ke *Auth Controller*. *Controller* meneruskan permintaan ke *Auth Service* untuk melakukan validasi kredensial

terhadap data pada entitas *User*. Jika valid, sistem menghasilkan token autentikasi dan mengambil data pengguna. Berdasarkan *role*, sistem mengarahkan pengguna ke halaman *chat* (mahasiswa) atau *dashboard* (admin). Jika invalid, sistem mengembalikan *respons unauthorized*.



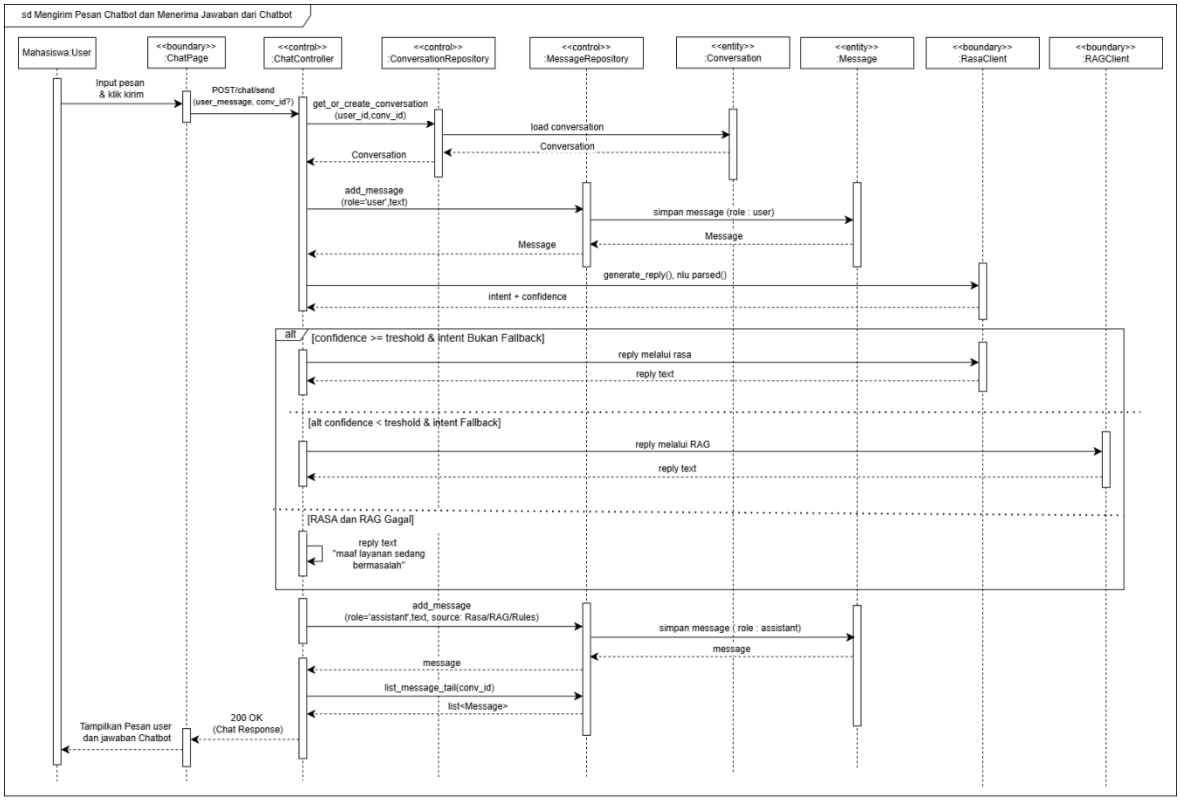
Gambar 4. 23 Sequence Diagram Autentikasi Pengguna (SD-CB-01)

4.7.1.2 *Sequence Diagram* Mengirim Pesan *Chatbot* dan Menerima Jawaban *Chatbot* (SD-CB-02)

Pada Gambar 4.24 disajikan *Sequence Diagram* untuk *Use case* Mengirim Pesan *Chatbot* dan Menerima Jawaban (SD-CB-02) yang menggambarkan interaksi dinamis antara mahasiswa dan sistem. Alur dimulai ketika aktor Mahasiswa memasukkan pertanyaan pada antarmuka :*ChatPage* dan menekan tombol kirim, yang memicu pengiriman permintaan (*request*) ke :*ChatController*. Sebagai langkah awal validasi, :*ChatController* mengakses :*Conversation Repository* untuk memuat data percakapan yang sedang aktif atau membuat sesi baru jika belum tersedia. Setelah konteks percakapan terbentuk, pesan dari pengguna disimpan ke dalam basis data melalui pesan add_message() ke :*MessageRepository*.

Proses inti penentuan jawaban dikendalikan sepenuhnya oleh :*ChatController* yang digambarkan dalam blok *combined fragment* tipe *alternative*. Pada skenario utama, *Controller* mengirimkan teks ke :*RasaClient* untuk analisis *intent*. Apabila skor *confidence*

di atas ambang batas, jawaban dari Rasa digunakan. Sebaliknya, jika *intent* tidak dikenali atau skor rendah, *Controller* mengalihkan logika ke skenario *fallback* dengan memanggil :RAGClient untuk pencarian jawaban berbasis dokumen. Jika kedua layanan eksternal tersebut gagal, sistem menetapkan pesan standar mengenai gangguan layanan. Setelah jawaban final didapatkan, :*ChatController* menyimpannya kembali melalui :*MessageRepository*, mengambil daftar riwayat pesan terbaru, dan mengembalikan objek respon ke :*ChatPage* untuk ditampilkan kepada Mahasiswa.



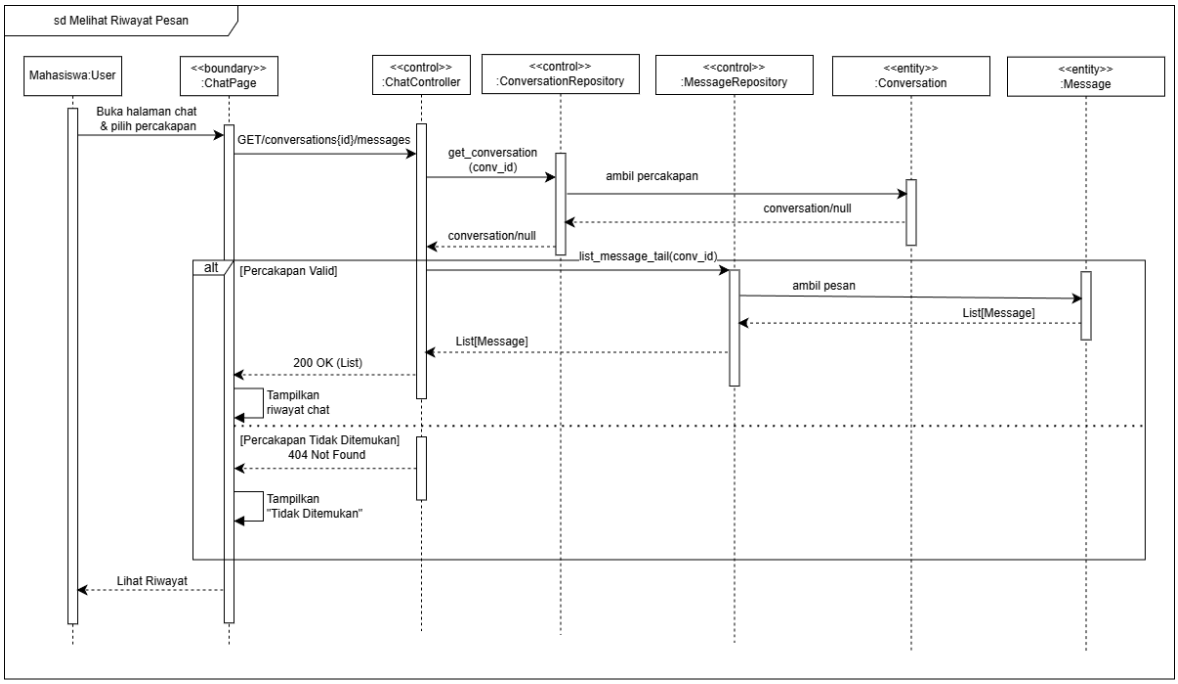
Gambar 4. 24 Sequence Diagram Mengirim Pesan Chatbot dan Menerima Jawaban dari Chatbot (SD-CB-02)

4.7.1.3 *Sequence Diagram* Melihat Riwayat Pesan (SD-CB-03)

Sequence Diagram pada Gambar 4.25 Melihat Riwayat Pesan (SD-CB-03) menggambarkan alur interaksi ketika mahasiswa membuka halaman *chat* untuk melihat riwayat percakapan yang pernah dilakukan. Proses diawali dengan permintaan sistem untuk mengambil daftar percakapan milik pengguna yang sedang *login*. Sistem memanggil *ConversationRepository* untuk memperoleh daftar percakapan dan mengembalikannya ke halaman *chat* untuk ditampilkan pada sidebar. Selanjutnya, ketika mahasiswa memilih salah satu percakapan pada sidebar, sistem mengirimkan permintaan untuk mengambil riwayat pesan berdasarkan *conversation_id*. *ChatController* memverifikasi keberadaan percakapan

melalui *ConversationRepository*, kemudian mengambil seluruh pesan terkait melalui *MessageRepository*. Daftar pesan yang diper*oleh* dikembalikan ke halaman *chat* dan ditampilkan pada area percakapan.

Pada skenario alternatif, apabila daftar percakapan kosong atau percakapan yang dipilih tidak ditemukan, sistem mengembalikan *respons* yang sesuai dan halaman *chat* menampilkan pesan informasi kepada pengguna. *Sequence* diagram ini memastikan bahwa seluruh alur dalam *use case Melihat Riwayat Pesan* telah dimodelkan secara konsisten sesuai prinsip ICONIX.

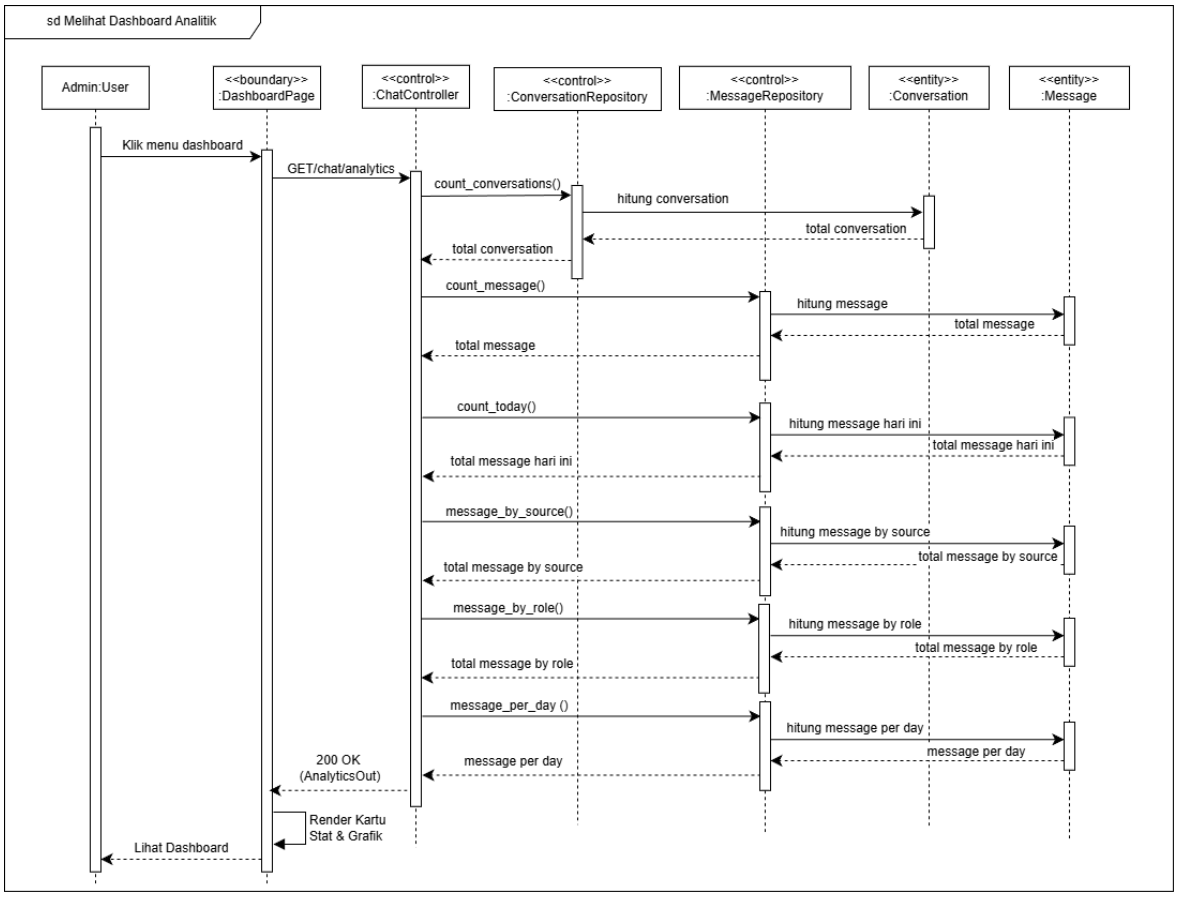


Gambar 4. 25 Sequence Diagram Melihat Riwayat Pesan (SD-CB-03)

4.7.1.4 *Sequence Diagram* Melihat *Dashboard* Analitik (SD-CB-04)

Sequence Diagram pada Gambar 4.26 Melihat *Dashboard* Analitik (SD-CB-04) memvisualisasikan alur sistem dalam menyajikan data statistik kepada admin. Proses bermula ketika aktor Admin mengakses menu *dashboard* pada :*DashboardPage* , yang mengirimkan permintaan GET /*chat*/*analytics* ke :*ChatController*. *Controller* kemudian melakukan serangkaian pengambilan data secara berurutan untuk menyusun laporan analitik lengkap.

Pertama, *Controller* memanggil *count_Conversations*() pada :*Conversation Repository* yang mengakses entitas :*Conversation* untuk mendapatkan total percakapan. Selanjutnya, *Controller* beralih ke :*MessageRepository* untuk menjalankan berbagai fungsi agregasi terhadap entitas *:Message*, meliputi perhitungan total pesan (*count_message*), aktivitas hari ini (*count_today*), serta distribusi pesan berdasarkan sumber, peran, dan tren harian. Setelah seluruh data terkumpul, :*ChatController* mengemasnya menjadi objek JSON dan mengembalikan respon 200 OK ke antarmuka untuk dirender menjadi grafik visual.

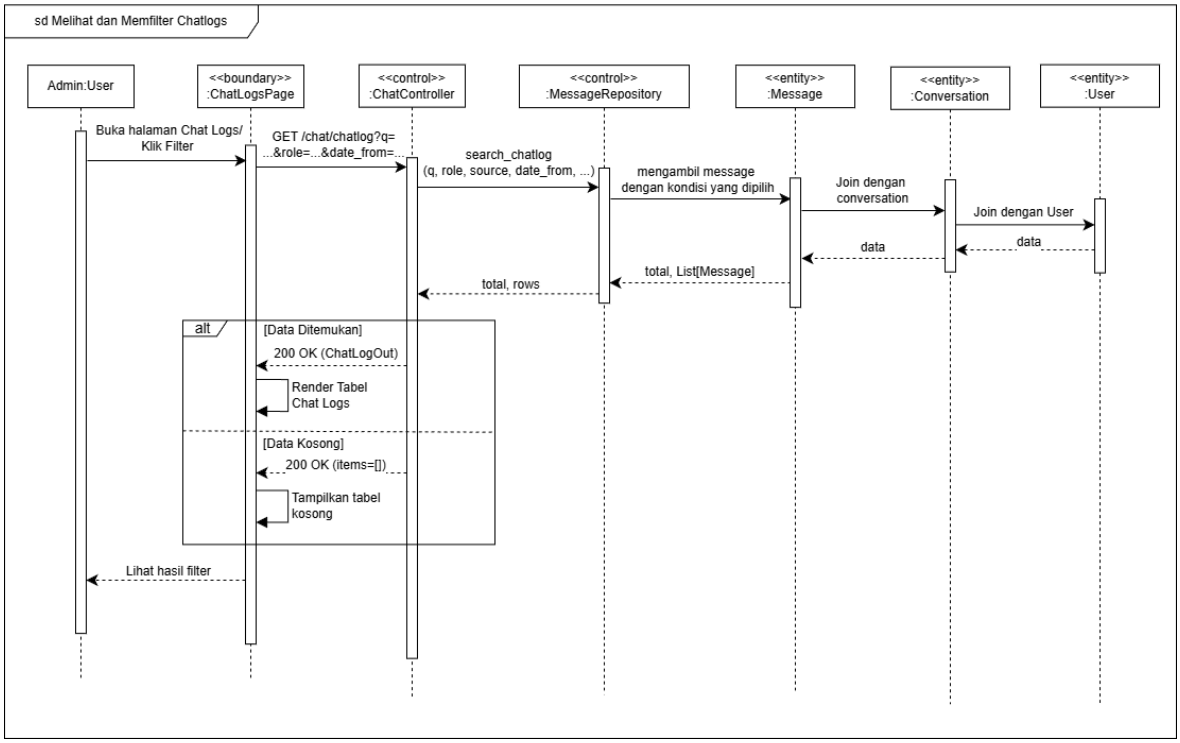


Gambar 4. 26 Sequence Diagram Melihat Dashboard Analitik (SD-CB-04)

4.7.1.5 *Sequence Diagram* Melihat & Memfilter *Chat Logs* (SD-CB-05)

Sequence Diagram pada Gambar 4.27 Melihat & Memfilter *Chat Logs* (SD-CB-05) memvisualisasikan interaksi sistem saat admin melakukan penelusuran data riwayat pesan menggunakan kriteria tertentu. Alur dimulai ketika Admin mengakses halaman atau menerapkan filter pada :*Chat*Logs*Page*, yang mengirimkan permintaan HTTP GET ke :*ChatController* beserta parameter penyaring seperti *query*, *role*, atau rentang tanggal. *Controller* kemudian meneruskan parameter tersebut ke :*MessageRepository* melalui fungsi search_*chatlog*(). Repository melakukan pengambilan data (query) utama terhadap entitas

:Message, yang digambarkan secara relasional terhubung (join) dengan entitas :*Conversation* dan :*User* guna melengkapi informasi identitas pengguna. Berdasarkan hasil kueri, alur terbagi dalam blok alternative; jika data ditemukan, *Controller* mengembalikan objek JSON berisi daftar log (200 OK) untuk dirender ke dalam tabel, sedangkan jika hasil pencarian nihil, sistem mengembalikan daftar kosong yang ditampilkan antarmuka sebagai tabel tanpa data.

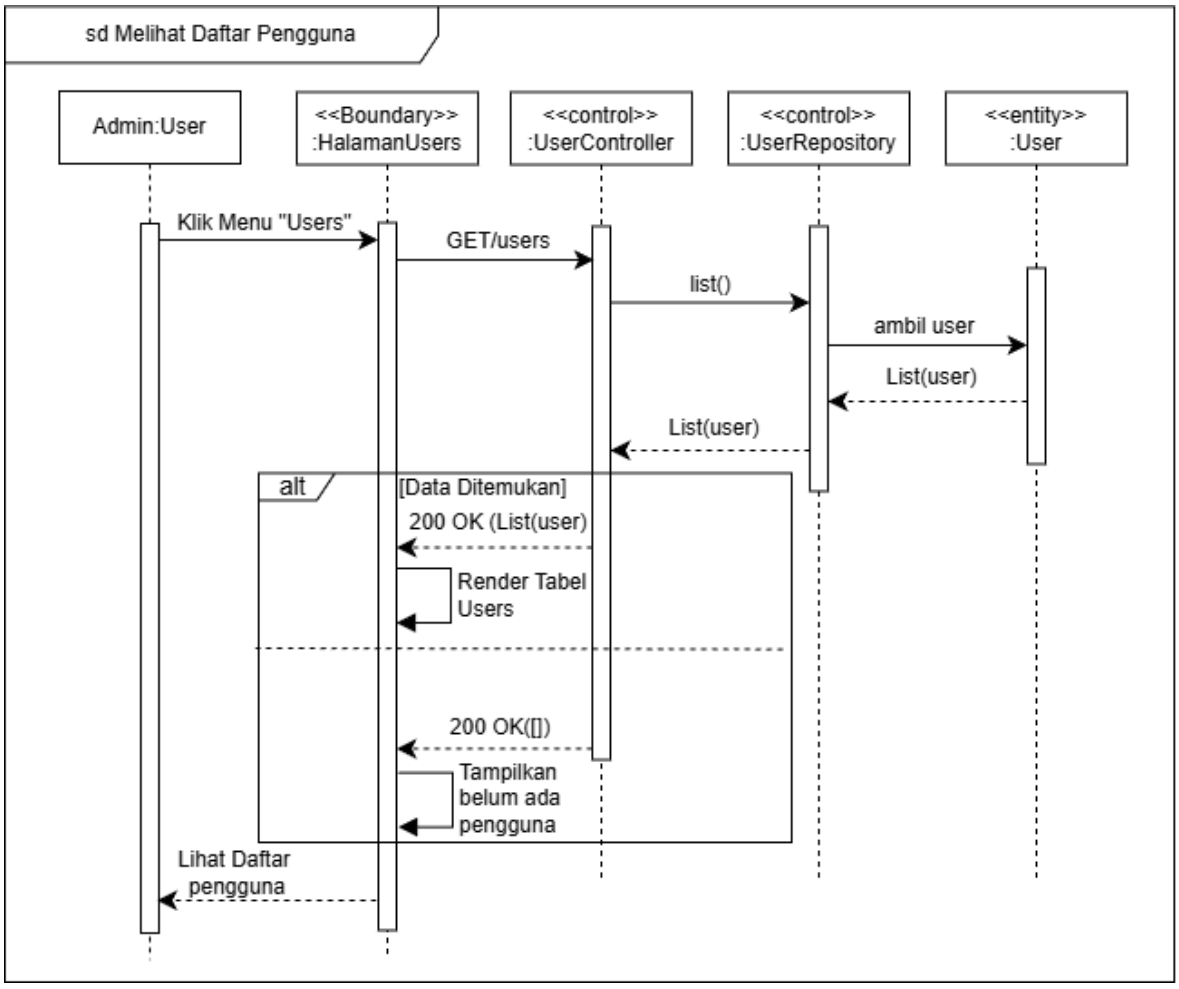


Gambar 4. 27 Sequence Diagram Melihat & Memfilter Chat Logs (SD-CB-05)

4.7.1.6 *Sequence Diagram* Melihat Daftar Pengguna (SD-CB-06)

Sequence Diagram pada Gambar 4.28 Melihat Daftar Pengguna (SD-CB-06) menggambarkan secara rinci bagaimana sistem menangani permintaan administrator untuk mengakses dan menampilkan seluruh data akun pengguna yang tersimpan di dalam basis data. Alur interaksi ini diawali dengan aksi aktor Admin yang menekan menu "*User*s" pada antarmuka :Halaman*User*s, yang secara otomatis mengirimkan permintaan HTTP bertipe GET /*user*s menuju :*UserController* sebagai pintu gerbang logika utama. Selanjutnya, *Controller* meneruskan tanggung jawab pengambilan data dengan memanggil fungsi *list*() pada :*UserRepository*, yang bertugas mengeksekusi perintah *query* langsung ke entitas :*User* guna mengambil semua *record* pengguna yang tersedia. Berdasarkan hasil pengembalian data dari *repository*, :*UserController* akan menjalankan logika percabangan untuk menentukan *respon* yang tepat; apabila data pengguna berhasil ditemukan, *Controller*

mengirimkan *respon* sukses 200 OK berisi daftar objek pengguna (*List*[*User*]) yang kemudian dirender oleh antarmuka menjadi tabel interaktif, namun jika basis data kosong, sistem akan mengembalikan daftar kosong yang direspon oleh antarmuka dengan menampilkan notifikasi informatif "Belum ada pengguna" kepada admin.

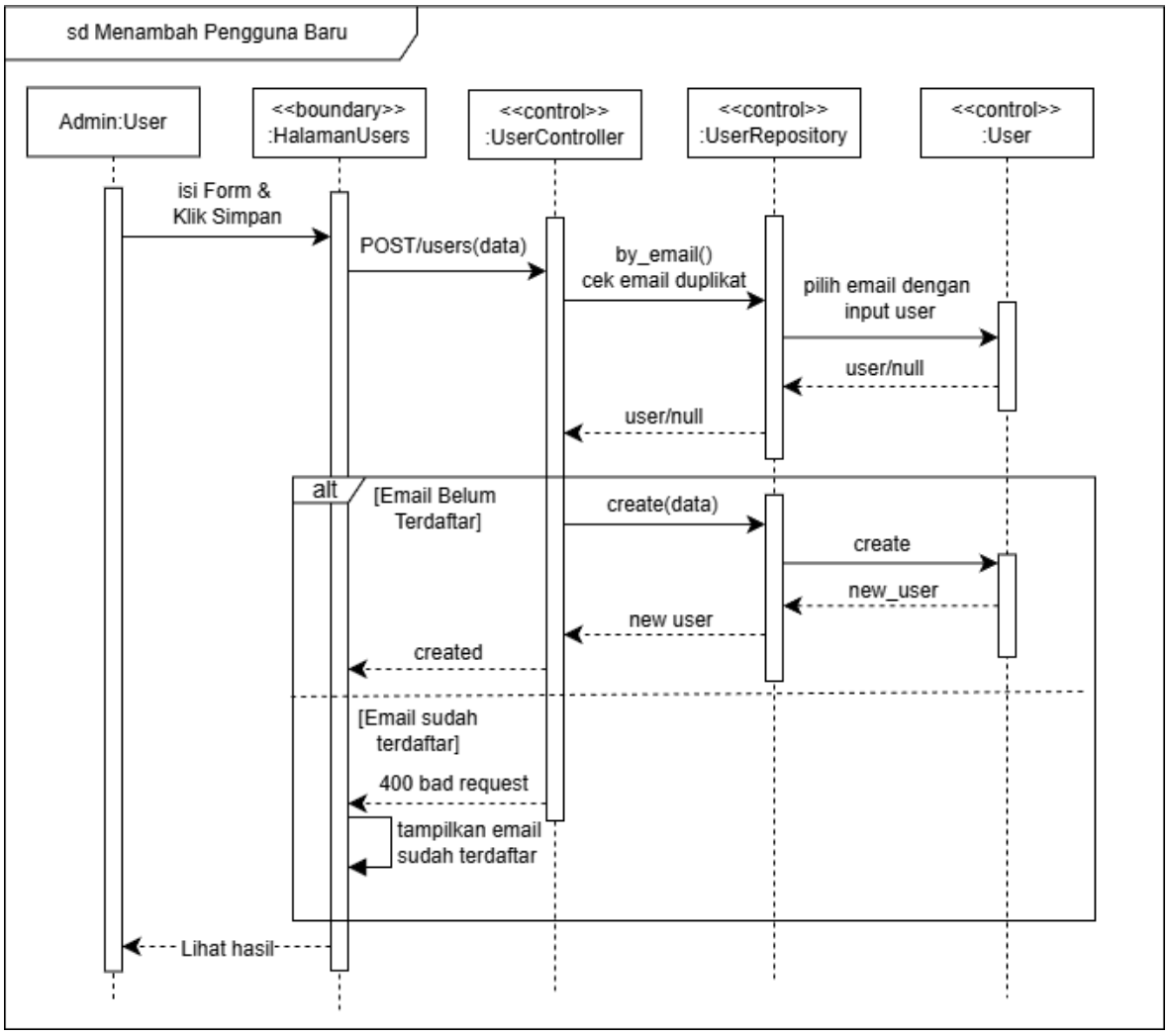


Gambar 4. 28 Sequence Diagram Melihat Daftar Pengguna (SD-CB-06)

4.7.1.7 *Sequence Diagram* Menambah Pengguna Baru (SD-CB-07)

Sequence Diagram pada Gambar 4.29 Menambah Pengguna Baru (SD-CB-07) memvisualisasikan alur interaksi sistem saat administrator mendaftarkan akun pengguna baru. Proses ini diinisiasi oleh aksi Admin yang mengisi formulir dan menekan tombol simpan pada :Halaman*User*s, yang kemudian mengirimkan permintaan HTTP POST /*user*s berisi data akun ke :*UserController*. Sebagai langkah validasi awal, *Controller* memanggil metode by_email() pada :*UserRepository* untuk memeriksa ketersediaan email dalam entitas :*User*. Berdasarkan hasil pemeriksaan tersebut, alur sistem terbagi dalam blok *alternative*; jika email belum terdaftar (hasil null), *Controller* melanjutkan proses dengan

memerintahkan *repository* untuk menyimpan data pengguna baru melalui metode *create*() dan mengembalikan *respon* sukses 201 *Create*d. Sebaliknya, jika email sudah digunakan, :*UserController* menghentikan proses dan mengembalikan status 400 *Bad Request* yang direspon antarmuka dengan menampilkan pesan kesalahan "Email sudah terdaftar".

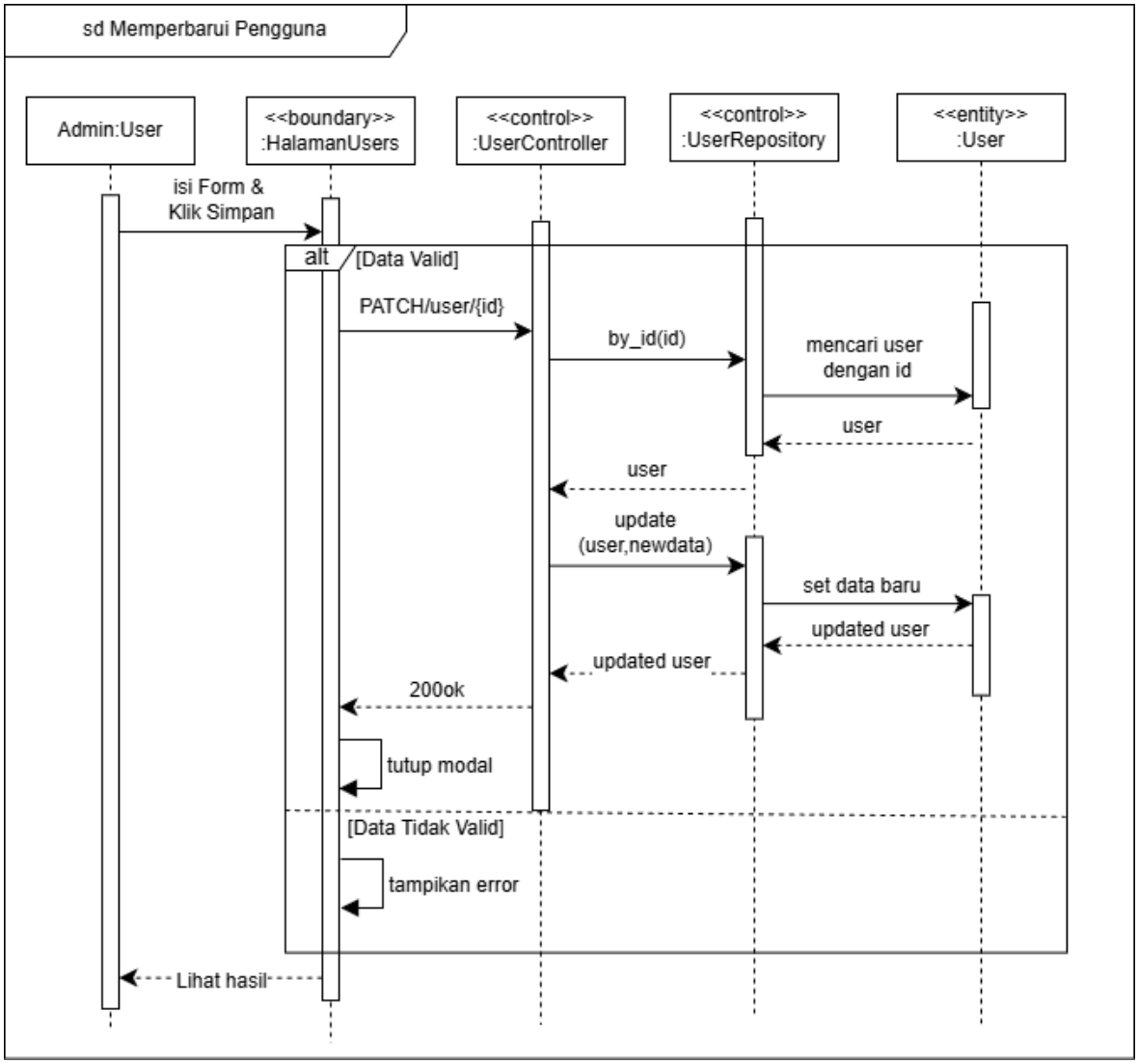


Gambar 4. 29 Sequence Diagram Menambah Pengguna Baru (SD-CB-07)

4.7.1.8 *Sequence Diagram* Memperbarui Pengguna (SD-CB-08)

Sequence Diagram pada Gambar 4.30 Memperbarui Pengguna (SD-CB-08) memvisualisasikan mekanisme pemutakhiran data akun pengguna yang dilakukan oleh administrator. Alur dimulai ketika Admin mengubah informasi pada formulir edit dan menekan tombol simpan di :Halaman*User*s. Jika *input* data divalidasi benar, antarmuka mengirimkan permintaan HTTP PATCH /*user*/{id} ke :*UserController*. *Controller* memproses permintaan ini dalam dua tahap; pertama, ia memanggil metode by_id() pada :*UserRepository* untuk mengambil objek data pengguna lama dari entitas :*User*. Setelah

objek ditemukan, *Controller* memanggil fungsi *update*() pada *repository* yang bertugas menerapkan perubahan data baru ke dalam entitas :*User* dan menyimpannya ke basis data. Setelah proses pembaruan berhasil, sistem mengembalikan status 200 OK yang direspon oleh antarmuka dengan menutup modal edit, sedangkan jika data tidak valid, sistem menampilkan pesan kesalahan tanpa melanjutkan proses ke server.

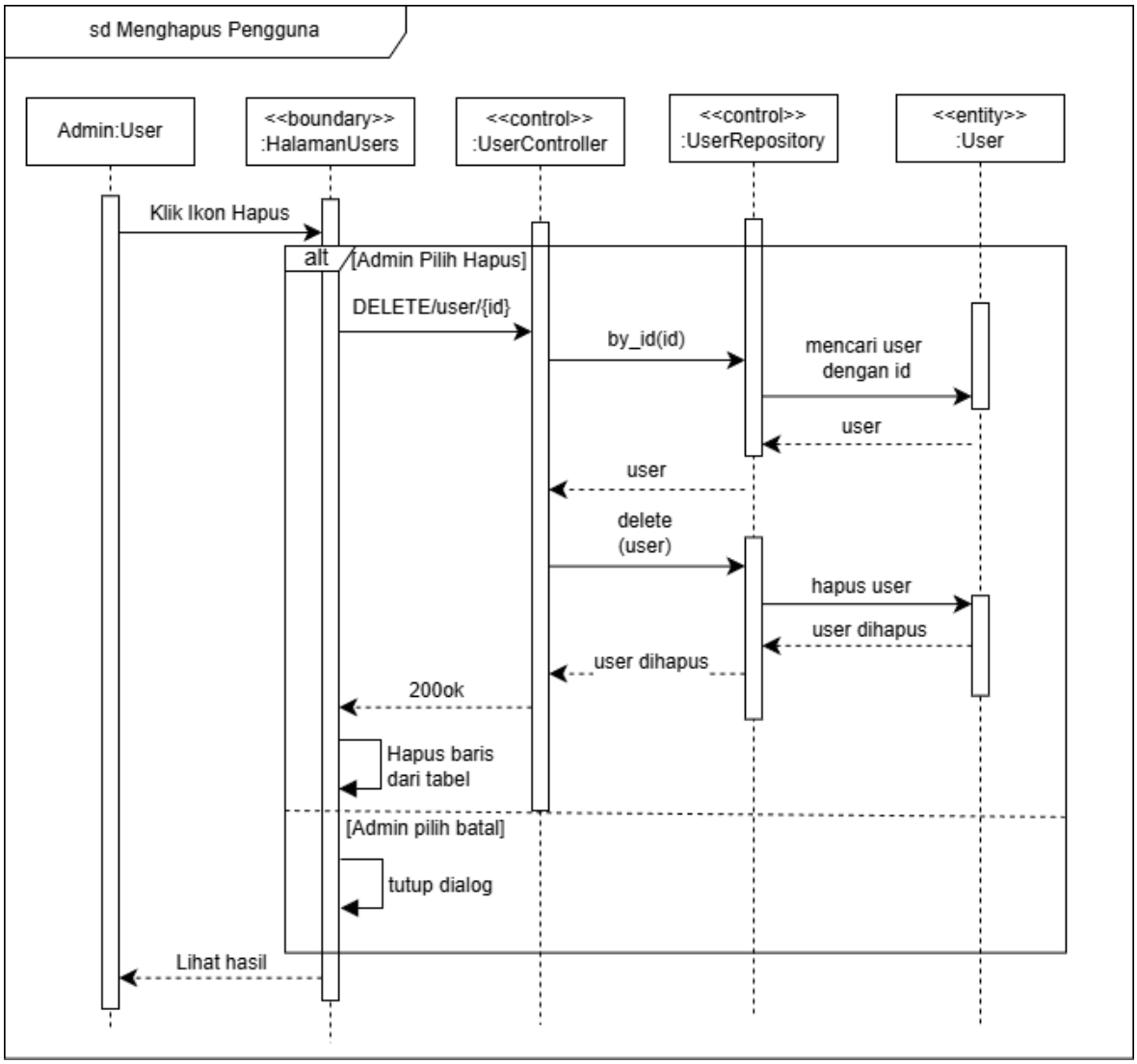


Gambar 4. 30 Sequence Diagram Memperbarui Pengguna (SD-CB-08)

4.7.1.9 *Sequence Diagram* Menghapus Pengguna (SD-CB-09)

Sequence Diagram pada Gambar 4.31 Menghapus Pengguna (SD-CB-09) memvisualisasikan mekanisme penghapusan akun pengguna dari basis data sistem. Alur dimulai ketika Admin menekan ikon hapus pada baris tabel di :Halaman*User*s, yang memicu tampilan dialog konfirmasi. Logika interaksi terbagi dalam blok *alternative*; jika admin mengonfirmasi penghapusan, antarmuka mengirimkan permintaan HTTP *DELETE*

/*user*/{id} ke :*UserController*. *Controller* terlebih dahulu memvalidasi target dengan memanggil metode by_id() pada :*UserRepository* untuk mengambil objek pengguna dari entitas :*User*. Setelah data dipastikan ada, *Controller* memanggil fungsi *delete*() pada *repository* untuk menghapus data tersebut secara permanen. Proses ini diakhiri dengan pengembalian status 200 OK yang direspon antarmuka dengan menghapus baris pengguna dari tabel, sedangkan jika admin memilih batal pada dialog konfirmasi, sistem hanya menutup dialog tanpa melakukan perubahan apa pun pada server.



Gambar 4.31 *Sequence Diagram* Menghapus Pengguna (SD-CB-09)

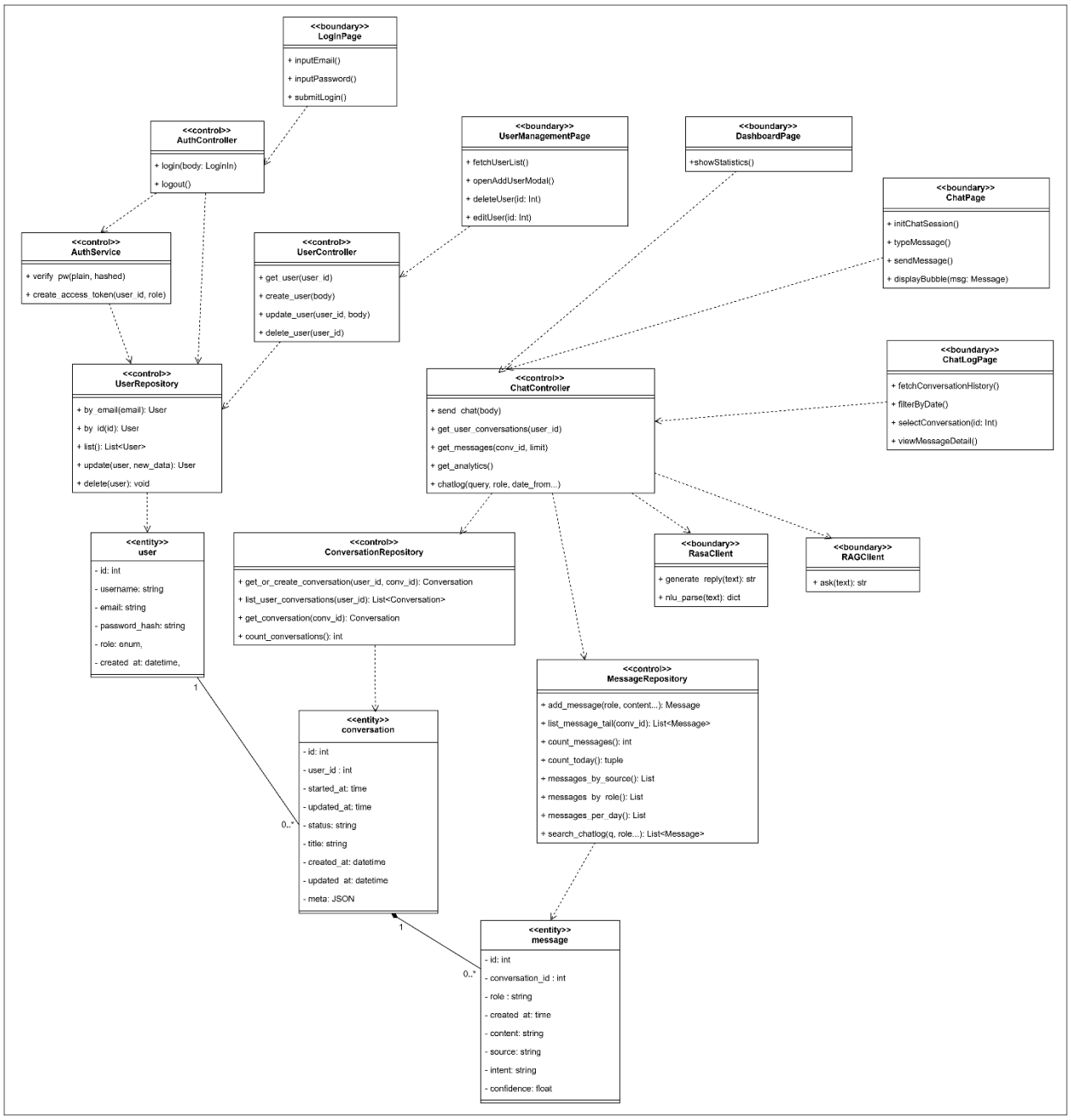
4.7.2 *Class Diagram*

Tahap *detailed design* merupakan fase krusial di mana model analisis diterjemahkan menjadi spesifikasi teknis yang siap diimplementasikan. Mengacu pada metodologi yang digunakan pada penelitian ini perancangan *Class Diagram* ini disusun sebagai hasil elaborasi

dari *Robustness Diagram* dan *Sequence Diagram* yang telah dirancang sebelumnya. *Class Diagram* pada tahap ini berfungsi sebagai peta struktural sistem yang paling vital karena merepresentasikan *blueprint* objek yang digunakan sebagai acuan utama dalam proses implementasi kode program.

Berdasarkan Gambar 4.32, *Class Diagram* sistem menggambarkan struktur statis yang mendefinisikan tipe-tipe objek, atribut penyimpanan data, serta metode layanan dalam sistem. Arsitektur diagram terbagi menjadi empat lapisan utama, yaitu *Controller* (seperti *ChatController*), *Service* untuk logika bisnis khusus, *Repository* sebagai akses data, dan *Entity* yang merepresentasikan tabel basis data. Hubungan antar kelas digambarkan menggunakan simbol notasi standar sesuai Tabel 2.4. Hubungan kebergantungan (*dependency*) dengan garis putus-putus digunakan untuk menghubungkan kelas pada lapisan *Controller* dengan *Repository* atau *Service*, yang menandakan bahwa *Controller* hanya memanggil metode kelas lain secara sementara tanpa menyimpannya sebagai atribut permanen. Kemudian ada *ChatPage, LoginPage, DashboardPage, ChatlogPage, dan ManagementUserPage* mewakili objek boundary dan untuk implementasi nanti akan diimplementasikan sebagai *interface*.

Selanjutnya, hubungan struktural antar entitas didefinisikan melalui asosiasi dan komposisi. Hubungan asosiasi (*association*) diterapkan antara entitas *User* dan *Conversation* , yang menunjukkan bahwa satu objek pengguna dapat memiliki hubungan kepemilikan terhadap banyak percakapan (*one-to-many*). Sementara itu, hubungan komposisi (*composition*) dengan simbol wajik hitam diterapkan antara entitas *Conversation* dan *Message*. Sesuai dengan teori Booch dkk. (2005), ini merepresentasikan hubungan *whole-part* yang kuat, di mana keberadaan *Message* sepenuhnya bergantung pada *Conversation* . Secara teknis, hal ini diimplementasikan dengan mekanisme *cascade delete*, yang berarti jika sebuah data percakapan dihapus dari sistem, maka seluruh pesan yang ada di dalamnya juga akan terhapus secara otomatis demi menjaga integritas data.



Gambar 4. 32 Class Diagram

4.8 *Milestone* 3: *Critical Design Review*

Tahap *Critical Design Review* (CDR) merupakan langkah evaluasi akhir dalam metodologi ICONIX *Process* sebelum memasuki tahap implementasi kode program. Sesuai dengan pedoman Rosenberg dkk. (2007), tujuan utama dari CDR adalah memastikan validitas desain rinci (*detailed design*) dengan memverifikasi konsistensi antara perilaku

sistem yang digambarkan pada desain dinamis (*Sequence Diagram*) dengan struktur kode yang didefinisikan pada desain statis (*Class Diagram*).

Pada penelitian ini, evaluasi CDR dilakukan menggunakan metode *Cross-Artifact Checking*. Proses ini melibatkan peninjauan terhadap seluruh *Sequence Diagram* (SD-CB-01 s.d. SD-CB-09) untuk memastikan bahwa setiap pesan (*message*) yang dikirimkan antar objek telah didefinisikan sebagai operasi atau metode (*method*) yang valid di dalam kelas-kelas terkait pada *Class Diagram*. Langkah ini dilakukan untuk menjamin tidak ada celah logika (*logic gaps*) dan memastikan seluruh fungsionalitas yang dirancang dapat diimplementasikan secara teknis. Hasil pemeriksaan keterlacakan antara *Use case*, *Sequence Diagram*, serta alokasi *Class* dan *Method* dirangkum dalam Tabel 4.15.

Tabel 4. 15 Hasil *Cross-Artifact Checking* Tahap CDR

No	*Use case*	*Sequence* Diagram	Class Diagram	Method / Operasi
1	UC-CB-01 Autentikasi Pengguna	SD-CB-01 Autentikasi Pengguna	*AuthController, AuthService, UserRepository*	*login*(), verify_pw(), by_email()
2	UC-CB-02 Mengirim Pesan *Chatbot* dan Menerima Jawaban	SD-CB-02 Mengirim Pesan *Chatbot*	*ChatController, RasaClient, RAGClient, ConversationRepository, MessageRepository*	send_*chat*(), generate_reply(), ask(), add_message()
3	UC-CB-03 Melihat Riwayat Pesan	SD-CB-03 Melihat Riwayat Pesan	*ChatController, ConversationRepository, MessageRepository*	get_conversation(), list_user_conversations(), get_messages(), list_message_tail()
4	UC-CB-04 Melihat *Dashboard* Analitik	SD-CB-04 Melihat *Dashboard* Analitik	*ChatController, ConversationRepository, MessageRepository*	get_analytics(), count_conversations(), count_messages()
5	UC-CB-05 Melihat dan	SD-CB-05 Melihat &	*ChatController, MessageRepository*	*chatlog*(), search_*chatlog*()

No	*Use case*	*Sequence* Diagram	Class Diagram	Method / Operasi
	Memfilter *Chat Logs*	Memfilter *Chat Logs*		
6	UC-CB-06 Melihat Daftar Pengguna	SD-CB-06 Melihat Daftar Pengguna	*UserController, UserRepository*	get_users(), list()
7	UC-CB-07 Menambah Pengguna Baru	SD-CB-07 Menambah Pengguna Baru	*UserController, UserRepository*	create_user(), by_email(), create()
8	UC-CB-08 Memperbarui Pengguna	SD-CB-08 Memperbarui Pengguna	*UserController, UserRepository*	update_user(), by_id(), update()
9	UC-CB-09 Menghapus Pengguna	SD-CB-09 Menghapus Pengguna	*UserController, UserRepository*	delete_user(), by_id(), delete()

Berdasarkan Tabel 4.15, evaluasi *Cross-Artifact Checking* menunjukkan adanya konsistensi penuh antara alur interaksi dan definisi struktur kelas. Pemeriksaan ini memvalidasi tiga aspek fungsionalitas utama sistem. Pertama, pada aspek Autentikasi Pengguna (*Use Case* UC-CB-01), *Sequence Diagram* SD-CB-01 menunjukkan alur *login* pengguna yang melibatkan interaksi antara *AuthController*, *AuthService*, dan *UserRepository*. Alur tersebut telah terpetakan secara langsung ke metode *login*(), *verify_pw*(), dan *by_email*() pada masing-masing kelas. Hal ini memastikan bahwa proses autentikasi pengguna telah memiliki spesifikasi teknis yang lengkap dan konsisten dengan desain kelas yang disusun.

Kedua, pada aspek fungsionalitas inti *chatbot* (*Use Case* UC-CB-02 dan UC-CB-03), hasil pemeriksaan menunjukkan bahwa interaksi pengiriman pesan, mekanisme pemrosesan jawaban menggunakan pendekatan *hybrid* Rasa dan RAG, serta penampilan riwayat percakapan telah terdefinisi dengan baik. Pada SD-CB-02, logika pemanggilan layanan Rasa dan RAG dipetakan ke metode *generate_reply*() dan *ask*() pada kelas RasaClient dan RAGClient. Selain itu, proses penyimpanan dan pengambilan pesan serta

percakapan pada SD-CB-03 telah didukung oleh metode *get_conversation*(), *list_user_conversations*(), *get_messages*(), dan *list_message_tail*() pada lapisan *repository*. Hal ini membuktikan bahwa alur logika *chat*bot telah sepenuhnya terakomodasi dalam desain kelas.

Ketiga, pada aspek fitur administrasi (*Use Case* UC-CB-04 hingga UC-CB-09), seluruh kebutuhan sistem untuk analitik dan pengelolaan data pengguna telah memiliki keterkaitan yang jelas antara *sequence* diagram dan operasi pada *class* diagram. Fitur *dashboard* analitik (SD-CB-04) didukung oleh metode agregasi seperti *get_analytics*(), *count_conversations*(), dan *count_messages*() pada *ConversationRepository* dan *MessageRepository*. Sementara itu, fungsi pengelolaan pengguna yang meliputi penambahan, pembaruan, dan penghapusan data telah dipetakan secara konsisten ke metode *create*(), *update*(), dan *delete*() pada *UserRepository*, serta operasi pendukung seperti *by_email*() dan *by_id*().

Dengan demikian, hasil *Cross-Artifact Checking* pada tahap *Critical Design Review* menunjukkan bahwa seluruh perilaku sistem yang dirancang telah memiliki representasi teknis yang lengkap dan konsisten pada *class diagram*. Tidak ditemukan adanya operasi yang terlewat *(no missing operation)*, sehingga desain sistem dinyatakan matang dan siap untuk dilanjutkan ke tahap *Implementation*.

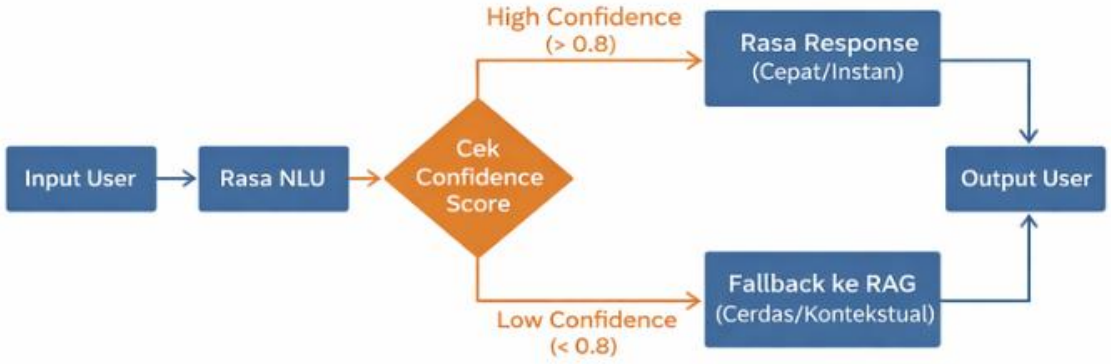
4.9 Tahap *Implementation*

Tahap selanjutnya setelah melakukan tahap desain adalah implementasi. *Class diagram* dan *Sequence Diagram* yang telah dibuat pada tahap sebelumnya diimplementasikan menjadi kode program menggunakan bahasa pemrograman yang telah ditentukan.

Pada tahap ini, rancangan *entity*, *boundary*, dan *Controller* yang dihasilkan dari proses ICONIX direalisasikan ke dalam struktur modul pada sisi *back-end* dan *front-end*. Implementasi dilakukan dengan tetap menjaga kesesuaian antara desain dan kode program, sehingga setiap kelas pada *class diagram* memiliki padanan yang jelas pada *source code* dan dapat ditelusuri kembali (*traceable*) ke kebutuhan fungsional dan *use case* yang telah didefinisikan sebelumnya.

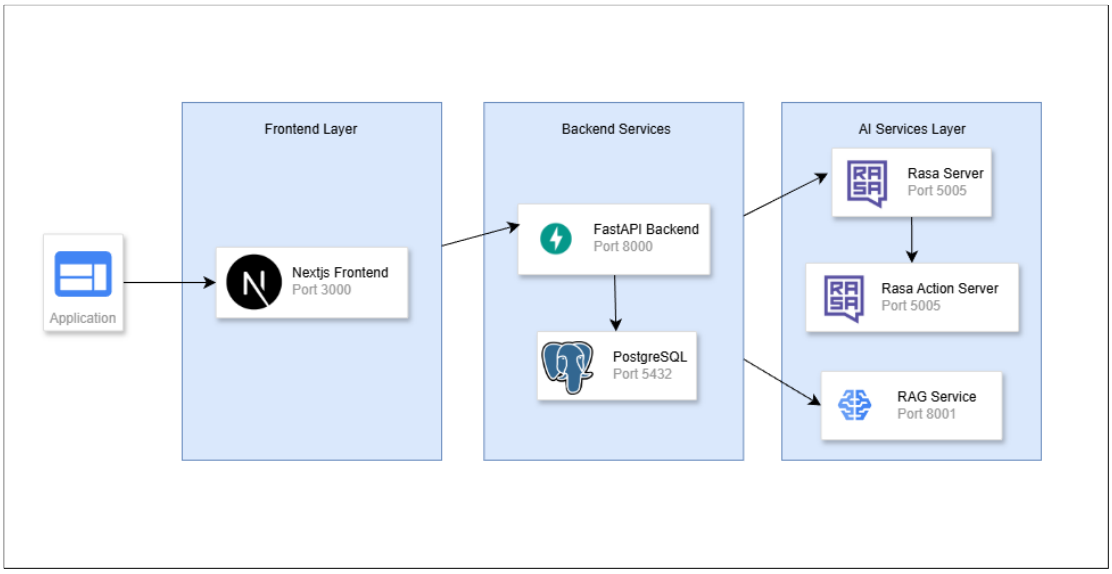
4.9.1 Arsitektur Sistem

Arsitektur sistem pada penelitian ini, seperti yang ditunjukkan pada Gambar 4.33, dirancang untuk mendukung pengembangan *chatbot* layanan akademik berbasis web dengan pendekatan *hybrid* (mengombinasikan Rasa dan RAG).



Gambar 4. 33 *Hybrid* Rasa dan RAG

Secara visual dapat dilihat pada gambar 4.34, arsitektur ini terbagi secara jelas ke dalam tiga lapisan logis utama yang direpresentasikan oleh kotak-kotak biru: *Frontend Layer*, *Backend Services*, dan *AI Services Layer*. Alur sistem dimulai dari blok "*Application*" yang mengarah masuk ke *Frontend Layer*. Di lapisan ini, pengguna berinteraksi melalui antarmuka Next.js *Frontend* yang berjalan pada Port 3000. Pesan dari *frontend* kemudian diteruskan ke lapisan *Backend Services*, diterima oleh FastAPI *Backend* pada Port 8000 yang berperan sebagai orkestrator pusat. Di dalam lapisan backend ini, FastAPI terhubung langsung dengan PostgreSQL pada Port 5432 untuk menyimpan seluruh data percakapan. Untuk pemrosesan kecerdasan buatan, FastAPI *Backend* mendistribusikan tugas ke *AI Services Layer*. *Backend* mengirimkan pesan ke Rasa Server pada Port 5005 untuk klasifikasi intent dan manajemen dialog; Rasa Server ini selanjutnya memiliki koneksi internal ke Rasa *Action Server* pada Port 5005 saat dibutuhkan eksekusi *custom action*. Selain itu, FastAPI *Backend* juga memiliki jalur koneksi langsung ke RAG *Service* pada *Port* 8001 di lapisan AI yang sama, yang digunakan sebagai mekanisme *fallback* untuk menghasilkan jawaban generatif berbasis pencarian dokumen jika *intent* tidak ditemukan oleh Rasa.



Gambar 4.34 Arsitektur Sistem *Chatbot Hybrid* Berbasis Rasa dan RAG

4.9.2 Lingkungan Implementasi

Spesifikasi perangkat lunak dan perangkat keras yang digunakan dalam pembuatan *Chatbot* Layanan Akademik di Fakultas Sains dan Matematika Universitas Diponegoro adalah :

1. CPU : AMD Ryzen 7 5800U
2. Sistem Operasi : Windows 11
3. Bahasa Pemrograman : *TypeScript, Python*
4. *Framework* : fastAPI(*Back End*), Next.js (*Front End*)
5. DBMS : PostgreSQL (PGAdmin)
6. Tools : VisualStudio*Code* , DBeaver, Postman

4.9.3 Pemetaan Kelas

Sebelum memulai penulisan kode program, dilakukan pemetaan terhadap seluruh kelas yang telah diidentifikasi pada tahap desain. Daftar kelas ini disusun berdasarkan objek-objek (*lifelines*) yang muncul dalam *Sequence Diagram* dan telah didefinisikan strukturnya

dalam *Class Diagram*. Detail pemetaan kelas terhadap kategori *entity, boundary*, dan *Controller* ditunjukkan pada Tabel 4.16.

Tabel 4.16 Pemetaan Kelas

No	Nama Desain Kelas	Tipe Kelas
1	*User*	*Entity*
2	*Conversation*	*Entity*
3	*Message*	*Entity*
4	*UserRepository*	*Control*
5	*ConversationRepository*	*Control*
6	*MessageRepository*	*Control*
7	*AuthService*	*Control*
8	*ConversationService*	*Control*
9	*AnalyticsService*	*Control*
10	*AuthController*	*Control*
11	*ChatController*	*Control*
12	*UserController*	*Control*
13	*RasaClient*	*Boundary*
14	*RAGClient*	*Boundary*
15	*LoginPage*	*Boundary*
16	*ChatPage*	*Boundary*
17	*DashboardPage*	*Boundary*
18	*ChatLogsPage*	*Boundary*
19	*UsersPage*	*Boundary*

4.9.4 Implementasi *Source Code*

Implementasi kode program pada sistem Chatbot FSM menerapkan arsitektur berlapis (*layered architecture*) untuk memisahkan tanggung jawab teknis ke dalam beberapa lapisan utama, yaitu *Controller, Service*, *Repository, Model*, dan *Schema*/DTO. Pemisahan ini bertujuan untuk menjaga modularitas kode, mempermudah pengujian, serta meningkatkan keterbacaan struktur proyek.

Pada sisi server (*back-end*), sistem dibangun menggunakan kerangka kerja FastAPI dengan bahasa pemrograman Python. Lapisan *Controller* direalisasikan dalam bentuk router yang bertugas menangani permintaan HTTP (*request)* dan mengembalikan *respons* (*response*). Lapisan *Service* berisi logika bisnis utama aplikasi, sedangkan lapisan *Repository* berfungsi sebagai jembatan interaksi langsung dengan basis data menggunakan teknik *Object-Relational Mapping* (ORM). Entitas domain yang telah dirancang (*User, Conversation, Message*) diimplementasikan sebagai model basis data pada direktori *models*.

Sementara itu, pada sisi klien (*front-end*), tampilan antarmuka pengguna dikembangkan menggunakan kerangka kerja Next.js (berbasis *React*) dengan bahasa pemrograman *TypeScript*. Halaman-halaman utama seperti *Login*, Percakapan *Chatbot*, *Dashboard* Admin, *Chat Logs*, serta Manajemen Pengguna diimplementasikan sebagai komponen halaman (*pages*) yang berkomunikasi dengan *back-end* melalui protokol REST API.

Hasil implementasi *source code* yang memetakan desain kelas terhadap file implementasi ditunjukkan pada Tabel 4.17.

Tabel 4.17 Implementasi *Source Code*

No	Nama Desain Kelas	Tipe Kelas	Nama File / Path Implementasi
1	*User*	*Entity*	*backend/app/models/user.py*
2	*Conversation*	*Entity*	*backend/app/models/chat.py*
3	*Message*	*Entity*	*backend/app/models/chat.py*
4	*UserRepository*	*Repository*	*backend/app/repositories/user_repo.py*
5	*ConversationRepository*	*Repository*	*backend/app/repositories/conversation_repo.py*
6	*MessageRepository*	*Repository*	*backend/app/repositories/message_repo.py*
7	*AuthService*	*Service*	*backend/app/services/auth_service.py*
8	*ConversationService*	*Service*	*backend/app/services/conversation_service.py*
9	*AnalyticsService*	*Service*	*backend/app/services/analytics_service.py*
10	*AuthController*	*Controller*	*backend/app/controllers/auth_controller.py*
11	*ChatController*	*Controller*	*backend/app/controllers/chat_controller.py*
12	*UserController*	*Controller*	*backend/app/controllers/user_controller.py*
13	*RasaClient*	*Client*	*backend/app/clients/rasa_client.py*
14	*RAGClient*	*Client*	*backend/app/clients/rag_client.py*
15	*LoginPage*	*Boundary*	*frontend/app/(auth)/login/page.tsx*
16	*ChatPage*	*Boundary*	*frontend/app/(chat)/page.tsx*
17	*DashboardPage*	*Boundary*	*frontend/app/admin/dashboard/page.tsx*
18	*ChatLogsPage*	*Boundary*	*frontend/app/admin/chat-logs/page.tsx*
19	*UsersPage*	*Boundary*	*frontend/app/admin/users/page.tsx*

Pengaturan parameter juga dilakukan berdasarkan analisis percobaan beberapa skenario dibawah ini dengan mencoba beberapa variasi *threshold* yang menunjukkan keseimbangan penggunaan model pada sistem yang ditunjukkan pada tabel 4.18.

Tabel 4. 18 Percobaan Variasi *Threshold* pada Beberapa Skenario

No	Pertanyaan	Threshold	Jawaban Riil	Model
1	Gimana caranya dapat surat rekomendasi beasiswa? (Pertanyaan Normal)	0.7	Rekomendasi Beasiswa (±3 hari 45 menit): 1) Isi formulir rekomendasi → ke Subbag Akademik 2) BAK teliti & verifikasi 3) BAK beri paraf & nomor surat 4) Wadek tanda tangan 5) Ambil surat di BAK.	Rasa
		0.8	Rekomendasi Beasiswa (±3 hari 45 menit): 1) Isi formulir rekomendasi → ke Subbag Akademik 2) BAK teliti & verifikasi 3) BAK beri paraf & nomor surat 4) Wadek tanda tangan 5) Ambil surat di BAK.	Rasa

No	Pertanyaan	Threshold	Jawaban Riil	Model
		0.9	Halo! 👋 Tentu, saya siap bantu. Untuk mendapatkan surat rekomendasi beasiswa, kamu bisa mengikuti langkah-langkah berikut ini: • • Unduh & Isi Form: Link: **Link unduh formulir:** https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing • Verifikasi BAK: Verifikasi surat pengajuan (1 hari). • Paraf & Nomor: Pemberian nomor surat (10 menit). • TTD Wadek: Penandatanganan (2 hari). • Pengambilan: Ambil di BAK (5 menit).	RAG
2	"cara mnta izin aktif abis cuti gmn y?" (Typo & SPOK Tidak Lengkap)	0.7	Halo! 👋 Tentu, SIMA-BOT siap membantu. Untuk mengajukan izin aktif kembali setelah cuti akademik, kamu bisa mengikuti langkah-langkah berikut ini: • Melapor ke Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan: • Kamu perlu membawa surat izin cuti akademik semester yang sudah kamu miliki. • Jangan lupa siapkan juga fotokopi Kartu Tanda Mahasiswa (KTM) kamu ya. • Proses ini biasanya memakan waktu sekitar 5 menit. • Registrasi Online melalui SSO: • Setelah melapor, kamu bisa langsung melakukan registrasi online melalui SSO (Single Sign-On) masing-masing mahasiswa.	RAG

No	Pertanyaan	Threshold	Jawaban Riil	Model
			• Langkah ini juga diperkirakan memakan waktu sekitar 5 menit. • Status Mahasiswa Aktif: • Setelah semua langkah di atas selesai, kamu akan terdaftar kembali sebagai peserta kuliah atau mahasiswa aktif di Fakultas. • Proses ini juga diperkirakan memakan waktu sekitar 5 menit. Semoga informasinya membantu ya! Jika ada pertanyaan lain, jangan ragu untuk bertanya. 😊 ⏱️ *Waktu respons: 4.28 detik*	
		0.8	Halo! 👋 Tentu saja, SIMA-BOT siap membantu. Untuk prosedur permohonan izin aktif kembali setelah cuti, langkah-langkahnya cukup mudah kok. 😊 Berikut adalah cara mengajukan izin aktif setelah cuti: • Melapor ke Ketua Program Studi dan Subbag Akademik: • Pertama, kamu perlu membawa surat izin cuti akademik semestermu dan melapor kepada Ketua Program Studimu serta Subbag Akademik dan Kemahasiswaan. • Jangan lupa siapkan Surat Izin Cuti dan Fotokopi Kartu Tanda Mahasiswa (KTM)-mu ya! • Proses ini biasanya hanya memakan waktu sekitar 5 menit. • Melakukan Registrasi Online:	RAG

No	Pertanyaan	Threshold	Jawaban Riil	Model
			• Setelah itu, kamu bisa langsung melakukan registrasi online melalui SSO (Single Sign On) masing-masing mahasiswa. • Langkah ini juga cepat, sekitar 5 menit saja. • Status Mahasiswa Aktif: • Setelah semua langkah di atas selesai, kamu akan terdaftar kembali sebagai peserta kuliah atau mahasiswa aktif di Fakultas. • Proses ini juga diperkirakan memakan waktu sekitar 5 menit. Semoga informasinya membantu ya! Jika ada pertanyaan lain, jangan sungkan untuk bertanya. 😊 ⏱️ *Waktu respons: 7.74 detik*	
		0.9	Halo! 👋 Tentu saja, SIMA-BOT siap membantu. Untuk prosedur permohonan izin aktif kembali setelah cuti, langkah-langkahnya cukup mudah kok. 😊 Berikut adalah cara mengajukan izin aktif setelah cuti: • Melapor ke Ketua Program Studi dan Subbag Akademik: • Pertama, kamu perlu membawa surat izin cuti akademik semestermu dan melapor kepada Ketua Program Studimu serta Subbag Akademik dan Kemahasiswaan. • Jangan lupa siapkan Surat Izin Cuti dan Fotokopi Kartu Tanda Mahasiswa (KTM)-mu ya!	RAG

No	Pertanyaan	Threshold	Jawaban Riil	Model
			• Proses ini biasanya hanya memakan waktu sekitar 5 menit. • Melakukan Registrasi Online: • Setelah itu, kamu bisa langsung melakukan registrasi online melalui SSO (Single Sign On) masing-masing mahasiswa. • Langkah ini juga cepat, sekitar 5 menit saja. • Status Mahasiswa Aktif: • Setelah semua langkah di atas selesai, kamu akan terdaftar kembali sebagai peserta kuliah atau mahasiswa aktif di Fakultas. • Proses ini juga diperkirakan memakan waktu sekitar 5 menit. Semoga informasinya membantu ya! Jika ada pertanyaan lain, jangan sungkan untuk bertanya. 😊 ⏱️ *Waktu respons: 5.34 detik*	
3	Bagaimana cara untuk melegalisir ijazah?	0.7	Legalisir Ijazah/Transkrip (maks. 3 hari kerja): 1) Serahkan fotokopi + asli ke Subbag Akademik 2) Petugas cek keabsahan & cap 3) Supervisor paraf 4) Dekan/Wadek tanda tangan.	Rasa
		0.8	Legalisir Ijazah/Transkrip (maks. 3 hari kerja): 1) Serahkan fotokopi + asli ke Subbag Akademik 2) Petugas cek keabsahan & cap 3) Supervisor paraf 4) Dekan/Wadek tanda tangan.	Rasa
		0.9	Legalisir Ijazah/Transkrip (maks. 3 hari kerja):	Rasa

No	Pertanyaan	Threshold	Jawaban Riil	Model
			1) Serahkan fotokopi + asli ke Subbag Akademik 2) Petugas cek keabsahan & cap 3) Supervisor paraf 4) Dekan/Wadek tanda tangan.	
4	Dimana saya dapat mengetahui ruang kelas saya?	0.7	Maaf 😔, saya belum menemukan informasi yang sesuai di dokumen kampus.	RAG
		0.8	Maaf 😔, saya belum menemukan informasi yang sesuai di dokumen kampus.	RAG
		0.9	Maaf 😔, saya belum menemukan informasi yang sesuai di dokumen kampus.	RAG

Berdasarkan hasil pengujian pada tabel 4.18 penelitian ini mentapkan *threshold* sebesar 0,8 sebagai parameter optimal dalam mekanisme pergantian sistem *hybrid*. Pemilihan angka 0,8 didasarkan pada beberapa pertimbangan seperti dapat memberikan keseimbangan yang optimal dengan tetap mempertahankan akurasi serta efisiensi sistem. Pada pengujian pertanyaan dengan struktur kalimat normal dan formal (Pertanyaan 1 dan 3), *threshold* 0,8 memungkinkan model Rasa memberikan jawaban yang cepat dan tepat tanpa harus melempar ke RAG. Hal ini penting untuk menjaga *response time* tetap rendah pada pertanyaan-pertanyaan administratif yang bersifat rutin.

Kemudian *threshold* 0,8 juga menunjukkan fleksibilitas yang baik dalam menangani variasi input pengguna dan juga dapat mendeteksi input yang memiliki kesalahan pengetikan atau struktur yang tidak lengkap dengan mengalihkan proses pencarian jawaban ke modul RAG. Dengan demikian, pengguna tetap mendapatkan jawaban yang relevan.

Secara keseluruhan, implementas *threshold* 0,8 memastikan sistem dapat berfungsi secara responsif dalam menjawab kebutuhan administratif dasar, sekaligus tetap cerdas dan teliti dalam memproses informasi akademik yang lebih mendalam melalui integrasi RAG ketika kepercayaan model Rasa berada di bawah batas optimal.

4.9.5 Implementasi *Database*

Bagian ini merupakan hasil dari perancangan data berupa tabel dalam basis data. Berikut adalah tabel hasil implementasi dari rancangan basis data yang digunakan dalam melakukan pengembangan *Chatbot* di FSM.

1. Tabel : *users*
   Primary Key : id
   Foreign Key : -
   Jumlah Field : 7

Informasi detail terkait table *user* dapat dilihat pada tabel 4.19

Tabel 4.19 Implementasi Database *User*

Nama Field	Deskripsi	Tipe Data	Null
id	Identitas unik pengguna (auto increment)	serial4	No
*user*name	Nama pengguna	varchar(100)	No
email	Email pengguna	varchar(120)	No
password_hash	Password yang telah di-hash	text	No
role	Peran pengguna (admin/mahasiswa)	public."*role*" enum	No
avatar	Foto profil pengguna	varchar(250)	Yes
*create*d_at	Waktu pembuatan data	timestamp	No

2. Tabel : *Conversations*
   Primary Key : id
   Foreign Key : *user*_id
   Jumlah Field : 8

Informasi detail terkait table *Conversations* dapat dilihat pada tabel 4.20

Tabel 4.20 Implementasi Database *Conversations*

Nama Field	Deskripsi	Tipe Data	Null
id	Identitas unik percakapan	serial4	No
*user*_id	ID pengguna pemilik percakapan	int4	Yes
started_at	Waktu percakapan dimulai	timestamptz	No
*update*d_at	Waktu percakapan terakhir diperbarui	timestamptz	No

Nama Field	Deskripsi	Tipe Data	Null
closed_at	Waktu percakapan ditutup	timestamptz	Yes
status	Status percakapan (open/closed/dll)	public.*Conversation* status	No
title	Judul percakapan	varchar(160)	Yes
meta	Informasi tambahan dalam format JSON	json	Yes

3. Tabel : *Messages*
   Primary Key : id
   Foreign Key : conversation_id
   Jumlah Field : 10

Informasi detail terkait table *Messages* dapat dilihat pada tabel 4.21

Tabel 4.21 Implementasi Database *Messages*

Nama Field	Deskripsi	Tipe Data	Null
id	Identitas unik pesan	serial4	No
Conversation _id	ID percakapan tempat pesan berada	int4	No
role	Peran pengirim (*user*/assistant)	public.message*role* enum	No
content	Isi pesan	text	No
*create*d_at	Waktu pesan dibuat	timestamptz	No
source	Sumber jawaban (Rasa/RAG/system)	varchar(16)	Yes
intent	*Intent* hasil prediksi Rasa	varchar(64)	Yes
confidence	*Confidence* score dari *intent*	int4	Yes
latency_ms	Durasi proses (ms)	int4	Yes
meta	Metadata tambahan dalam format JSON	json	Yes

4.9.6 Implementasi Antarmuka

Implementasi antarmuka untuk Sistem *Chatbot* Layanan Akademik berbasis web dikembangkan pada sisi *front-end* menggunakan bahasa pemrograman *TypeScript* dengan *Framework* Next.js. Antarmuka ini dibangun berdasarkan rancangan prototipe yang telah dihasilkan pada tahap requirements, meliputi halaman *login*, halaman percakapan *Chatbot*, halaman *dashboard* admin, halaman manajemen pengguna, serta halaman *chat* logs.

Setiap komponen antarmuka diimplementasikan menggunakan pendekatan component-*based* sesuai standar Next.js, sehingga memudahkan proses pengelolaan tampilan dan pemeliharaan kode. Antarmuka juga dirancang *respons*if agar dapat diakses dengan baik melalui desktop maupun perangkat mobile. Hasil implementasi antarmuka secara lengkap disajikan pada Lampiran 4.

4.9.7 Lingkungan Pengujian

Proses pengujian sistem dilakukan menggunakan perangkat keras dan perangkat lunak yang tersedia selama pengembangan. Spesifikasi perangkat keras yang digunakan dalam pengujian Sistem *Chatbot* Layanan Akademik FSM UNDIP ditampilkan pada Tabel 4.22.

Tabel 4.22 Spesifikasi Perangkat Keras

Komponen	Deskripsi
CPU	AMD Ryzen 7 5800U
RAM	16 GB
Penyimpanan	SSD 512 GB
Sistem Operasi	Windows 11

Sedangkan spesifikasi perangkat lunak yang digunakan pada tahap pengujian ditampilkan pada Tabel 4.24.

Tabel 4.23 Spesifikasi Perangkat Lunak

Nama	Versi / Jenis	Deskripsi
TypeScript	Terbaru	Bahasa pemrograman untuk *front-end* dan beberapa bagian logika aplikasi.

Python	Terbaru	Digunakan untuk pengembangan layanan *back*end dan modul integrasi Rasa/RAG.
FastAPI	Terbaru	*Framework back*end untuk menangani API layanan *Chatbot*.
Next.js	Terbaru	*Framework front-end* untuk membangun antarmuka web *Chatbot*.
PostgreSQL	14+	Sistem manajemen basis data yang digunakan untuk menyimpan data *Chatbot*.
PGAdmin / DBeaver	Terbaru	*Tools* manajemen database untuk administrasi PostgreSQL.
Visual Studio Code	Terbaru	Editor kode utama untuk pengembangan sistem.
Postman	Terbaru	Tools untuk pengujian API.
Google Chrome	Terbaru	Peramban web yang digunakan untuk melakukan pengujian antarmuka.

4.9.8 Rencana Pengujian

Tahap awal dalam pengujian perangkat lunak adalah menyusun rencana pengujian yang mengacu pada kebutuhan fungsional sistem yang telah dirancang sebelumnya. Rencana pengujian mencakup skenario uji, kelas uji, dan butir uji yang disusun untuk memastikan bahwa seluruh fungsi kritis seperti autentikasi, percakapan *Chatbot*, pengelolaan percakapan, *dashboard* analitik, pengelolaan pengguna, dan *chat* logs berjalan sesuai spesifikasi. Dokumentasi lengkap mengenai rencana pengujian ditampilkan pada Lampiran 5.

4.9.9 Pembahasan Hasil Pengujian

Pelaksanaan pengujian fungsional sistem dilakukan dengan melibatkan Tirza Aurellia Wijaya (NIM 24060122130047), mahasiswa Program Studi Informatika, yang bertindak sebagai penguji independen (peer tester) untuk memvalidasi alur kerja aplikasi dari sudut pandang pengguna. Berdasarkan eksekusi terhadap 21 butir uji yang tercantum pada Lampiran 6, diperoleh hasil bahwa seluruh fungsi sistem mendapatkan kesimpulan Diterima karena telah memenuhi kriteria evaluasi yang ditetapkan. Pengujian ini

membuktikan bahwa sistem memiliki keamanan akses yang solid melalui mekanisme *Role-Based Access Control* (RBAC) serta manajemen data yang akurat, baik pada pengelolaan riwayat pesan maupun fungsi administratif administrator. Secara khusus, keberhasilan butir uji DUPL-05 dan DUPL-06 mengonfirmasi keandalan arsitektur *hybrid* chatbot, di mana sistem mampu secara cerdas menentukan sumber jawaban antara model Rasa untuk interaksi umum dan model RAG untuk informasi prosedural akademik yang mendalam berdasarkan dokumen resmi.

BAB V
KESIMPULAN DAN SARAN

Bab ini menyajikan kesimpulan dari seluruh rangkaian penelitian yang telah dilakukan, mulai dari tahap analisis kebutuhan, perancangan, implementasi, hingga pengujian sistem *Chatbot* Layanan Akademik di Fakultas Sains dan Matematika Universitas Diponegoro. Selain itu, bagian ini juga memuat saran-saran yang ditujukan bagi pengembangan sistem selanjutnya agar dapat memberikan manfaat yang lebih luas bagi civitas akademika.

5.1 Kesimpulan

Berdasarkan hasil perancangan, pembangunan, dan pengujian sistem *Chatbot* Layanan Akademik di Fakultas Sains dan Matematika Universitas Diponegoro, dapat disimpulkan beberapa hal sebagai berikut:

1. Penelitian ini berhasil merancang dan membangun sebuah sistem *Chatbot hybrid* berbasis web menggunakan pendekatan *intent classification* menggunakan NLU-Rasa dan LLM-RAG untuk menjawab pertanyaan akademik berdasarkan 7 SOP dokumen di FSM UNDIP. Implementasi mekanisme *fallback* dengan parameter *threshold* yang tepat terbukti mampu menjaga relevansi jawaban baik untuk pertanyaan administratif rutin maupun kueri prosedural yang bersifat mendalam.

2. Penggunaan metodologi ICONIX Process terbukti efektif dalam menjembatani antara kebutuhan fungsional dengan implementasi teknis. Tahapan *Requirement Analysis* hingga *Implementation* memastikan sistem yang dibangun memiliki alur yang jelas, terdokumentasi melalui *use case* dan *sequence diagram*.

3. Pengujian yang dilakukan menunjukkan bahwa seluruh fungsi utama seperti autentikasi pengguna, percakapan *Chatbot*, pengelolaan riwayat percakapan, pengelolaan pengguna oleh admin, *chat logs*, serta *dashboard* analitik dapat sesuai dengan kebutuhan fungsional yang ditetapkan.

5.2 Saran

Berdasarkan hasil pengembangan dan pengujian yang telah dilakukan, terdapat beberapa saran untuk meningkatkan kualitas dan kemampuan sistem di masa mendatang, yaitu:

1. Penyesuaian Model LLM

   Untuk *deployment* pada lingkungan dengan pengguna yang besar disarankan untuk menggunakan LLM versi lokal supaya tidak terkendala dengan biaya token.

2. Peningkatan Manajemen *Knowledge Base*

   Sistem RAG dapat dikembangkan lebih lanjut dengan memperluas sumber pengetahuan, mengefektifkan proses *embedding*, menambahkan versi dokumen historis, serta mengotomatisasi pembaruan *knowledge* untuk menjaga relevansi informasi.

3. Integrasi *Live Chat* dengan Admin

   Sistem dapat dilengkapi fitur *handover* berupa *live chat* dengan admin ketika *Chatbot* tidak dapat menjawab pertanyaan atau *confidence* sangat rendah, sehingga meningkatkan kualitas layanan dan memberikan opsi bantuan langsung kepada mahasiswa.

4. Pengembangan *Mobile Version*

   Sistem dapat diperluas ke *platform mobile* (Android/iOS) agar mahasiswa dapat mengakses layanan secara lebih fleksibel dan nyaman.

DAFTAR PUSTAKA

Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. *Machine Learning with Applications*, *2*, 100006. https://doi.org/10.1016/j.mlwa.2020.100006

Agustha, E. B., Adhy, S., & Nugraheni, D. M. K. (2024). Pengembangan Aplikasi Monitoring Informasi Proyek Berbasis Web Menggunakan Metode ICONIX Process (Studi Kasus: PT Telkom Akses Witel Semarang). *Jurnal Masyarakat Informatika*, *15*(2), 81–96. https://doi.org/10.14710/jmasif.15.2.62416

Anindyati, L. (2023). Analisis dan perancangan aplikasi chatbot menggunakan *Framework* Rasa dan sistem informasi pemeliharaan aplikasi (Studi kasus: Chatbot penerimaan mahasiswa baru Politeknik Astra). *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, *10*, 291–300. https://doi.org/10.25126/jtiik.2022106409

Berryman, John., & Ziegler, Albert. (2025). *Prompt engineering for LLMs : the art and science of building large language model-based applications*. O'Reilly Media, Inc.

Booch, G., Jacobson, I., & Rumbaugh, J. (2005). *The Unified Modeling Language user guide (2nd ed.).* (2 ed.). Pearson Education, Inc.

Desikan, S., & Ramesh, G. (2006). *Software Testing: Principles and Practice.* Pearson Education India.

Elysia, S., & Herianto. (2024). Chatbot Berbasis Retrieval Augmented Generation (RAG) untuk Peningkatan Layanan Informasi Sekolah. *Journal TIFDA (Technology Information and Data Analytic)*, *1*(2), 52–58. https://doi.org/10.70491/tifda.v1i2.52

Farid Faqih, M., Lestari, W., & Muhtarom, M. (2025). Pengembangan Arsitektur Chatbot untuk Layanan Informasi Akademik Berbasis Natural Language Processing (NLP). *Seminar Nasional Teknologi Informasi dan Bisnis (SENATIB)*, 2025. https://doi.org/10.47701/xdfspq49

Fontaine, D. (2021). *The Art of PostgreSQL* (2 ed.). Self-published. https://theartofpostgresql.com/

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3 ed.). Addison-Wesley.

Gourley, D., & Totty, B. (2002). *HTTP The Definitive Guide*. O'Reilly Media, Inc.

Hakim, C., Pornomo, J., & Iman, N. (2024). Evaluasi beban kerja staf administrasi dalam mendukung perbaikan kualitas pelayanan akademik. *Jurnal Teknologi dan Manajemen Industri*, *5*. https://doi.org/https://doi.org/10.26751/jatmi.v5i2.2483

Hidayat, L. R., Pasek, G., Wijaya, S., & Dwiyansaputra, R. (2025). *Optimalisasi Layanan Sistem Informasi Mahasiswa dengan Integrasi Telegram : Chatbot Retrieval-Augmented-Generation berbasis Large Language Model (Optimization of Student Information System Services with Telegram Integration : Chatbot Retrieval-Augmented Generation based on Large Language Model)*. http://jtika.if.unram.ac.id/index.php/JTIKA/

Husain, M. L., Wibisono, Y., & Anisyah, A. (2025). Development of an Academic Services Chatbot Based on Retrieval-Augmented Generation (RAG). *Brilliance: Research of Artificial Intelligence*, *5*(2), 727–735. https://doi.org/10.47709/brilliance.v5i2.6719

Ichsanudin, I., Pratama, R., & Sisephaputra, B. (2024). Pengembangan Sistem Helpdesk Menggunakan Chatbot Dengan Metode Retrieval-Augmented Generation (RAG). *Journal of Informatics and Computer Science*, *06*.

Kite Metric. (2025). *Rasa, LLMs, & RAG_ Building Powerful Conversational AI*. Https://Kitemetric.Com/Blogs/Rasa-Llms-and-Rag-Building-Powerful-Conversational-Ai. https://kitemetric.com/blogs/rasa-llms-and-rag-building-powerful-conversational-ai

Kong, Xiaoquan., & Wang, Guan. (2021). *Conversational AI with Rasa : build, automate, and deploy AI-powered text and voice-based assistants and chatbots*. Packt Publishing.

Leloudas, P. (2023). Introduction to Software Testing: A Practical Guide to Testing, Design, Automation, and Execution. Dalam *Introduction to Software Testing: A Practical Guide to Testing, Design, Automation, and Execution*. Apress Media LLC. https://doi.org/10.1007/978-1-4842-9514-4

Lubanovic, Bill. (2024). *FastAPI : modern Python web development*. O'Reilly Media, Inc.

Luo, P. (2025, Agustus 3). *Modern Semantic Chatbot Architecture: A Hybrid Strategy with Rasa and Large Language Models*. https://medium.com/@k4917552/modern-semantic-chatbot-architecture-a-hybrid-strategy-with-rasa-and-large-language-models-d7e91405dbee

Muhammad, R., Ardimansyah, M., & Afini, A. (2022). *Mengembangkan sistem percakapan otomatis berbasis layanan pesan instan*.

Notokoesoemo, B. P. W. (2023). *Sistem chatbot sebagai bantuan layanan informasi akademik menggunakan Framework Rasa Open Source di UPN "Veteran" Jakarta*. https://repository.upnvj.ac.id/25035/

Rasa Technologies. (2026, Januari 22). *RAG in Rasa*.. https://rasa.com/docs/learn/guides/integrating-rag/.

Riva, Michele. (2022). *Real-World Next.js : build scalable, high-performance, and modern web applications using Next.js, the React Framework for production*. Packt Publishing.

Rosenberg, Doug., Stephens, Matt., & Collins-Cope, Mark. (2005). *Agile development with ICONIX process : people, process, and pragmatism*. Apress.

Rosenberg, Doug., Stephens, Matt., & Suscheck, Charles. (2007). *Use case driven object modeling with UML : theory and practice*. Apress.

Rothman, D. (2024). *RAG-Driven Generative AI: Build Custom Retrieval Augmented Generation Pipelines With LlamaIndex, Deep Lake, and Pinecone*.

Ruindungan, D., & Jacobus, A. (2021). Chatbot Development for an Interactive Academic Information Services using the Rasa Open Source *Framework*. *Jurnal Teknik Elektro dan Komputer*. https://ejournal.unsrat.ac.id/index.php/elekdankom

Sugiyono. (2022). *Metode Penelitian Kualitatif*. Alfabeta Bandung.

Susetyo, D. P., Pranajaya, E., Setiawan, T., & Suryana, A. (2022). Kualitas Pelayanan Akademik dan Citra Institusi sebagai Determinan Kepuasan Mahasiswa. *Formosa Journal of Applied Sciences*, *1*(4). https://doi.org/10.55927/fjas.v1i4.1250

Yulianta, & Mursanto, P. (2008). Pengembangan aplikasi web dengan ICONIX Process dan UML studi kasus: Sistem manajemen ISI. *Jurnal Sistem Informasi*, *4*, 115–121.

Zangiabadi, hamid. (2025, Agustus 17). *Rasa, LLMs, and RAG — Powering a Solution for Conversational AI*. https://dev.to/hamid_zangiabadi/rasa-llms-and-rag-powering-a-solution-for-conversational-ai-3a5b

LAMPIRAN-LAMPIRAN

Lampiran 1. Uji Validitas GUI (*Requirements Review*)

HASIL VALIDASI GUI STORYBOARD

Hari/Tanggal: Senin, 10 November 2025

Tempat: Kantor Layanan Akademik FSM UNDIP

Tujuan: Melakukan validasi terhadap rancangan antarmuka sistem SIMA-BOT FSM untuk memastikan kesesuaian fitur dengan kebutuhan pengguna

Stakeholder:

- Umi Arbiati, S.Kom. (Supervisor Subbagian Akademik & Kemahasiswaan FSM UNDIP)

Tabel Validasi

No	Fitur / Halaman	Feedback Stakeholder
1	Halaman Chatbot Utama (Mahasiswa)	- Link dibuat aktif. jika ada pesan yg berisi link. - untuk pertanyaan yang tidak bisa dijawab, bisa dikasih kata maaf dan info kontak akademik.
2	Dashboard Admin & Analytics	tidak ada. sudah sesuai tampilan data dan grafik informatif.

3	Menu Logs Percakapan (Chat Logs)	Chat log bisa pertimbangin untuk nge folder per user.
4	Manajemen Pengguna (User Management)	tidak ada
5	Login & Autentikasi	tidak ada
6	Tambahan / Lain-lain	tidak ada.

Kesimpulan Validasi

Berdasarkan hasil uji dan penilaian antarmuka SIMA-BOT FSM, seluruh komponen GUI dinilai telah berjalan dgn baik.

secara umum tampilan sistem sdh konsisten, responsif, dan mudah dipahami oleh pengguna.

Saran dari stakeholder:

1. Mengaktifkan tautan link pada pesan chatbot agar dpt diklik langsung oleh pengguna"
2. menambahkan pesan khusus untuk pertanyaan yang tdk dapat dijawab oleh PASA maupun RAG
3. Menyusun chat log berdasarkan pengguna untuk pelacakan lebih mudah.

Pewawancara,

Muthia Zhafira Sahnah

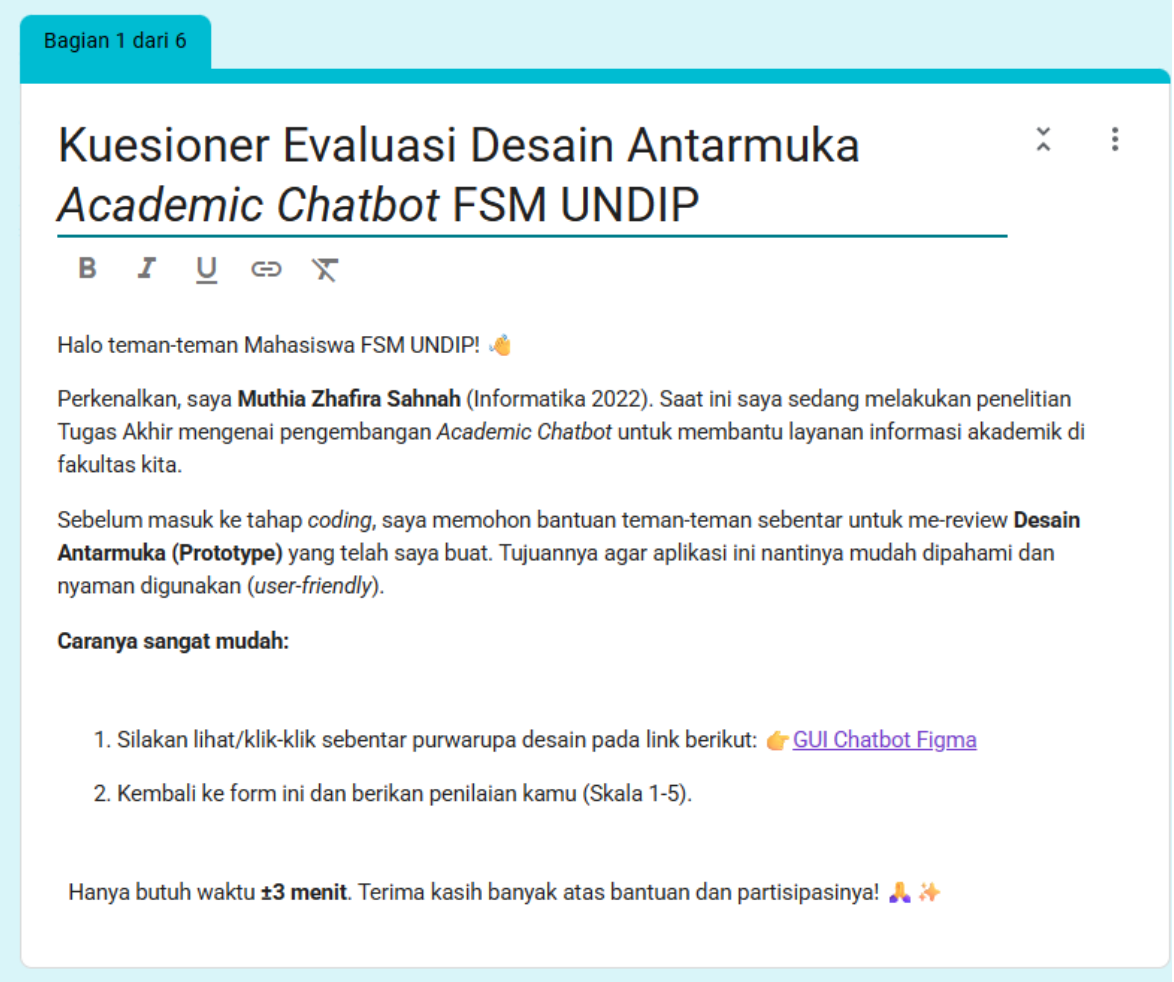
Narasumber,

Umi Arbiati, S.Kom.

Lampiran 2. Dokumentasi *Requirements Review*



Lampiran 3. Validasi GUI dengan Mahasiswa



Bagian 1 dari 6

Kuesioner Evaluasi Desain Antarmuka *Academic Chatbot* FSM UNDIP

Halo teman-teman Mahasiswa FSM UNDIP! 👋

Perkenalkan, saya **Muthia Zhafira Sahnah** (Informatika 2022). Saat ini saya sedang melakukan penelitian Tugas Akhir mengenai pengembangan *Academic Chatbot* untuk membantu layanan informasi akademik di fakultas kita.

Sebelum masuk ke tahap *coding*, saya memohon bantuan teman-teman sebentar untuk me-review **Desain Antarmuka (Prototype)** yang telah saya buat. Tujuannya agar aplikasi ini nantinya mudah dipahami dan nyaman digunakan (*user-friendly*).

Caranya sangat mudah:

1. Silakan lihat/klik-klik sebentar purwarupa desain pada link berikut: 👉GUI Chatbot Figma
2. Kembali ke form ini dan berikan penilaian kamu (Skala 1-5).

Hanya butuh waktu **±3 menit**. Terima kasih banyak atas bantuan dan partisipasinya! 🙏✨

Deskripsi kuisioner dan link prototipe

https://www.figma.com/proto/c8bZSLknlOGXRKRIgMyDw5/Chatbot-FSM?node-id=53-480&p=f&t=8szUHVONfTO9TUHQ-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=53%3A480

Profil Responden

Nama

5 jawaban

- Salsabila Nurivani
- Tirza Aurellia Wijaya
- David
- Hanif Herofa
- Mohammad Izza Hakiki

NIM

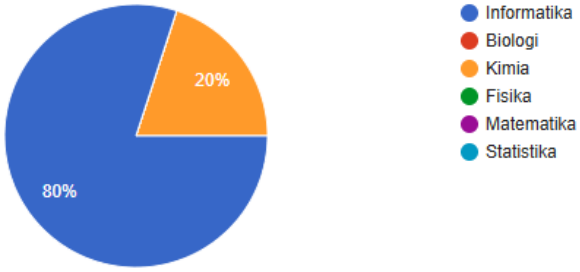
5 jawaban

- 24030121130084
- 24060122130047
- 24060122130074
- 24060122120015
- 24060123140139

Departemen

Salin diagram

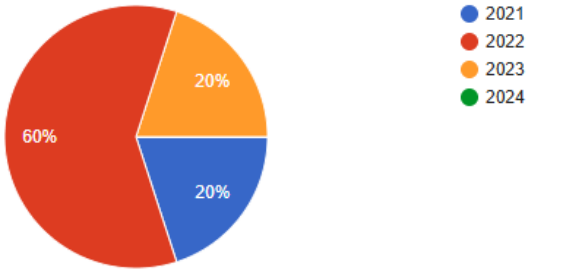
5 jawaban



Angkatan

Salin diagram

5 jawaban

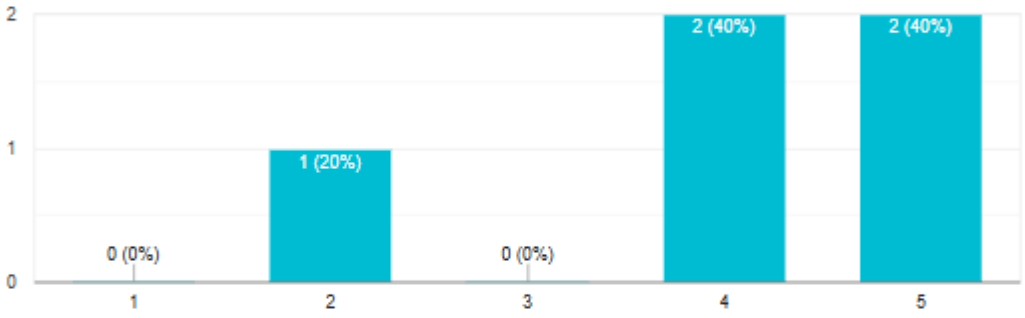


Aspek Tampilan Visual (Visual Design)

Keterbacaan Teks: Ukuran dan jenis huruf yang digunakan pada aplikasi mudah dibaca.

Salin diagram

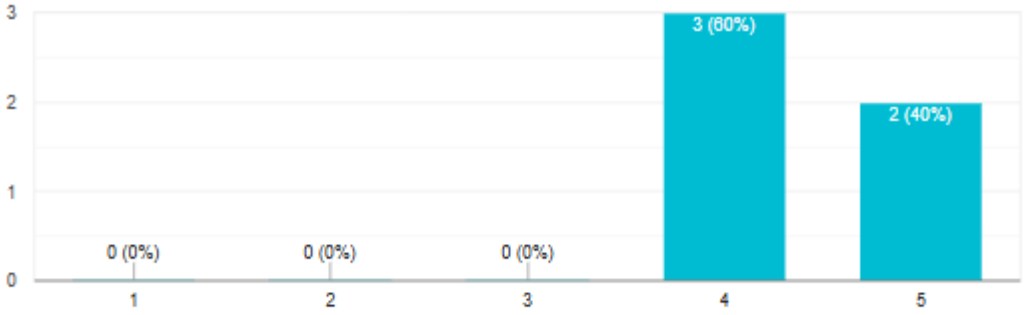
5 jawaban



Tata Letak (Layout): Penempatan elemen utama (kolom chat, *sidebar*, tombol kirim) tertata rapi dan tidak membingungkan.

Salin diagram

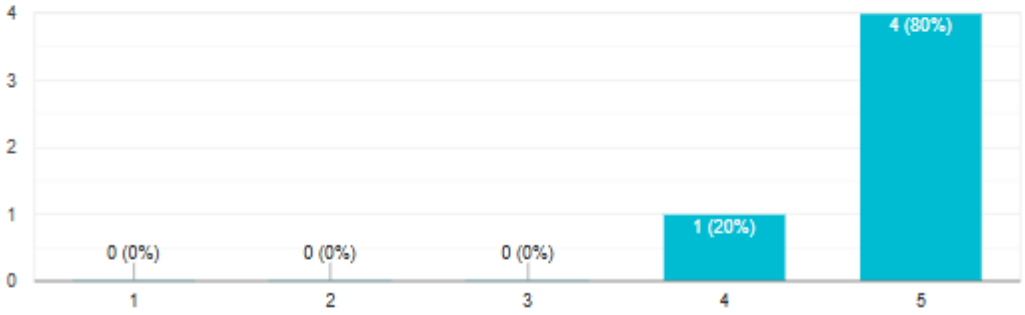
5 jawaban



Desain Ikon: Ikon yang digunakan mudah dipahami maknanya.

Salin diagram

5 jawaban

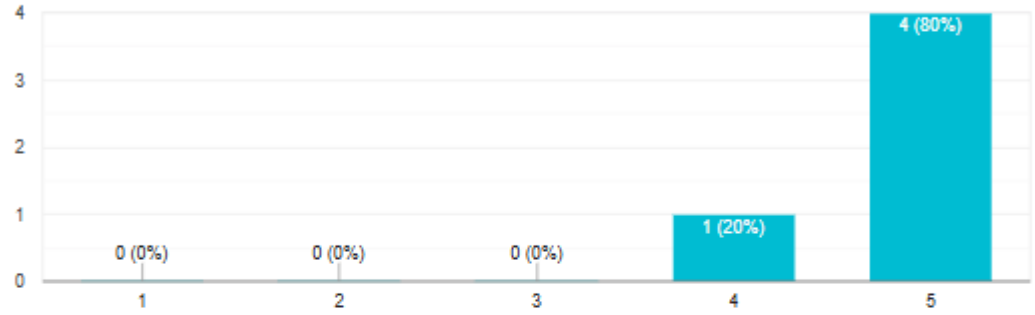


Aspek Kemudahan Penggunaan (Usability)

Navigasi: Saya dapat dengan mudah menemukan menu riwayat percakapan (*history*) pada *sidebar*.

Salin diagram

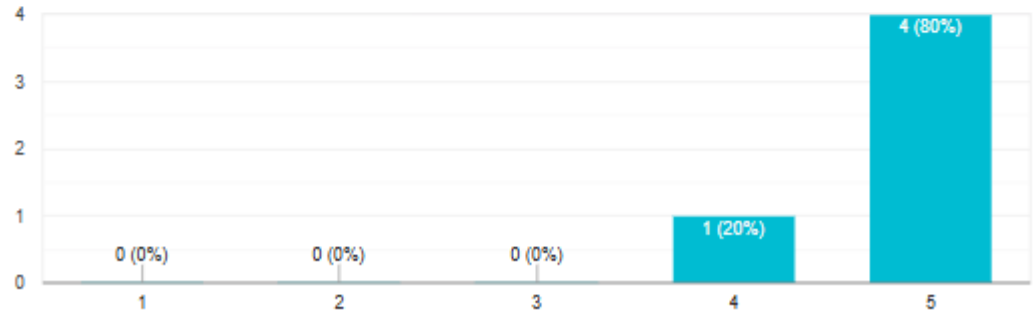
5 jawaban



Form Input: Kolom untuk mengetik pertanyaan mudah ditemukan dan diakses.

Salin diagram

5 jawaban

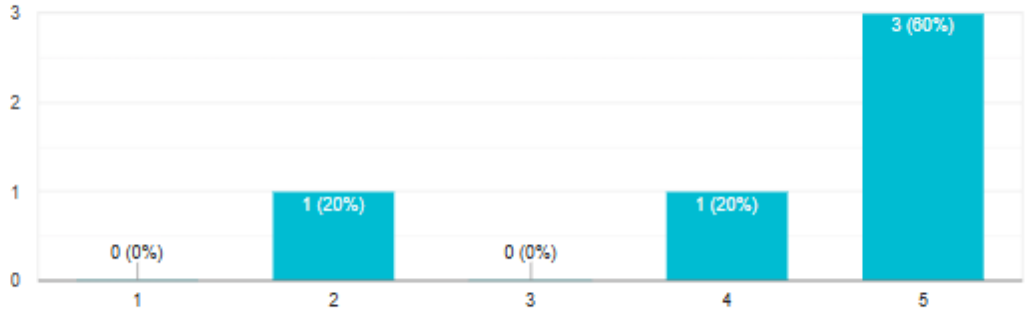


Aspek Pengalaman Pengguna (User Experience)

Kenyamanan: Tampilan aplikasi memberikan kesan bersih (*clean*) dan modern.

Salin diagram

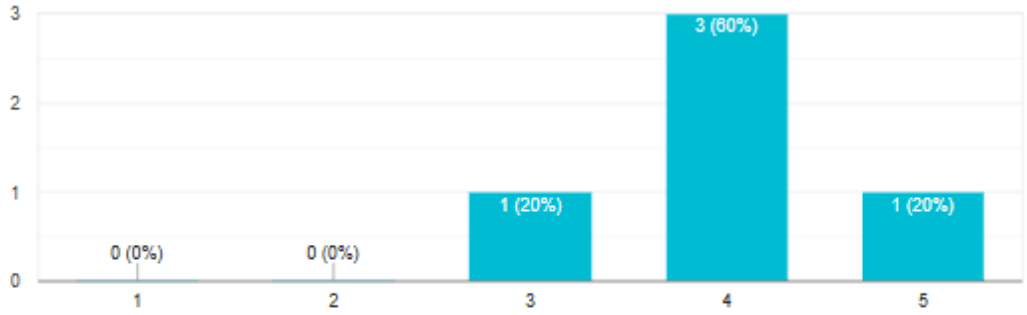
5 jawaban



Kesesuaian: Secara keseluruhan, desain antarmuka ini sudah memenuhi ekspektasi saya untuk sebuah aplikasi layanan akademik.

Salin diagram

5 jawaban



Saran untuk Peningkatan GUI

Saran untuk Peningkatan GUI

4 jawaban

font nya dibuat jarak antar spacingnya lebih besar agar tidak terkesan kecil kecil dan kumpul jadi satu

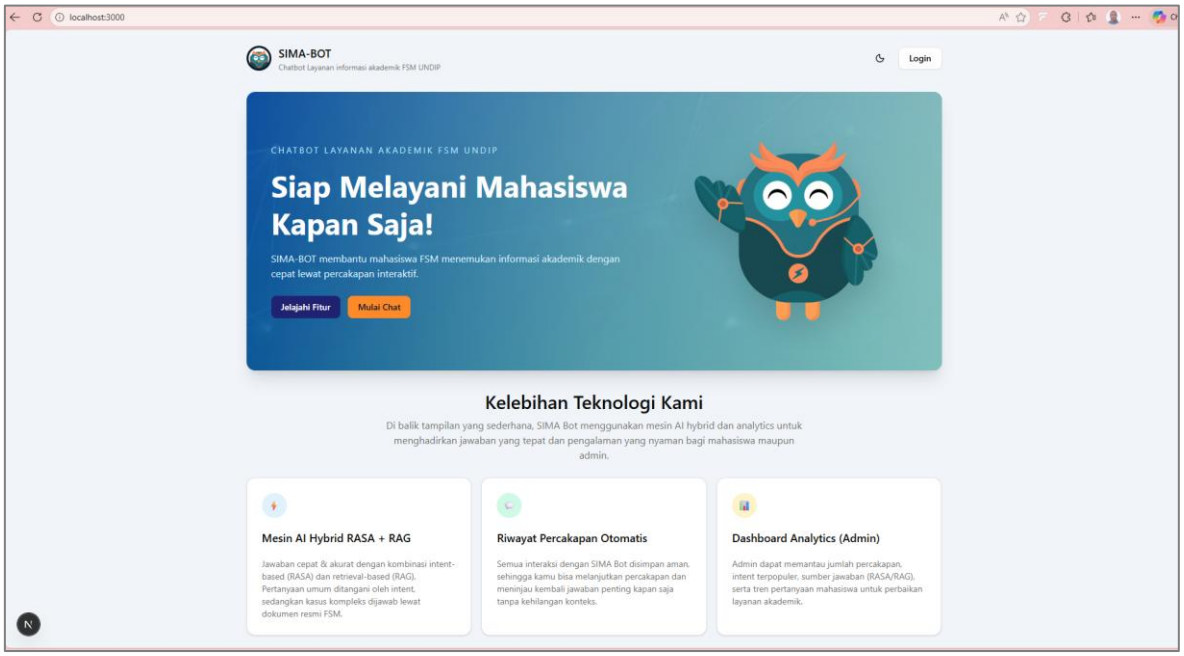
ada pengaturan light mode dan dark mode untuk peningkatan kenyamanan pengguna

beri akses warna lain pada halaman chat agar memperkaya visual

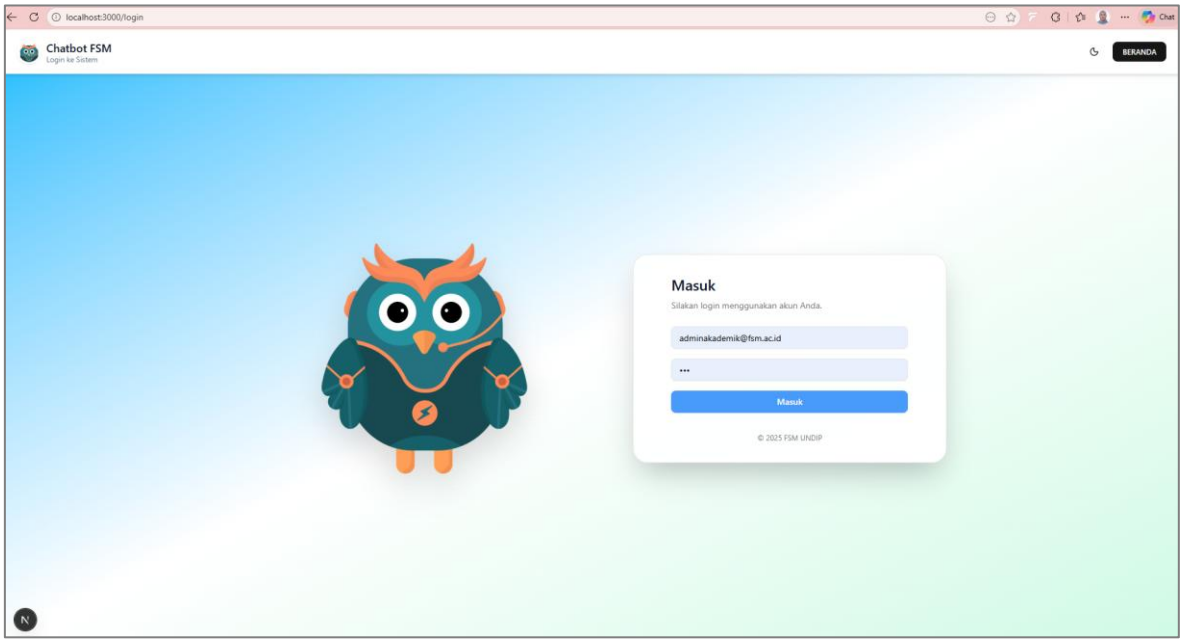
Desain aplikasinya sudah rapi, tapi layout di halaman login terasa terlalu renggang, jadi sebaiknya maskot dan form disatukan dalam satu card saja agar visualnya lebih padat dan fokus. Di bagian styling, teks di dalam bubble chat perlu ditambah padding supaya tidak terasa sesak dan lebih nyaman dibaca. Terakhir, penggunaan palet warna biru, gradien background, hingga spacing antar elemen harus dikunci agar lebih konsisten dan tidak terlalu banyak variasi ukuran yang berbeda.

Lampiran 4. Antarmuka

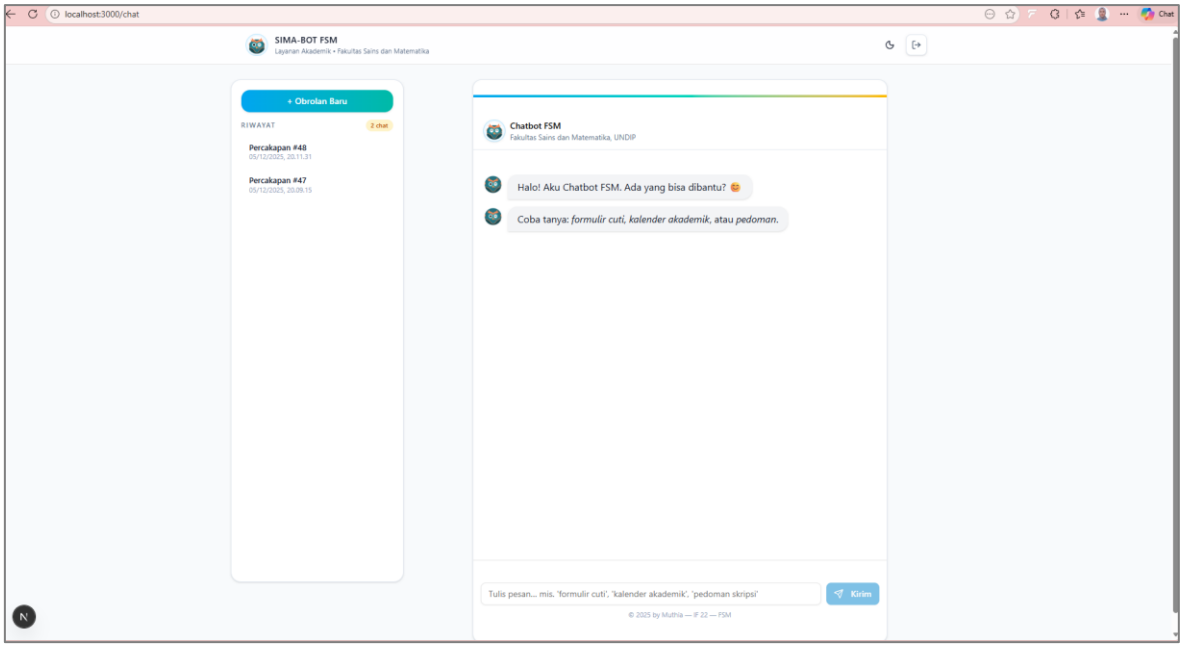
1. Antarmuka Beranda *Chatbot* FSM



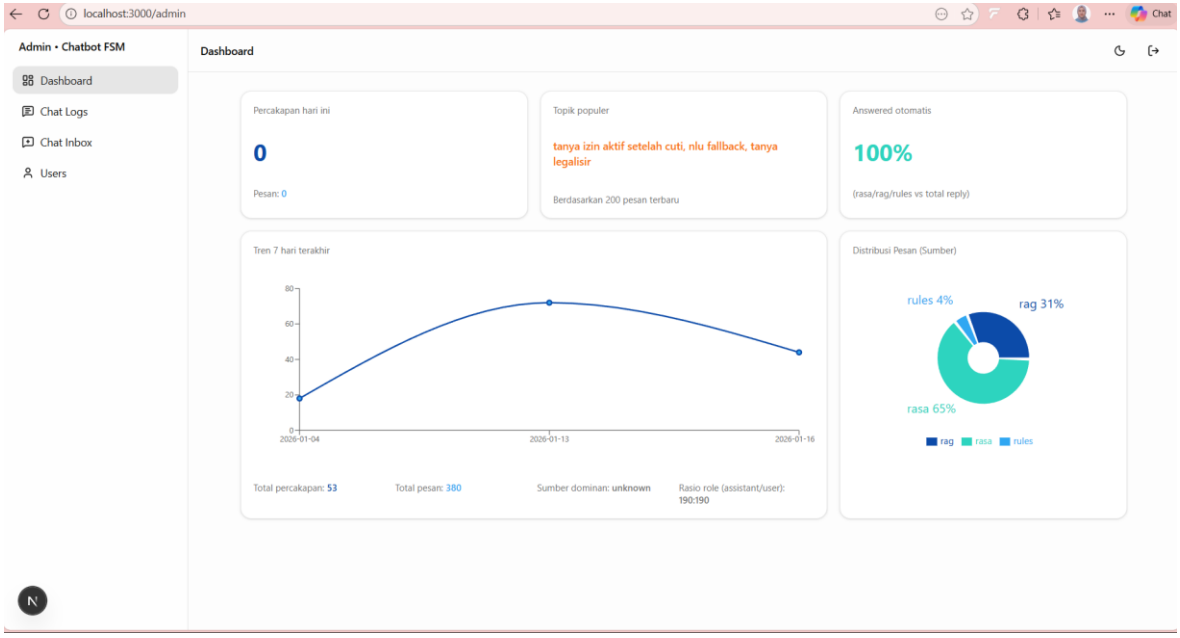
2. Antarmuka *Login Chatbot* FSM



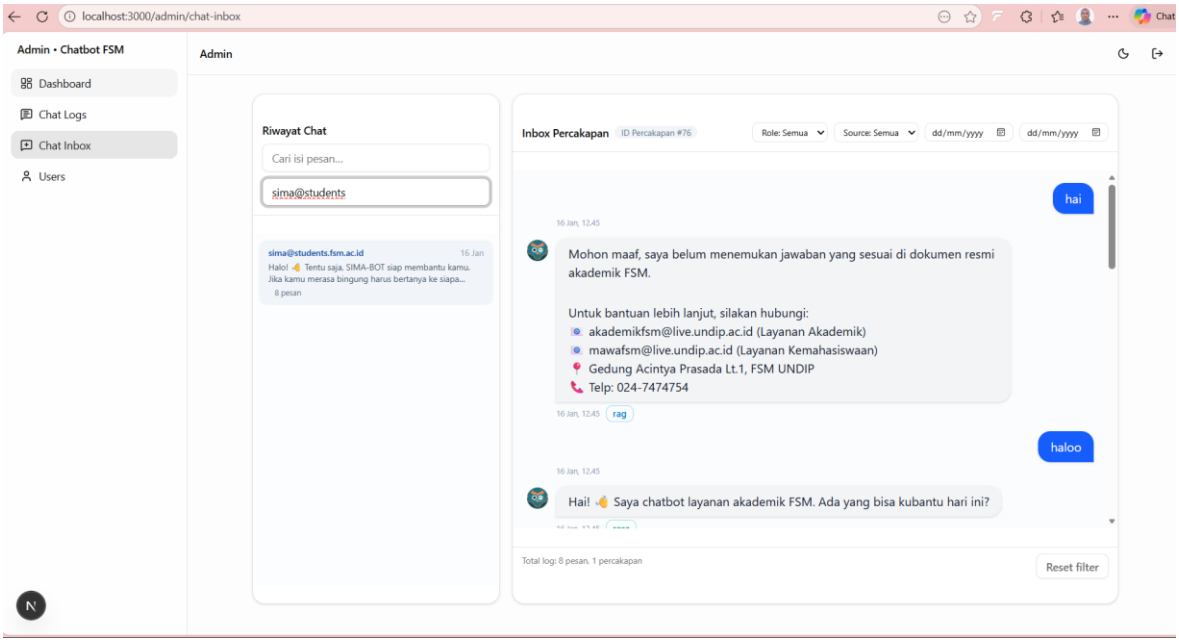
3. Antarmuka Halaman Chat *Chatbot* FSM



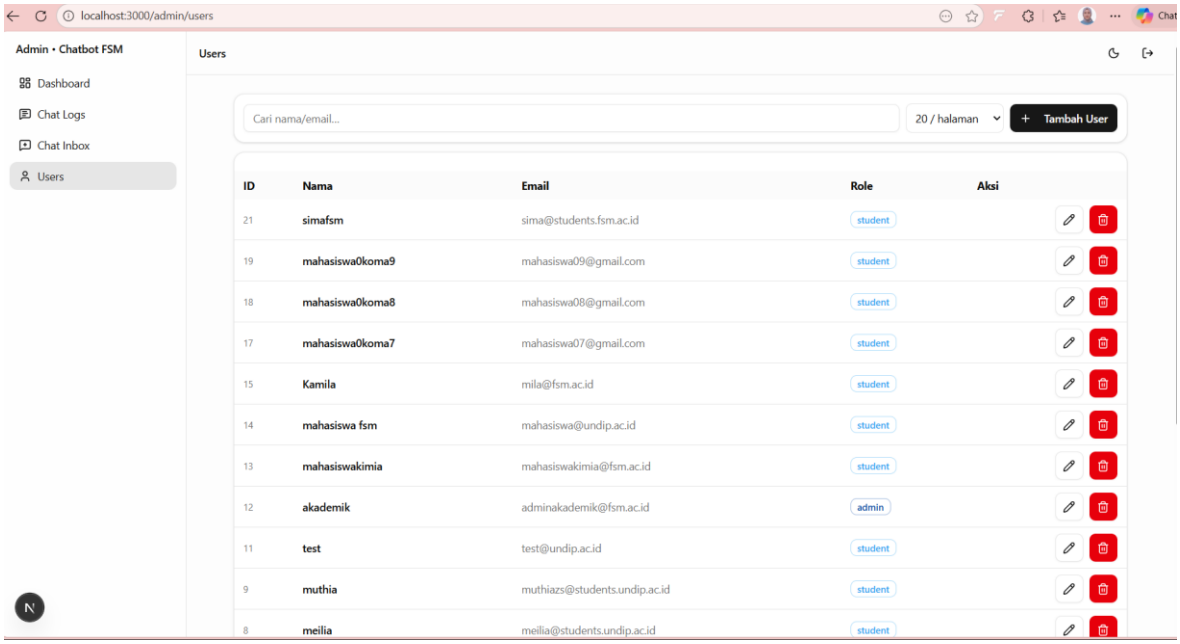
4. Antarmuka Halaman Dashboard Admin *Chatbot* FSM



5. Antarmuka ChatLogs Percakapan



6. Antarmuka Halaman Manajemen *User* FSM



Lampiran 5. Identifikasi dan Rencana Pengujian

KELAS UJI	BUTIR UJI	IDENTIFIKASI (SRS)	DUPL (ID Test Case)	TINGKAT PENGUJIAN	JENIS PENGUJIAN
Autentikasi & Keamanan	Login dengan email dan password valid (Mahasiswa/Admin)	SRS-CB-F-01	DUPL-01	Sistem	Black Box
	Login dengan kredensial tidak valid (Email/Password salah)	SRS-CB-F-01	DUPL-02	Sistem	Black Box
	Verifikasi pengalihan halaman berdasarkan Role (RBAC)	SRS-CB-F-02, SRS-CB-F-03	DUPL-03	Sistem	Black Box
	Logout dari sistem (Sesi berakhir)	SRS-CB-F-01	DUPL-04	Sistem	Black Box
Fungsionalitas Chatbot	Mengirim pesan & menerima balasan via Rasa (Intent dikenal)	SRS-CB-F-04	DUPL-05	Sistem	Black Box
	Mengirim pesan & menerima balasan via RAG (Fallback/Intent tidak dikenal)	SRS-CB-F-04	DUPL-06	Sistem	Black Box
	Penanganan error saat layanan Rasa/RAG tidak tersedia	SRS-CB-F-04	DUPL-07	Sistem	Black Box
	Penyimpanan riwayat pesan & metadata (log sistem)	SRS-CB-F-05	DUPL-08	Sistem	Black Box
Riwayat Percakapan	Melihat daftar riwayat percakapan (List Conversation)	SRS-CB-F-06	DUPL-09	Sistem	Black Box

	Melihat tampilan saat riwayat percakapan kosong	SRS-CB-F-06	DUPL-10	Sistem	Black Box
	Membuka detail riwayat pesan dari ID percakapan valid	SRS-CB-F-07	DUPL-11	Sistem	Black Box
	Akses ID percakapan yang tidak valid (URL manipulation)	SRS-CB-F-07	DUPL-12	Sistem	Black Box
Dashboard Admin	Melihat visualisasi data pada Dashboard Analitik	SRS-CB-F-08	DUPL-13	Sistem	Black Box
	Melihat dan memfilter Chat Logs (berdasarkan kriteria)	SRS-CB-F-09	DUPL-14	Sistem	Black Box
	Pencarian Chat Logs dengan hasil tidak ditemukan	SRS-CB-F-09	DUPL-15	Sistem	Black Box
Manajemen Pengguna (CRUD)	Melihat daftar pengguna terdaftar	SRS-CB-F-10	DUPL-16	Sistem	Black Box
	Menambah pengguna baru dengan data valid	SRS-CB-F-10	DUPL-17	Sistem	Black Box
	Validasi penambahan pengguna dengan email duplikat	SRS-CB-F-10	DUPL-18	Sistem	Black Box
	Memperbarui data pengguna (Edit)	SRS-CB-F-10	DUPL-19	Sistem	Black Box
	Validasi update pengguna dengan format salah	SRS-CB-F-10	DUPL-20	Sistem	Black Box
	Menghapus pengguna dengan konfirmasi	SRS-CB-F-10	DUPL-21	Sistem	Black Box

Lampiran 6. Deskripsi dan Hasil Uji

KODE UJI	DESKRIPSI (Butir Uji)	PROSEDUR PENGUJIAN	MASUKAN	KELUARAN YANG DIHARAPKAN	HASIL PENGUJIAN (OBSERVASI)	KESIMPULAN
DUPL-01	Login dengan email dan password valid	1. Buka halaman login. 2. Masukkan email & password valid. 3. Klik tombol Masuk.	1. Email: mahasiswa@undip.ac.id; Pass: 123456 2. Email: adminakademik@fsm.ac.id; Pass: 123	Sistem menerima kredensial dan mengarahkan ke halaman utama (Chat/Dashboard) sesuai role.	Saat login menggunakan akun mahasiswa, sistem langsung mengarahkan ke halaman chat. Sedangkan saat login menggunakan akun admin, halaman langsung berpindah ke dashboard admin.	**Diterima**
DUPL-02	Login dengan kredensial tidak valid	1. Buka halaman login. 2. Masukkan password yang salah.	Email: mahasiswa@undip.ac.id Pass: salah123	Sistem menolak login dan menampilkan pesan error.	Muncul notifikasi "Email atau password salah" pada layar dan sistem tetap bertahan di halaman login (tidak masuk).	**Diterima**

		3. Klik tombol Masuk.				
DUPL-03	Verifikasi pengalihan halaman berdasarkan Role (RBAC)	1. Login sebagai Mahasiswa. 2. Coba akses URL Dashboard Admin secara manual.	URL: /admin	Sistem menolak akses ke halaman Admin.	Ketika mencoba memaksa masuk ke URL admin saat login sebagai mahasiswa, sistem secara otomatis mengembalikan tampilan ke halaman utama chat.	**Diterima**
DUPL-04	Logout dari sistem	1. Login ke aplikasi. 2. Klik tombol Logout.	Tombol Logout	Sistem menghapus sesi dan kembali ke halaman login.	Setelah tombol logout ditekan, sesi pengguna berakhir dan halaman langsung dialihkan kembali ke form login.	**Diterima**
DUPL-05	Mengirim pesan & menerima balasan via Rasa	1. Masuk halaman chat. 2. Ketik sapaan umum. 3. Kirim pesan.	Pesan: "Halo, selamat pagi"	Chatbot membalas sapaan (Small Talk) sesuai intent Rasa.	Bot membalas dengan kalimat sapaan yang relevan ("Halo! Silakan tanya..."). Terlihat label "source: rasa" di bawah pesan balasan.	**Diterima**

DUPL-06	Mengirim pesan & menerima balasan via RAG	1. Masuk halaman chat. 2. Ketik pertanyaan spesifik tentang SOP. 3. Kirim pesan.	Pesan: "legalisir ijazah gimana sih urusnya"	Chatbot membalas dengan informasi berbasis dokumen (SOP) via RAG.	Bot memberikan jawaban panjang dan rinci mengenai langkah-langkah legalisir sesuai SOP (mulai dari penyerahan berkas hingga stempel). Di akhir pesan terdapat label "source: rag" dan waktu respons.	**Diterima**
DUPL-07	Penanganan error saat layanan tidak tersedia	1. Simulasikan server LLM/Rasa mati. 2. Kirim pesan.	Pesan: "Tes koneksi"	Sistem menampilkan pesan error yang user-friendly.	Saat server dimatikan, bot menampilkan pesan "Maaf, layanan sedang bermasalah" sehingga pengguna tahu terjadi gangguan, bukan *stuck*.	**Diterima**
DUPL-08	Penyimpanan riwayat pesan	1. Kirim sebuah pesan. 2. Cek pada sidebar history percakapan.	Tambahkan percakapan baru dan coba buka percakapan yang lama	Data pesan tersimpan lengkap.	Segera setelah pesan dikirim, judul percakapan baru muncul di sidebar. Data tersimpan di database dan bisa diakses kembali.	**Diterima**

DUPL-09	Melihat daftar riwayat percakapan	1. Login user lama. 2. Lihat sidebar kiri.	Akun user yang memiliki history chat	Daftar ID percakapan sebelumnya muncul di sidebar.	Saat login dengan akun lama, seluruh daftar riwayat percakapan sebelumnya berhasil dimuat di sidebar dan dapat diklik.	**Diterima**
DUPL-10	Melihat tampilan riwayat kosong	1. Login user baru. 2. Lihat sidebar kiri.	Akun user baru (fresh)	Sidebar menampilkan status kosong atau pesan info.	Pada akun yang baru dibuat, area sidebar menampilkan keterangan "Belum ada riwayat" dan area chat masih bersih.	**Diterima**
DUPL-11	Membuka detail riwayat pesan valid	1. Klik salah satu item di daftar percakapan.	Klik pada ID Percakapan di Sidebar	Pesan-pesan lama pada percakapan tersebut dimuat kembali.	Ketika salah satu riwayat diklik, seluruh log percakapan (tanya-jawab) yang lama muncul kembali dengan urutan yang benar di area chat.	**Diterima**
DUPL-12	Melihat visualisasi Dashboard Analitik	1. Login Admin. 2. Buka menu Dashboard.	Akun Admin	Grafik statistik (total pesan, intent) tampil.	Dashboard berhasil menampilkan grafik tren, diagram distribusi sumber, dan angka statistik (total	**Diterima**

					pesan/topik) secara visual tanpa error.	
DUPL-13	Melihat dan memfilter Chat Logs	1. Buka Chat Logs. 2. Filter berdasarkan role tertentu.	Filter: role "user"	Tabel hanya menampilkan log dengan intent tersebut.	Fitur filter berfungsi dengan baik; tabel hanya menyisakan baris data log yang memiliki role 'user' sesuai kriteria yang dipilih.	**Diterima**
DUPL-14	Pencarian Chat Logs tidak ditemukan	1. Buka Chat Logs. 2. Filter tanggal masa depan.	Tanggal: 01-01-2030	Tabel menampilkan status data kosong.	Ketika memfilter tanggal yang belum ada datanya, tabel menjadi kosong dan muncul pesan "Tidak ada data" (sistem menangani hasil kosong dengan baik).	**Diterima**
DUPL-15	Melihat daftar pengguna terdaftar	1. Buka menu User Management. 2. Lihat tabel user.	Klik menu User	Daftar semua user beserta rolenya tampil.	Tabel berhasil me-render daftar seluruh pengguna yang ada di database, lengkap dengan informasi Nama, Email, dan Role.	**Diterima**

DUPL-16	Menambah pengguna baru (Valid)	1. Klik Tambah User. 2. Isi form lengkap. 3. Simpan.	Nama: UserTes Email: tes@email.com Role: Mahasiswa	User baru berhasil disimpan dan modal tertutup.	Setelah tombol simpan ditekan, modal tertutup dan data user baru tersebut langsung muncul di baris tabel paling atas/bawah.	**Diterima**
DUPL-17	Validasi email duplikat	1. Klik Tambah User. 2. Isi email yang sudah ada.	Email: admin@fsm.ac.id (Sudah ada)	Sistem menolak penyimpanan.	Sistem memunculkan pesan peringatan error dan menolak menyimpan data saat mencoba mendaftarkan email yang sudah terdaftar.	**Diterima**
DUPL-18	Memperbarui data pengguna (Edit)	1. Pilih user. 2. Edit Nama. 3. Update.	Nama Baru: UserTes Edit	Data nama user berubah.	Setelah melakukan edit nama dan menyimpan, data pada tabel langsung terbarui dengan nama yang baru.	**Diterima**
DUPL-19	Validasi update format salah	1. Pilih user.	Email: bukanemail (tanpa @)	Sistem menolak update.	Sistem menampilkan validasi format email salah dan tombol simpan tidak memproses	**Diterima**

		2. Edit Email dengan format salah. 3. Update.			perubahan sampai format diperbaiki.	
DUPL-20	Menghapus pengguna (Konfirmasi)	1. Klik Hapus pada user. 2. Klik "Ya/Hapus" pada popup.	Tombol Konfirmasi Hapus	Data user terhapus dari sistem.	Setelah konfirmasi "Ya" dipilih, baris data user tersebut hilang dari tabel dan terhapus dari database.	**Diterima**
DUPL-21	Membatalkan penghapusan	1. Klik Hapus pada user. 2. Klik "Batal" pada popup.	Tombol Batal	Data user tidak terhapus.	Saat tombol "Batal" ditekan, popup tertutup dan data user tetap aman (masih ada) di tabel.	**Diterima**

Semarang, 22 Januari 2026

Penguji Sistem,



Tirza Aurellia Wijaya

NIM. 24060122130047